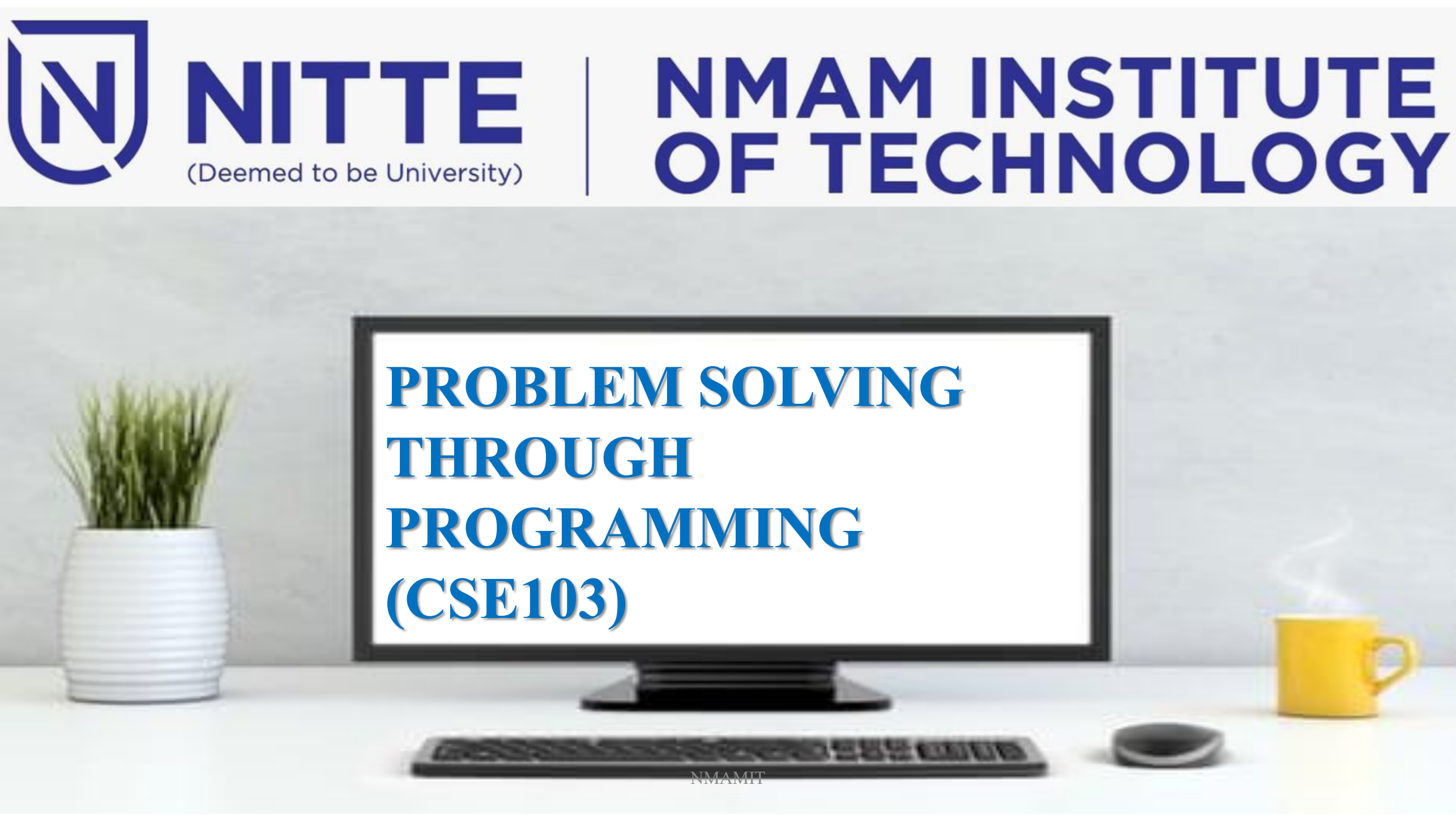




NITTE

(Deemed to be University)

NMAM INSTITUTE OF TECHNOLOGY



PROBLEM SOLVING THROUGH PROGRAMMING (CSE103)

UNIT-3

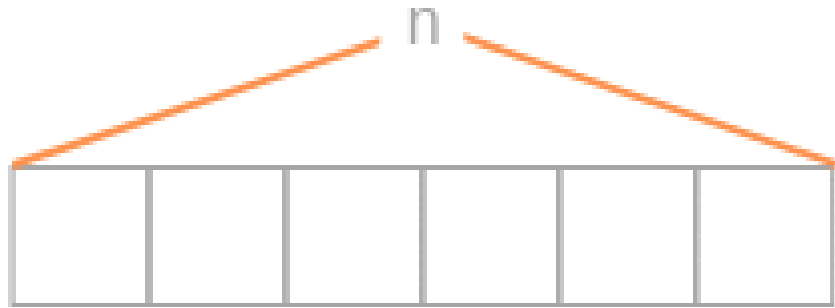
Arrays and Strings

Syllabus:

Arrays: Declaration of arrays, accessing the elements of an array, storing values in arrays, Operations on arrays, Passing arrays to functions, Multidimensional arrays-Two dimensional arrays, operations on two-dimensional arrays, two-dimensional arrays to functions, Example Programs on arrays: linear search, binary search, selection sort, bubble sort.

Strings: Declaring and Initializing strings, String manipulation functions, String input and output functions.

Arrays

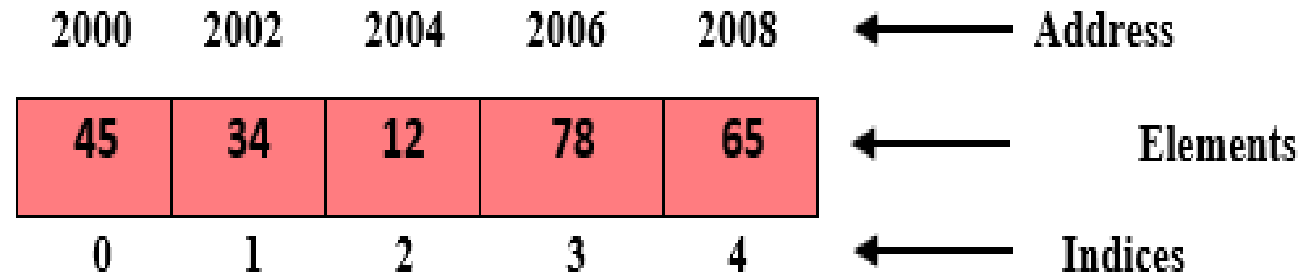


- **Declaration of arrays**
- **Accessing the elements of an array**
- **Storing values in arrays**
- **Operations on arrays**
- **Passing arrays to functions**
- **Two dimensional arrays**
- **Operations on two-dimensional arrays**
- **Two-dimensional arrays to functions**

- An **array** is a collection of **similar data items**.
- Data items are stored at **contiguous memory locations**.
- Data items are also known as **elements** of array.
- Elements of an array is accessed randomly using **indices**.

Array

- They are used to store collection of one primitive data types such as **int, float, char, double**, etc.



Definition of array:

- An array is fixed size sequenced collection of elements of same name and same type.

Types of Arrays

We can implement three types of arrays in C.

1) One dimensional array

2) Two-dimensional array

3) Multidimensional array

- **One dimensional array (1D array)**

- Fixed size sequenced collection of elements of same data type are having

- one subscript (index) is called one dimensional array.

- Example: `int a[5];`

Types of Arrays

- **Two-dimensional array (2D array)**

- Fixed size sequenced collection of elements of same data type are having **two subscripts (indices)** is called two-dimensional array.
- Example: `int a[5][4];`

- **Multi dimensional array**

- Fixed size sequenced collection of elements of same data type are having **more than one subscripts (indices)** is called multi dimensional array.
- Example: `int a[5][4][3];`

Declaration of Arrays: Single dimensional

We use the following general syntax for declaring a single dimensional array:

- **Syntax :**

datatype arrayname[size];

datatype – may be int, float, char, double, etc.

arrayname – is a valid name like identifiers.

size – number of elements store in array.

Example 1: int a[5];

Example 2: float avg[3];

Example 3: Char name[10];

Initialization of One-dimensional Arrays

- Initialization of an array means **storing the elements** in an array.
- Array can be initialized in 2 ways:
 1. At compile time
 2. At run time

1. Compile time initialization

Storing values in an array when they are **declared**.

Syntax :

```
datatype arrayname[size] = { v1, v2, v3,..., vn};
```

Initialization of One-dimensional Arrays

Values are separated by commas.

Example 1: `int a[3] = {10,20,30};`

Example 2: `float b[2] = {1.1,2.2};`

Initialization of One-dimensional Arrays

i)Basic Initialization:

Specifying the **size** of an array and its **values** during declaration.

Example: `int a[5] = {10, 20,30,40,50} ;`

10	20	30	40	50
a[0]	a[1]	a[2]	a[3]	a[4]

Initialization of One-dimensional Arrays

ii) Initialization without size:

If the array is initialized where it is declared, mentioning the size of the array is optional.

Example 1: `int a[ ] = {10, 20, 30, 40, 50} ;`

Example 2: `float b[ ] = {1.1, 2.2} ;`

Example 3: `char name[]={ 'j', 'o', 'h', 'n' };`

10	20	30	40	50
----	----	----	----	----

Initialization of One-dimensional Arrays

iii) Partial Initialization:

Storing a smaller number of elements in an array than its size.

Example 1: `int a[5]= {10,20,30};`

Example 2: `char name[5] = {'a', 'b', 'c'};`

Example 3: `char place[10] = "NITTE";`

10	20	30	0	0
a[0]	a[1]	a[2]	a[3]	a[4]

iv) Null Initialization:

Storing same values in all locations of an array.

Example: `int a[5]={0};`

0	0	0	0	0
a[0]	a[1]	a[2]	a[3]	a[4]

Initialization of One-dimensional Arrays

2. Run time Initialization

- Initialization of an array when program is under execution.

- **Example 1:** `int a[3];`

`a[0]=10;`

`a[1]=20;`

`a[2]=30;`

Using **for loop** to initialize elements of an array
(Reading elements into an array)

- Example 2:** `int a[3];`

`for(i=0; i<3; i++)`

`{`

`scanf("%d", &a[i]);`

`}`

Accessing elements of the Array

- We can access any array element using array name and subscript/index written inside pair of square brackets [].

For Example:

- Suppose we have an integer array of length 5 whose name is marks. **int marks[5] = {5,2,9,1,1};**
- Now we can access elements of array marks using subscript followed by array name.

marks[0] = First element of array marks = 5

marks[1] = Second element of array marks = 2

marks[2] = Third element of array marks = 9

Accessing elements of the Array

marks[3] = Fourth element of array marks = 1

marks[4] = Last element of array marks = 1

Remember array indexing starts from 0. Nth element in array is at index N-1.

Storing Values in Array :

You can also initialize the array during declaration like this:

int arr[5] = {1, 2, 3, 4 ,5}; OR (both are same)

int arr[] = {1, 2, 3, 4, 5}; Un-initialized array always contain garbage values.

Using scanf:

```
for(i=0; i<n; i++)  
    scanf("%d", & a[i]);
```

Operations on Arrays

- Change Value of Array elements:

int mark[5] = {19, 10, 8, 17, 9};

- // make the value of the third element to -1

mark[2] = -1;

- // make the value of the fifth element to 0

mark[4] = 0;

- Arithmetic Operation on array elements.
- Input and Output Array Elements
- Here's how you can take input from the user and store it in an array element.
- // take input and store it in the 3rd element

scanf("%d", &mark[2]);

Operations on Arrays

- `// take input and store it in the ith element`
- `scanf("%d", &mark[i-1]);`
- Here's how you can print an individual element of an array.
- `// print the first element of the array`
- `printf("%d", mark[0]);`
- `// print the third element of the array`
- `printf("%d", mark[2]);`
- `// print ith element of the array`
- `printf("%d", mark[i-1]);`

Write a program to read and print array elements

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int i, n, a[10];
```

```
    printf("Enter the size of the array\n");
```

```
    scanf("%d",&n);
```

```
    printf("Enter array elements\n");
```

```
    //Reading array elements
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        scanf("%d", &a[i]);
```

```
    }
```

```
    printf("Array elements are \n");
```

```
    //Printing array elements
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        printf("%d\t", a[i]);
```

```
    }
```

Output:

Enter the size of the array

3

Enter array elements

10 20 30

Array elements are

10 20 30

Passing 1D Arrays to Functions

Passing arrays to functions simply means **sending a whole list of elements** (instead of sending each element one by one) to another part of the program so that the function can work on them.

- Syntax:

- `void functionName(data type arr[], int size);`

- `void functionName(data type *arr, int size);`

- Arrays are always passed by reference in C.

Example – Passing Integer Array to Function

- `#include <stdio.h>`

```
// Function to display array elements
void displayArray(int arr[], int size) {
    for(int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}
```

```
int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Array elements: ");
    displayArray(numbers, size); // Passing array to function

    return 0;
}
```

- **Output:**
Array elements: 10 20 30 40 50

TWO-DIMENSIONAL ARRAY(2D ARRAY)

- Fixed size sequenced collection of elements of same data type are having **two subscripts (indices)** is called two-dimensional array.
 - It has 2 sets of **[] brackets**.
 - First subscript (index) for **number of rows**.
 - Second subscript (index) for **number of columns**.

	Col0	Col1	Col2
Row0	a[0][0]	a[0][1]	a[0][2]
Row1	a[1][0]	a[1][1]	a[1][2]

- Declaration**

Syntax :

```
datatype arrayname[number of rows][number of columns];
```

datatype – may be int, float, char, double, etc.

arrayname – is a valid name like identifiers.

numbers of rows/number of columns – rows **x** columns = elements in an array.

Example 1: int a[2][3];

Example 2: float avg[3][3];

Example 3: char name[5][20]; -> can store **5 names** with **20 characters** each.

Initialization of Two-dimensional Arrays

- Initialization of an array means **storing the elements** in an array.
- 2D Array can be initialized in 2 ways:
 1. At compile time
 2. At run time

1. Compile time initialization

Storing values in an array when they are **declared**.

Syntax :

```
datatype arrayname[number of rows][number of columns] = { v1, v2, v3,..., vn};
```

Values are separated by commas.

Example 1: `int a[2][3] = {{10,20,30},{40,50,60}};`

Example 2: `int a[2][3] = {10,20,30,40,50,60};`

Initialization of Two-dimensional Arrays

i)Basic Initialization:

Specifying the **number of rows** and **number of columns** of an array and its **values** during declaration.

Example:

```
int a[2][3] = {{10,20,30},{40,50,60}};
```

```
int a[2][3] = {10,20,30,40,50,60};
```

a[0][0]	a[0][1]	a[0][2]
10	20	30
40	50	60

ii)Initialization without size:

While initializing a 2-D array it is necessary to mention the column size , whereas **row size is optional**.

Example:

```
int a[2][3] = {{10,20,30},{40,50,60}};
```

```
int a[ ][3] = {10,20,30,40,50,60};
```

a[0][0]	a[0][1]	a[0][2]
10	20	30
40	50	60
a[1][0]	a[1][1]	a[1][2]

Initialization of Two-dimensional Arrays

iii)Partial Initialization:

Storing a smaller number of elements in an array than its **rows x columns** value.

Example:

```
int a[2][3] = { {10,20,30}, {40,50,60} };
```

OR

```
int a[2][3] = {10,20,30,40,50,60};
```

OR

```
int a[ ][3] = {10,20,30,40};
```

a[0][0]	a[0][1]	a[0][2]
10	20	30
40	0	0
a[1][0]	a[1][1]	a[1][2]

iv)Null Initialization:

Storing same values in all locations of an array.

Example: `int a[2][3]={0};`

a[0][0]	a[0][1]	a[0][2]
0	0	0
0	0	0
a[1][0]	a[1][1]	a[1][2]

Initialization of Two-dimensional Arrays

2. Run time Initialization

Initialization of an array when program is under execution.

Example 1: `int a[2][2];`

`a[0][0]=10;    a[0][1]=20;    a[1][0]=30;    a[1][1]=40;`

Using **for loop** to initialize elements of an array.

(Reading elements into an 2D array)

Example 2: `int i,j,a[10][10];`

```
.....  
.....  
for(i=0; i<2; i++)  
{  
    for(j=0; j<2; j++)  
    {  
        scanf("%d", &a[i][j]);  
    }  
}
```

TRACE

i=0, (0<2)

j=0, (0<2)
a[0][0]=?
j=0+1=1

(1<2)
a[0][1]=?
j=1+1=2

(2<2)
i=0+1= 1

(1<2)

j=0, (0<2)
a[1][0]=?
j=0+1=1

(1<2)
a[1][1]=?
j=1+1=2

(2<2)
i=1+1= 2

(2<2)

Printing Two-dimensional array

Example:

```
int i,j,a[10][10];  
.....  
.....  
for(i=0; i<2; i++)  
{  
    for(j=0; j<2; j++)  
    {  
        printf("%d\t", a[i][j]);  
    }  
    printf("\n");  
}
```

Output:

10	20
30	40

a[0][0]	a[0][1]
10	20
30	40
a[1][0]	a[1][1]

TRACE

i=0, (0<2)

j=0, (0<2)
a[0][0]= 10
j=0+1=1

(1<2)
a[0][1]= 20
j=1+1=2

(2<2)
i=0+1= 1

(1<2)

j=0, (0<2)
a[1][0]= 30
j=0+1=1

(1<2)
a[1][1]= 40
j=1+1=2

(2<2)
i=1+1= 2

(2<2)

Write a program to read and display two-dimensional array or matrix

```
#include<stdio.h>
void main()
{
    int a[10][10],m,n,i,j;
    printf("Enter number of rows and columns\n");
    scanf("%d%d",&m,&n);
    printf("Enter elements of matrix\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}
```

```
printf("matrix is\n");  
    for(i=0;i<m;i++)  
    {  
        for(j=0;j<n;j++)  
        {  
            printf("%d\t",a[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
Enter number of rows and columns  
2 3  
Enter elements of matrix  
1 2 3 4 5 6  
matrix is  
1      2      3  
4      5      6
```

Passing 2D Arrays to Functions

Passing Two-Dimensional Arrays to Functions means sending an entire table (rows and columns of data) from one part of your program to a function, so that the function can process or display it.

Syntax:

```
return_type function_name(data_type array_name[rows][columns]);
```

or (rows can be empty, but columns must be fixed)

```
return_type function_name(data_type array_name[][columns]);
```

Example Syntax

// Function definition

```
void displayMatrix(int arr[][3], int rows, int cols) {  
    // code to process array  
}
```

// Function call

```
int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };  
displayMatrix(matrix, 2, 3);
```

Example Programs on Arrays

Linear Search, Binary Search, Selection Sort, Bubble Sort

Searching

Searching is a process of finding out a particular element in a list of 'n' elements



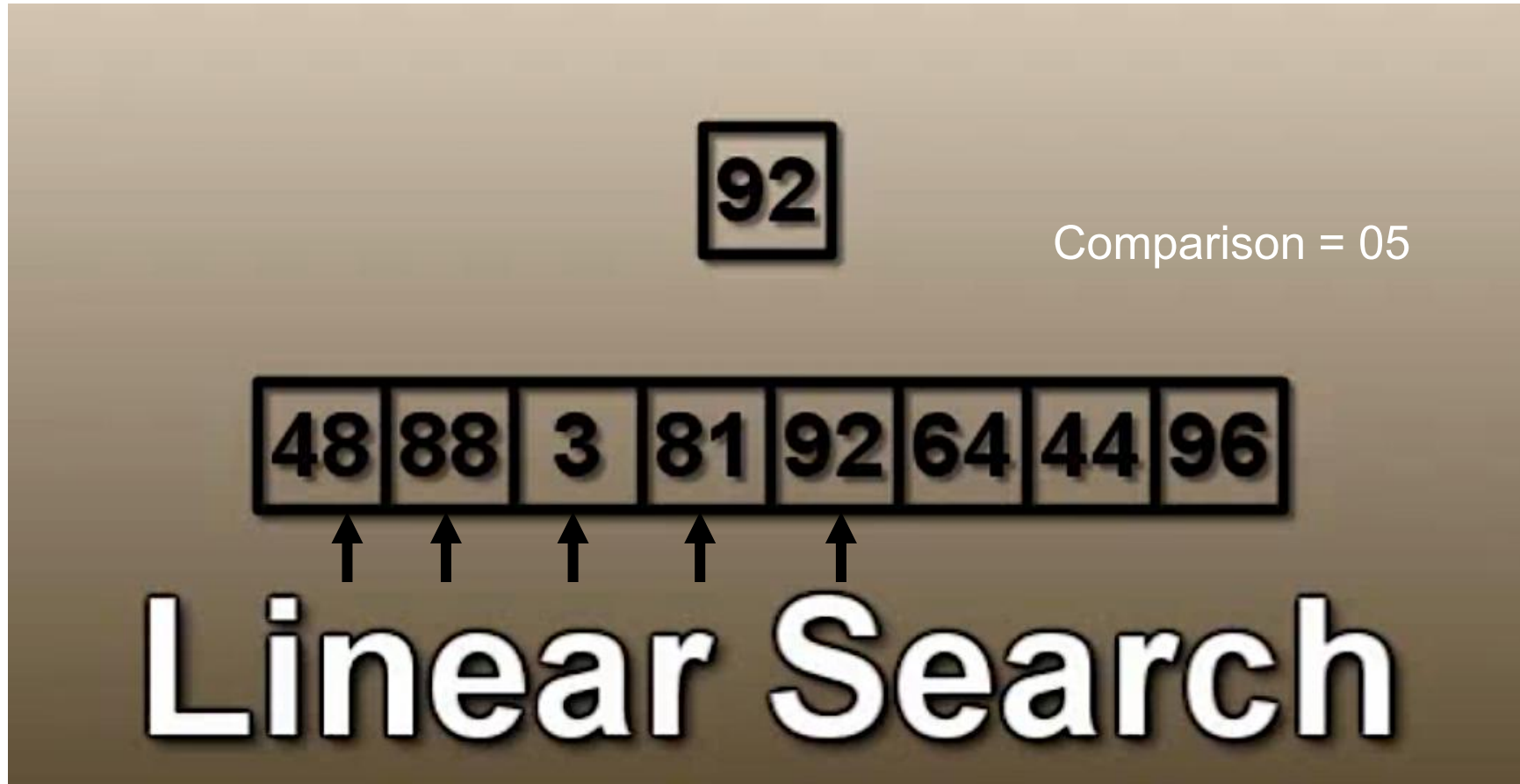
Methods of Searching

1. Linear Search (Sequential Search)
2. Binary Search

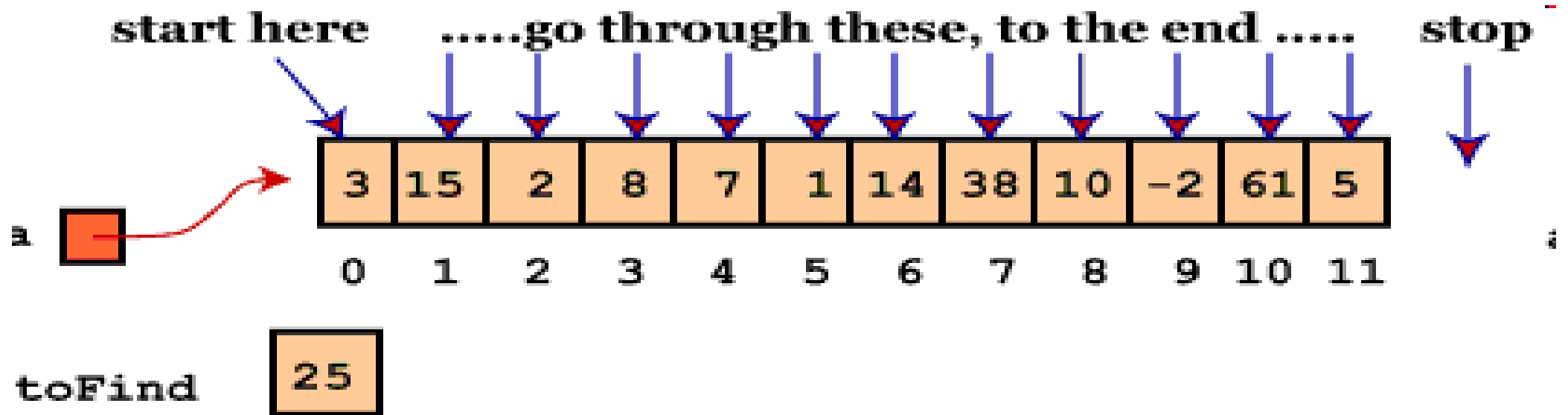
Linear Search(Sequential Search)

- In a linear search, we start at the top of the list, and compare the element there with the key
- If we have a match, the search terminates and the index number is returned

Linear Search



Linear Search



Example: Sequential Search with Array

```
int main()
{
    int A[10], i, n, Key;
    printf("Enter the size of an array: ");
    scanf("%d",&n);
    printf("Enter the elements of the array: ");
    for(i=0; i < n; i++)
        scanf("%d",&A[i]);
    printf("Enter the number to be searched: ");
    scanf("%d",&Key);
    for(i=0;i<n;i++){
        if(a[i] == Key){
            printf("The number is found at index %d",i);
            exit(0);
        }
    }
    printf("The number is not in the list");
    return 0;
}
```

Binary Search

Binary Search

- In a binary search, we look for the **key in the *middle*** of the list
- If we get a **match**, the **search is over**
- If the key is **greater than** the thing in the middle of the list, we **search the top half**
- If the **key is smaller**, we **search the bottom half**

Binary Search

0	1	2	3	4	5	6
6	13	14	25	33	43	51

Beg = 0

End = 6

Mid = $0 + 6/2$

Search Element = 25

If(key==a[mid])

Binary Search

0	1	2	3	4	5	6
6	13	14	25	33	43	51

Beg = 0

End = 6

Mid = $(0+6)/2$

Search Element = 13

If $13 < a[\text{mid}]$ then search in the left side.

Beg = 0

End = mid - 1

Mid = $(0+2)/2$

Binary Search

0	1	2	3	4	5	6
6	13	14	25	33	43	51

Beg = 0

End = 6

Mid = $(0+6)/2$

If key > a[mid]

Beg = mid+1

End = 6

Mid = $(4+6)/2$

Binary Search Program

```
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50}; // Array must be
sorted
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 40;
    int low = 0;
    int high = n - 1;
    int mid;
    int found = 0; // Flag to indicate if key is found
```

Binary Search Program (Conti..)

```
printf("Array elements: ");  
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}  
printf("\n");  
  
printf("Searching for key:  
%d\n", key);
```

```
while (low <= high)  
{  
    mid = low + (high - low) / 2;  
    if (arr[mid] == key)  
    {  
        printf("Element found at  
index:    %d\n", mid);  
        found = 1;  
        break;  
    }  
}
```

Binary Search Program (Conti..)

```
else if (arr[mid] < key)
{
    low = mid + 1;
}
else
{
    high = mid - 1;
}
}
```

```
if (!found) {
    printf("Element not
found in the array.\n");
}

return 0;
}
```

Difference between linear and binary search

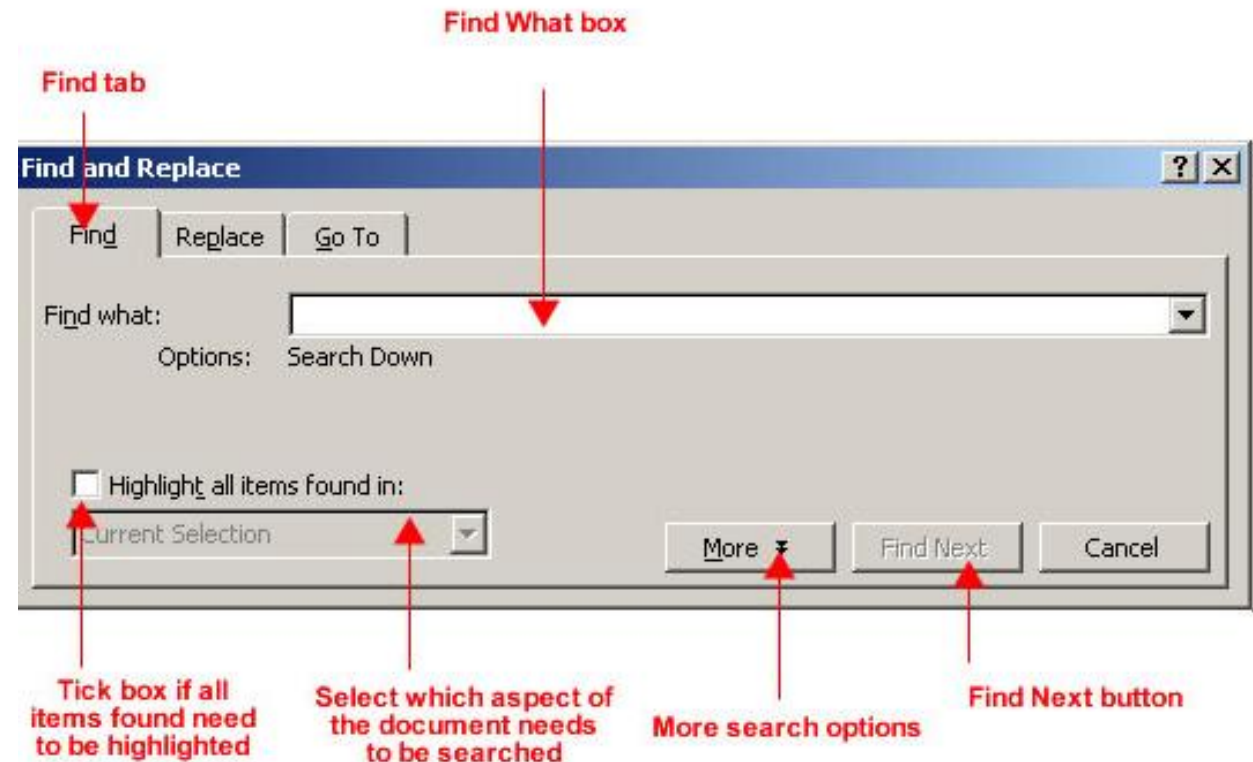
Linear Search

1. Data can be in any order.
2. Multidimensional array also can be used.
3. Time Complexity:- $O(n)$
4. Not an efficient method to be used if there is a large list.

Binary Search

1. Data should be in a sorted order.
2. Only single dimensional array is used.
3. Time Complexity:- $O(\log n)$
4. Efficient for large inputs also.

Application of Searching



Selection Sort

Example

77 , 11 , 33 , 2 , 6 , 1 ,
99

77 , 11 , 33 , 2 , 6 , 1 , 99

77	11	33	2	6	1	99
----	----	----	---	---	---	----

77	11	33	2	6	1	99
----	----	----	---	---	---	----



1	11	33	2	6	77	99
---	----	----	---	---	----	----



1	2	33	11	6	77	99
---	---	----	----	---	----	----



1	2	6	11	33	77	99
---	---	---	----	----	----	----



1	2	6	11	33	77	99
---	---	---	----	----	----	----



1	2	6	11	33	77	99
---	---	---	----	----	----	----



Selection Sort

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------

A[0]

A[1]

A[2]

A[3]

A[4]

A[5]



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
---	---	---	---	---	---



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
---	---	---	---	---	---



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
---	---	---	---	---	---



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
---	---	---	---	---	---



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Largest



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	2	6
---	---	---	---	---	---



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	2	6
---	---	---	---	---	---



Comparison

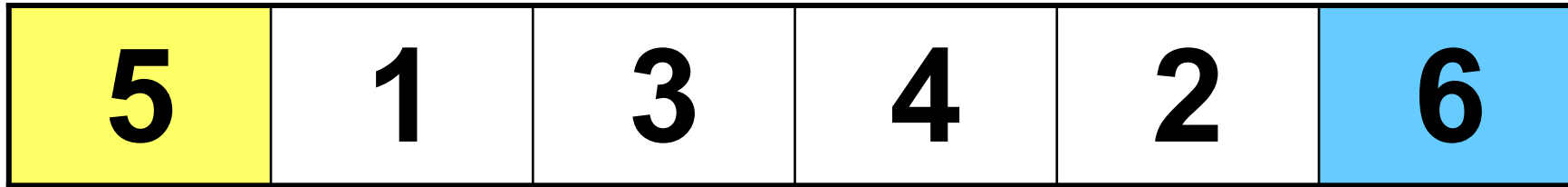


Data Movement



Sorted

Selection Sort



Comparison

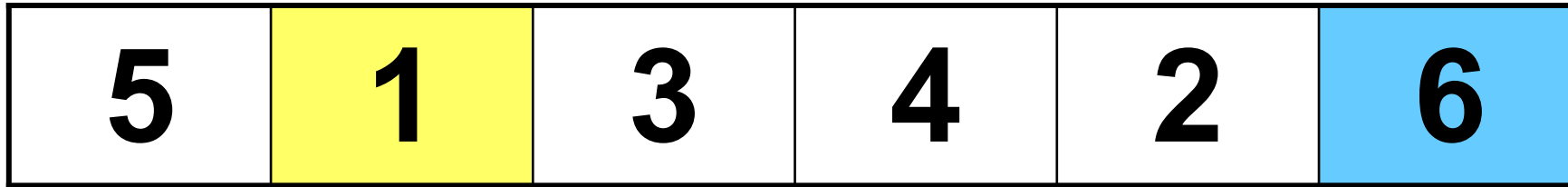


Data Movement



Sorted

Selection Sort



Comparison

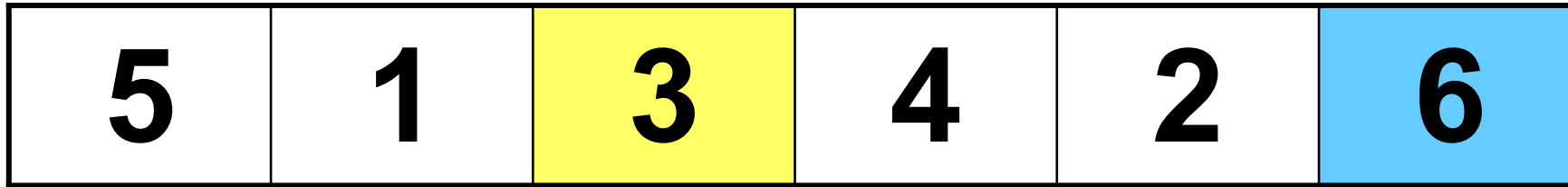


Data Movement



Sorted

Selection Sort



Comparison

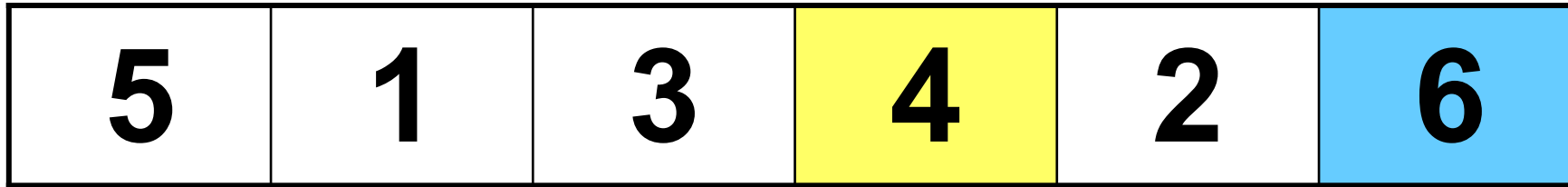


Data Movement



Sorted

Selection Sort



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	2	6
---	---	---	---	---	---



Comparison

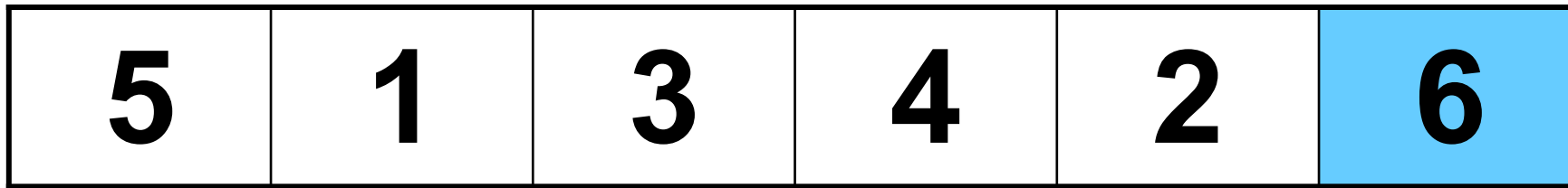


Data Movement



Sorted

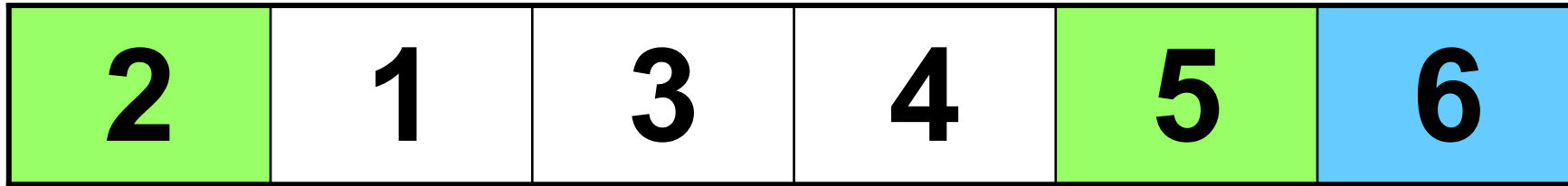
Selection Sort



↑
Largest

-  Comparison
-  Data Movement
-  Sorted

Selection Sort



-  Comparison
-  Data Movement
-  Sorted

Selection Sort

2	1	3	4	5	6
---	---	---	---	---	---



Comparison

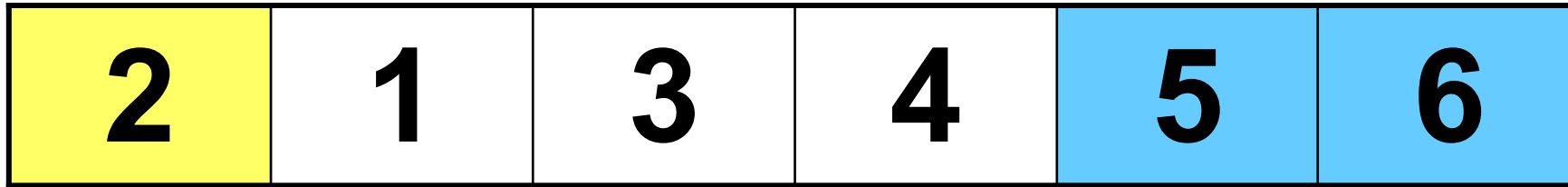


Data Movement



Sorted

Selection Sort



Comparison

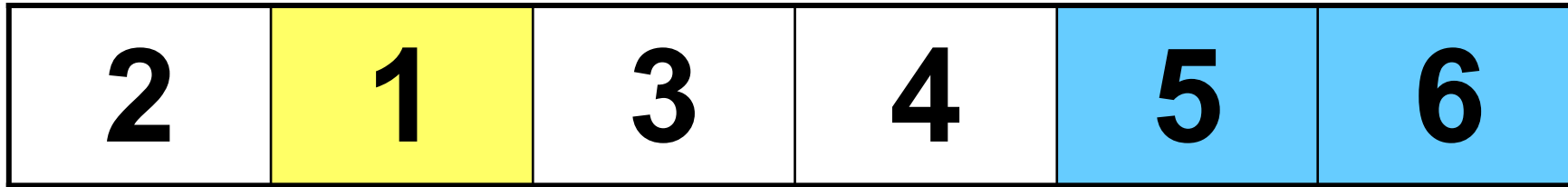


Data Movement



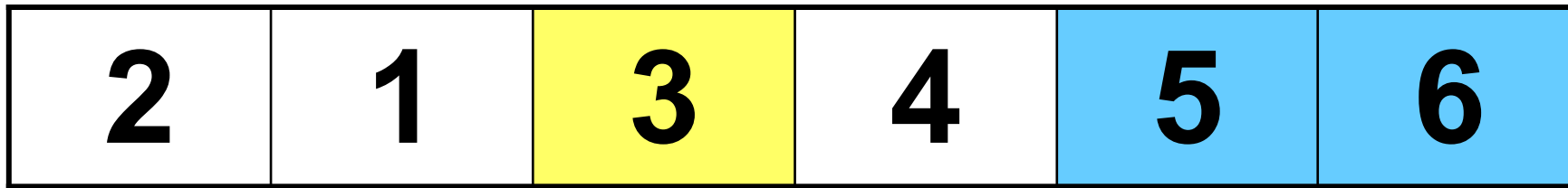
Sorted

Selection Sort



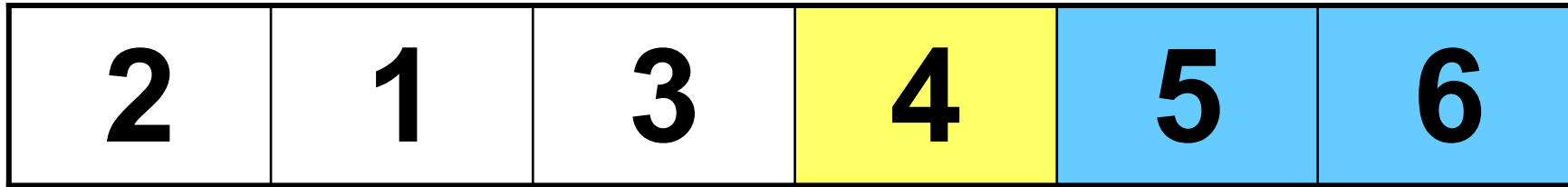
-  Comparison
-  Data Movement
-  Sorted

Selection Sort



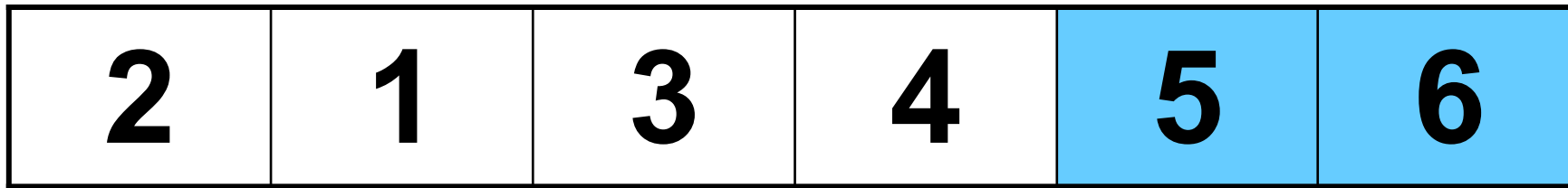
-  Comparison
-  Data Movement
-  Sorted

Selection Sort



-  Comparison
-  Data Movement
-  Sorted

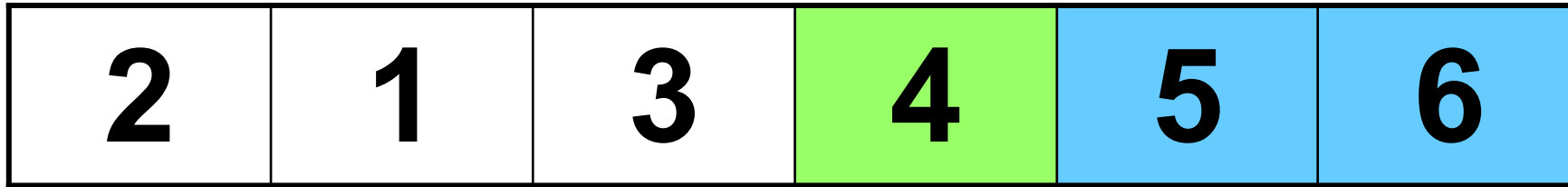
Selection Sort



↑
Largest

-  Comparison
-  Data Movement
-  Sorted

Selection Sort



Comparison



Data Movement



Sorted

Selection Sort

2	1	3	4	5	6
---	---	---	---	---	---



Comparison

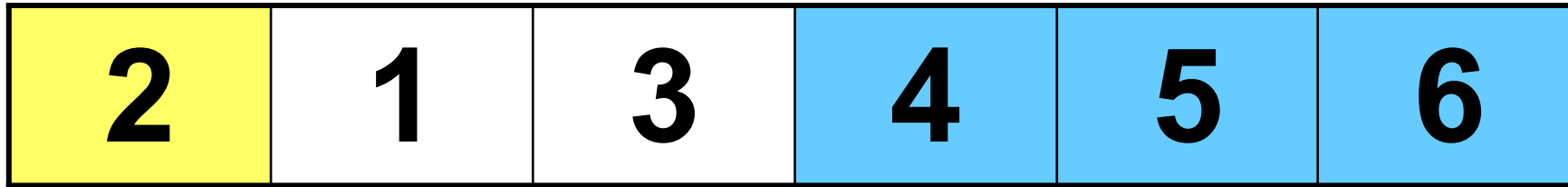


Data Movement



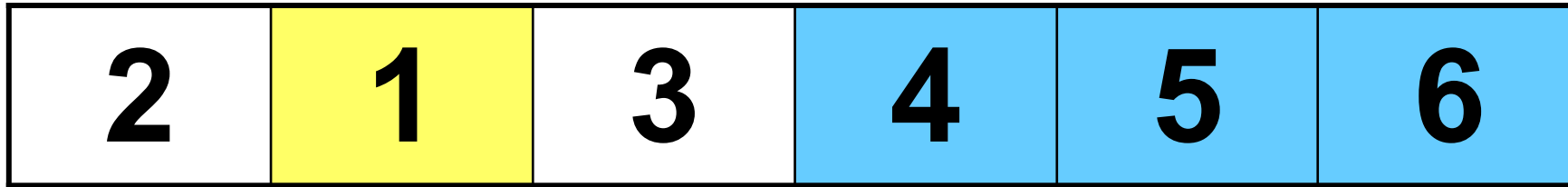
Sorted

Selection Sort



-  Comparison
-  Data Movement
-  Sorted

Selection Sort



Comparison

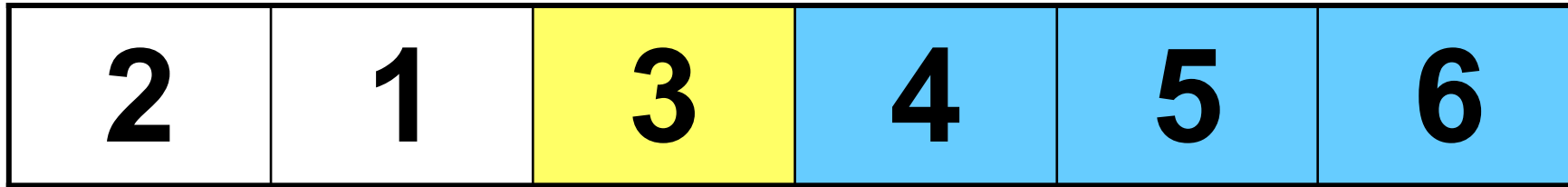


Data Movement



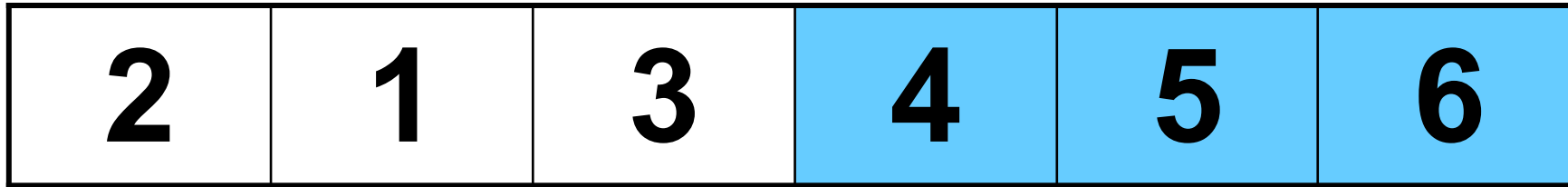
Sorted

Selection Sort



-  Comparison
-  Data Movement
-  Sorted

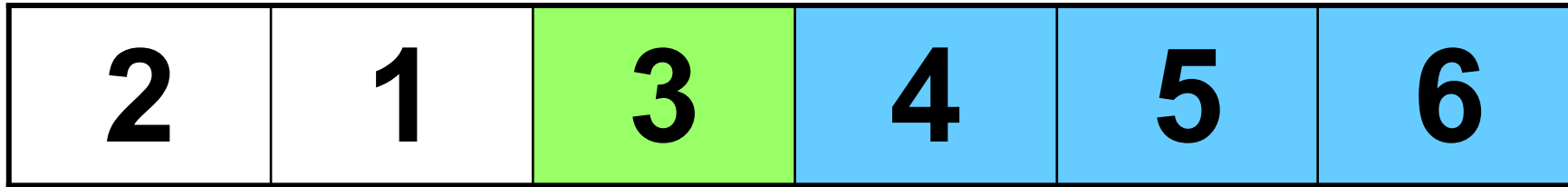
Selection Sort



↑
Largest

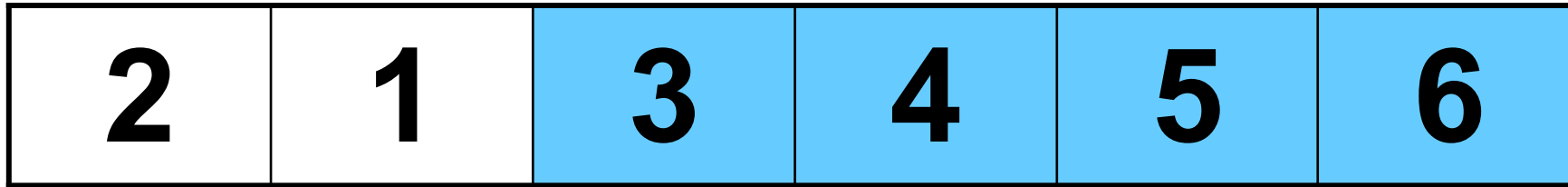
-  Comparison
-  Data Movement
-  Sorted

Selection Sort



-  Comparison
-  Data Movement
-  Sorted

Selection Sort



Comparison

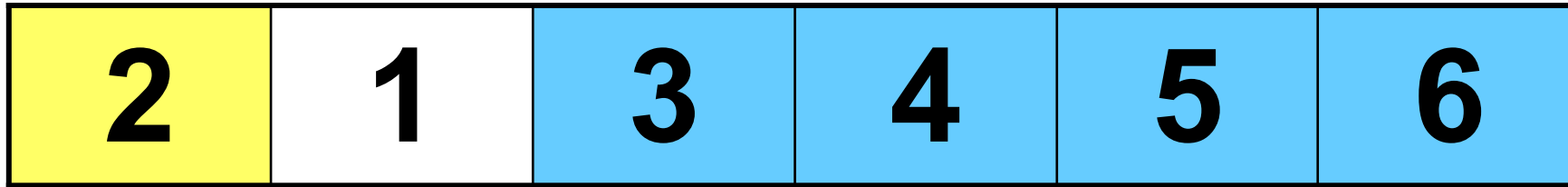


Data Movement



Sorted

Selection Sort



Comparison

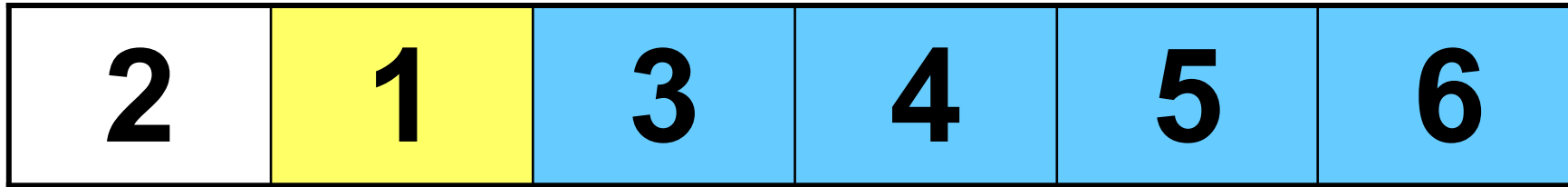


Data Movement



Sorted

Selection Sort



Comparison

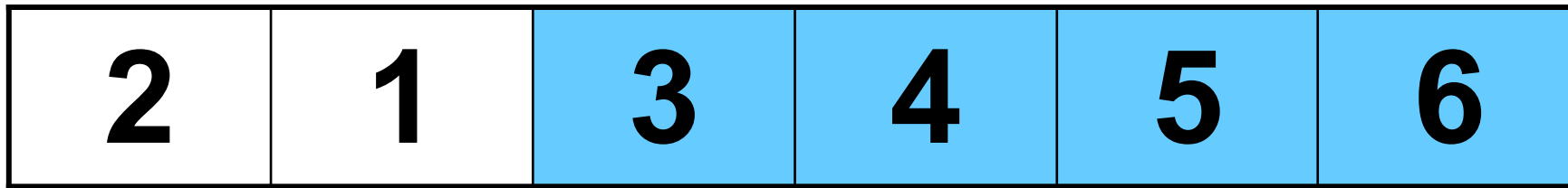


Data Movement



Sorted

Selection Sort



↑
Largest



Comparison

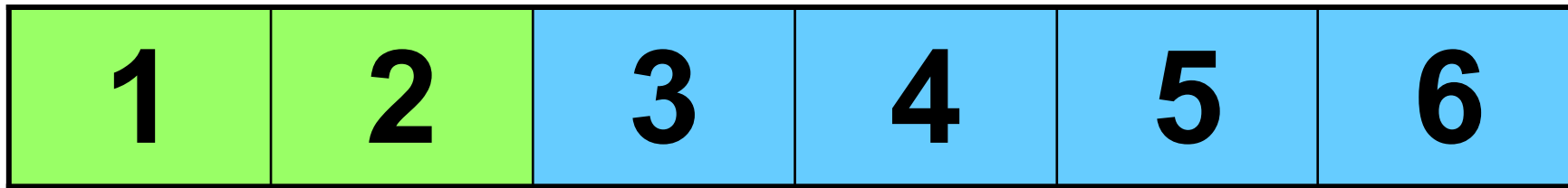


Data Movement



Sorted

Selection Sort



Comparison

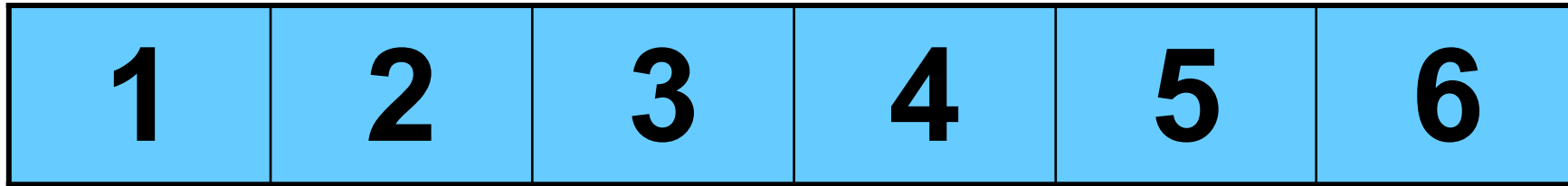


Data Movement



Sorted

Selection Sort



DONE!



Comparison



Data Movement



Sorted

Selection Sort

```
#include <stdio.h>

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    int i, j, min_idx, temp;
    printf("Original array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

Selection Sort (conti..)

```
for (i = 0; i < n - 1; i++) {  
    min_idx = i;  
    for (j = i + 1; j < n; j++) {  
        if (arr[j] < arr[min_idx])  
        {  
            min_idx = j;  
        }  
    }  
}  
  
// Swap the found minimum  
element with the first element  
temp = arr[min_idx];  
arr[min_idx] = arr[i];  
arr[i] = temp;  
}
```

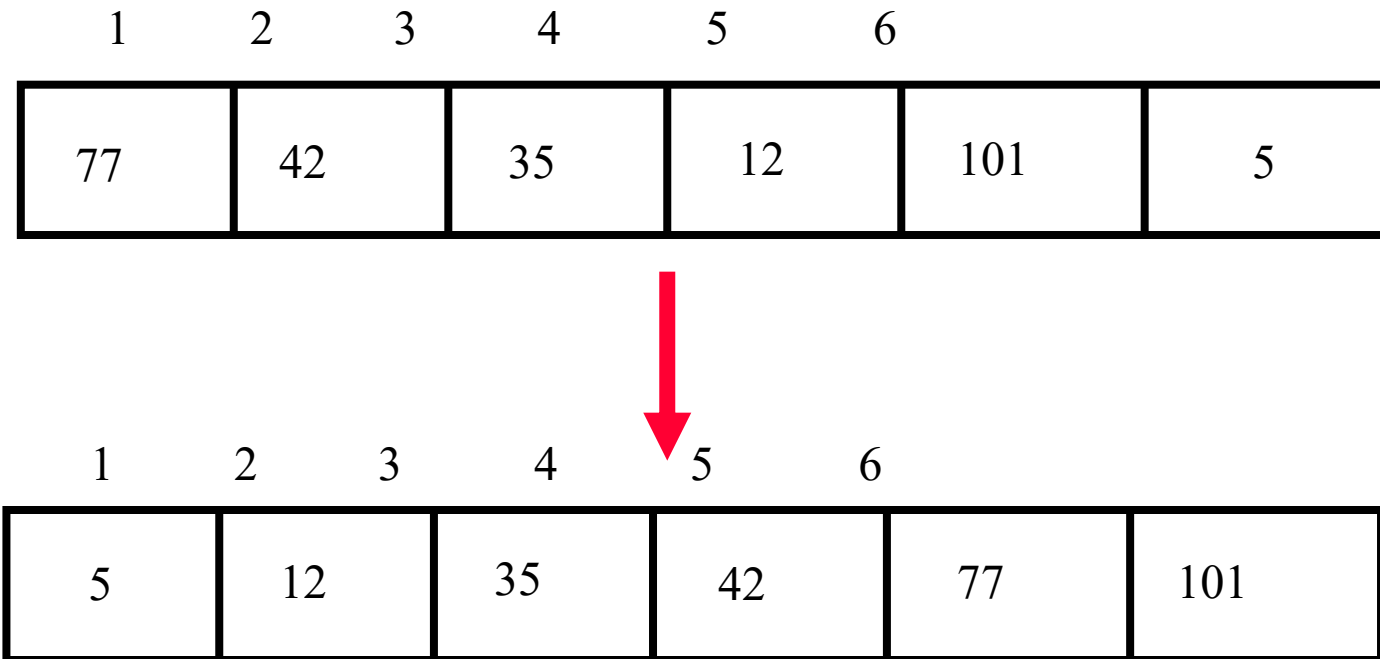
Selection Sort (conti..)

```
printf("Sorted array (Selection Sort): ");  
    for (i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
  
    return 0;  
}
```

Bubble Sort

Sorting

Sorting makes an unordered items list to an ordered one.



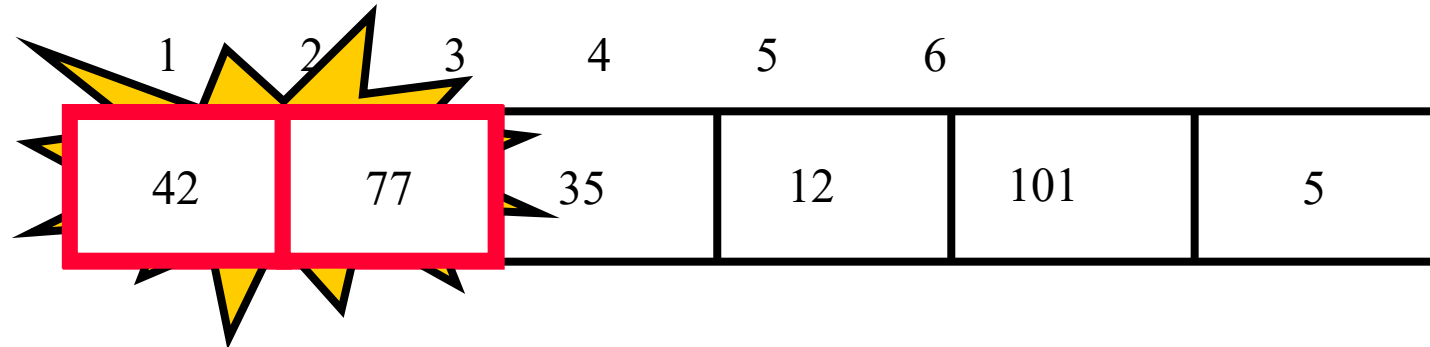
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the first element to the last
 - “**Bubble**” the **largest value** to the end using **pair-wise comparisons and swapping**

1	2	3	4	5	6
77	42	35	12	101	5

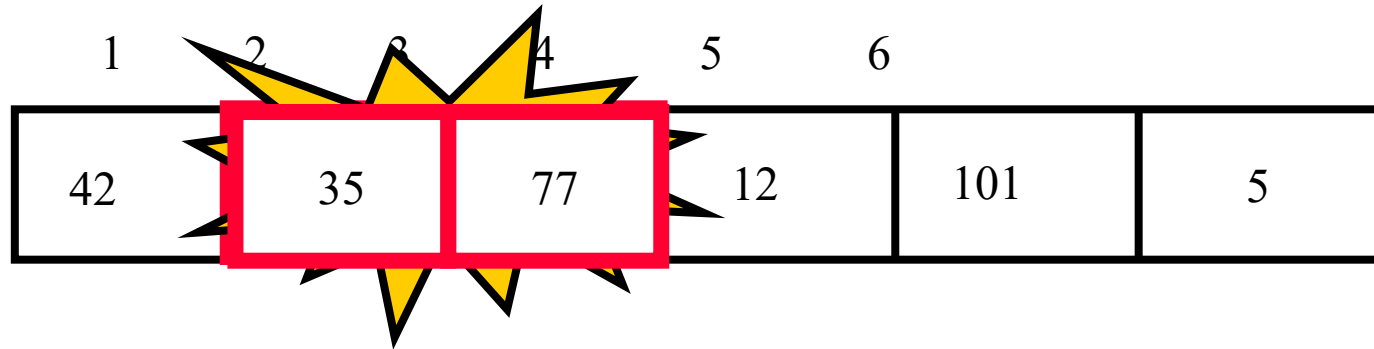
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - **Move from the front to the end**
 - **“Bubble” the largest value to the end using pair-wise comparisons and swapping**



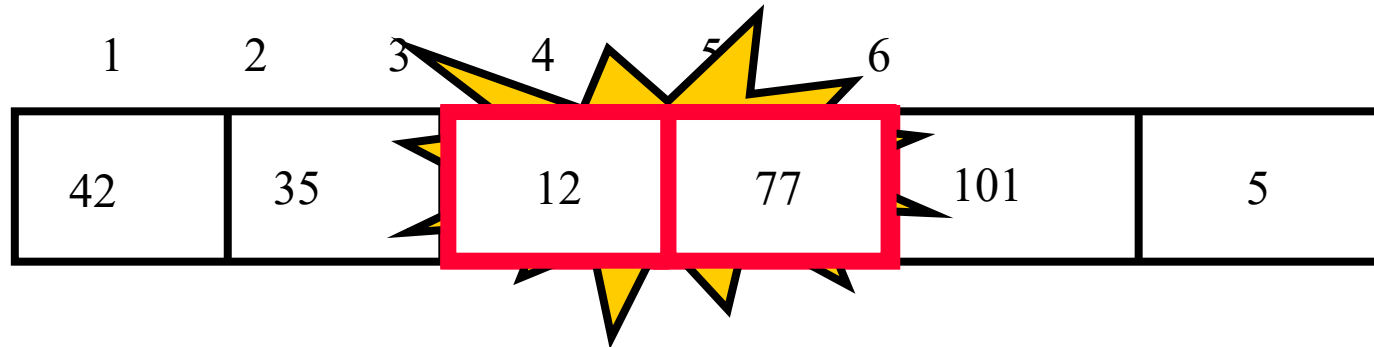
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - **Move from the front to the end**
 - **“Bubble” the largest value to the end using pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - **Move from the front to the end**
 - **“Bubble” the largest value to the end using pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

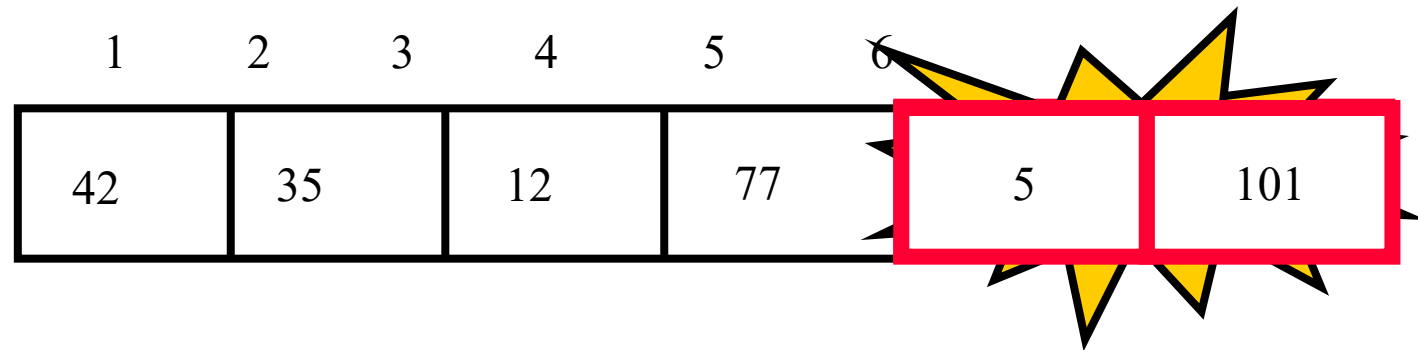
- **Traverse a collection of elements**
 - **Move from the front to the end**
 - **“Bubble” the largest value to the end using pair-wise comparisons and swapping**

1	2	3	4	5	6
42	35	12	77	101	5

No need to swap

"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - **Move from the front to the end**
 - **“Bubble” the largest value to the end using pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - **Move from the front to the end**
 - **“Bubble” the largest value to the end using pair-wise comparisons and swapping**

1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Bubble Sort Algorithm

We assume **list** is an array of **n** elements. We further assume that **swap** function swaps the values of the given array elements.

- **Step 1** – Check if the first element in the input array is greater than the next element in the array.
- **Step 2** – If it is greater, swap the two elements; otherwise move the pointer forward in the array.
- **Step 3** – Repeat Step 2 until we reach the end of the array.
- **Step 4** – Check if the elements are sorted; if not, repeat the same process (Step 1 to Step 3) from the last element of the array to the first.
- **Step 5** – The final output achieved is the sorted array.

Items of Interest

- Notice that only the largest value is correctly placed
- All other values are still out of order
- So we need to **repeat this process**

1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Repeat “Bubble Up” How Many Times?

- If we have N elements...
- And if each time we bubble an element, we place it in its correct location...
- Then we repeat the “bubble up” process $N - 1$ times.
- This guarantees we’ll correctly place all N elements.

“Bubbling” All the Elements

1	2	3	4	5	6
42	35	12	77	5	101
1	2	3	4	5	6
35	12	42	5	77	101
1	2	3	4	5	6
12	35	5	42	77	101
1	2	3	4	5	6
12	5	35	42	77	101
1	2	3	4	5	6
5	12	35	42	77	101

Reducing the Number of Comparisons

1	2	3	4	5	6
77	42	35	12	101	5

1	2	3	4	5	6
42	35	12	77	5	101

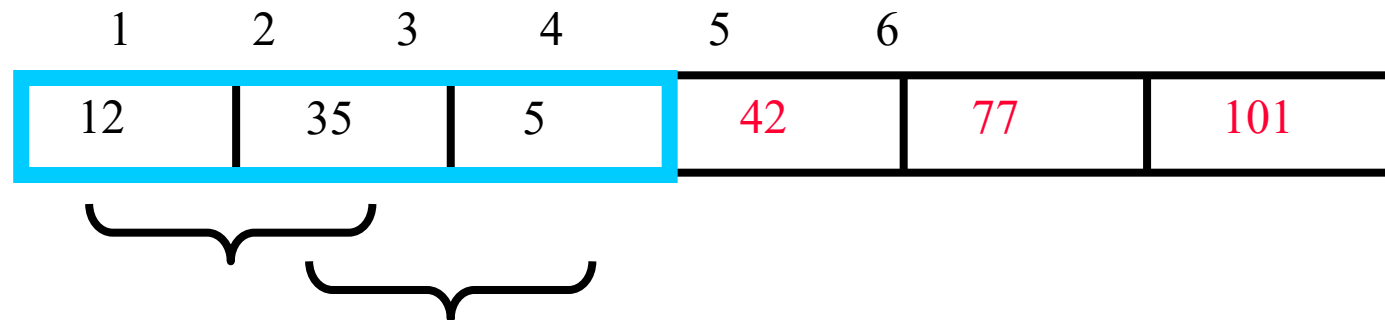
1	2	3	4	5	6
35	12	42	5	77	101

1	2	3	4	5	6
12	35	5	42	77	101

1	2	3	4	5	6
12	5	35	42	77	101

Reducing the Number of Comparisons

- On the N^{th} “bubble up”, we only need to do **MAX-N comparisons**.
- For example:
 - This is the 4th “bubble up”
 - MAX is 6
 - Thus we have **2 comparisons** to do



Bubble Sort Program

```
#include <stdio.h>

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);
    int i, j, temp;

    printf("Original array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

Bubble Sort Program

```
for (i = 0; i < n - 1; i++) {  
    for (j = 0; j < n - i - 1; j++) {  
        if (arr[j] > arr[j + 1]) {  
            // Swap arr[j] and arr[j+1]  
            temp = arr[j];  
            arr[j] = arr[j + 1];  
            arr[j + 1] = temp;  
        }  
    }  
}
```

```
printf("Sorted array (Bubble Sort):  
");  
for (i = 0; i < n; i++) {  
    printf("%d ", arr[i]);  
}  
printf("\n");  
  
return 0;  
}
```

Already Sorted Collections?

- What if the collection was already sorted?
- What if only a few elements were out of place and after a couple of “bubble ups,” the collection was sorted?
- We want to be able to **detect this and “stop early”!**

1	2	3	4	5	6
5	12	35	42	77	101

Using a Boolean “Flag”

- We can use a boolean variable to determine if any swapping occurred during the “bubble up.”
- If no swapping occurred, then we know that the collection is already sorted!
- This boolean “flag” needs to be reset after each “bubble up.”

```
did_swap isotype Boolean
did_swap <- true

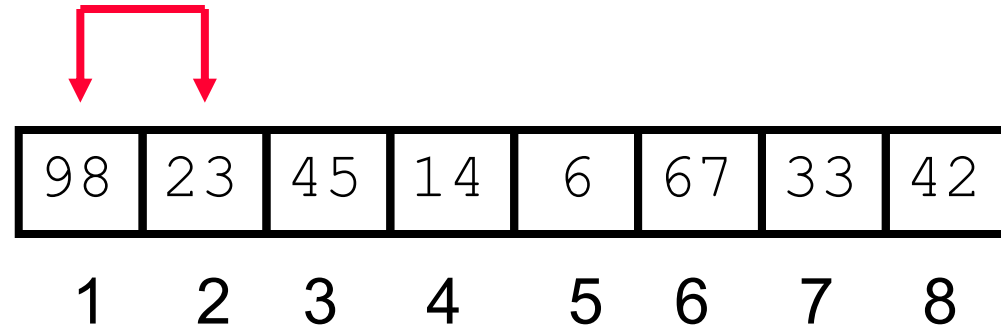
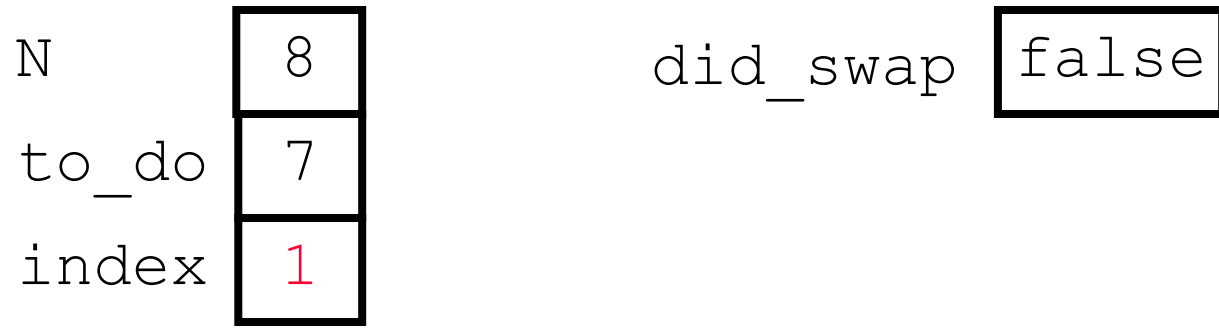
loop
  exitif ((to_do = 0) OR NOT(did_swap))
  index <- 1
  did_swap <- false
  loop
    exitif(index > to_do)
    if(A[index] > A[index + 1]) then
      Swap(A[index], A[index + 1])
      did_swap <- true
    endif
    index <- index + 1
  endloop
  to_do <- to_do - 1
endloop
```

An Animated Example

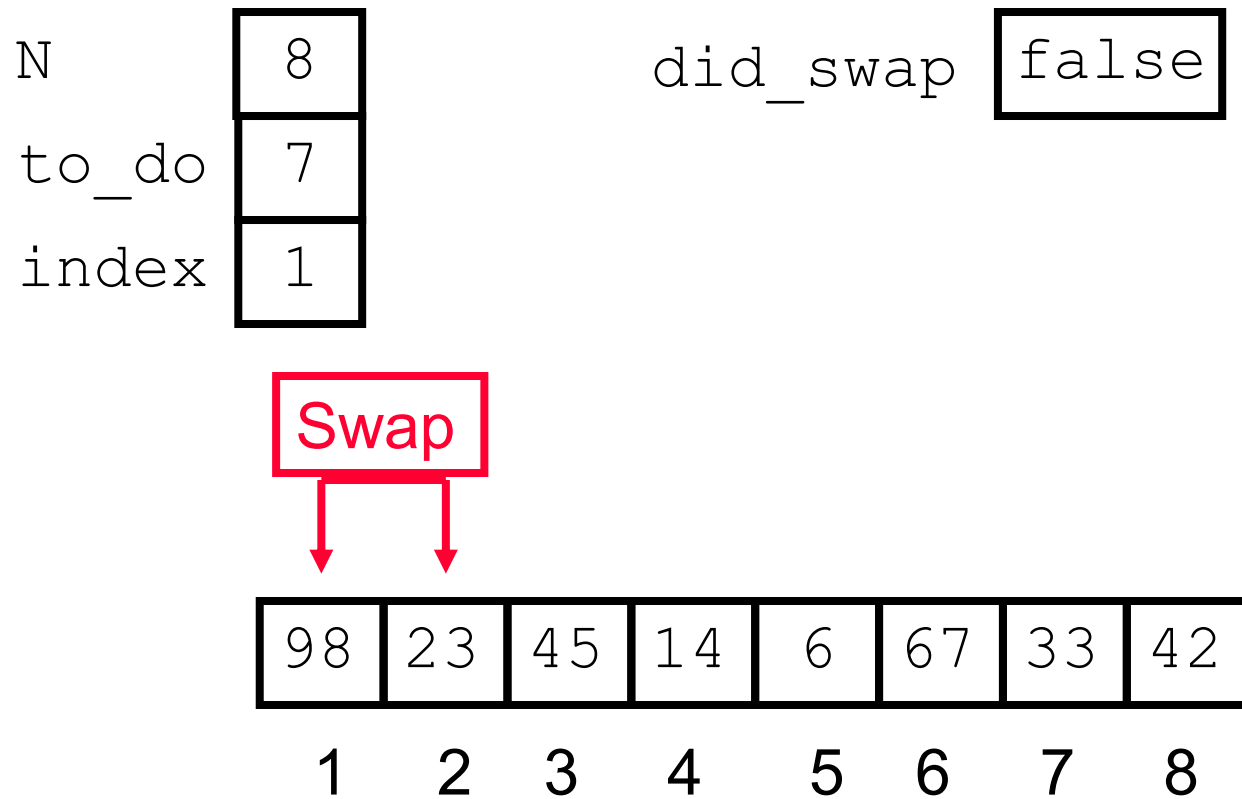
N	8	did_swap	true
to_do	7		
index			

98	23	45	14	6	67	33	42
1	2	3	4	5	6	7	8

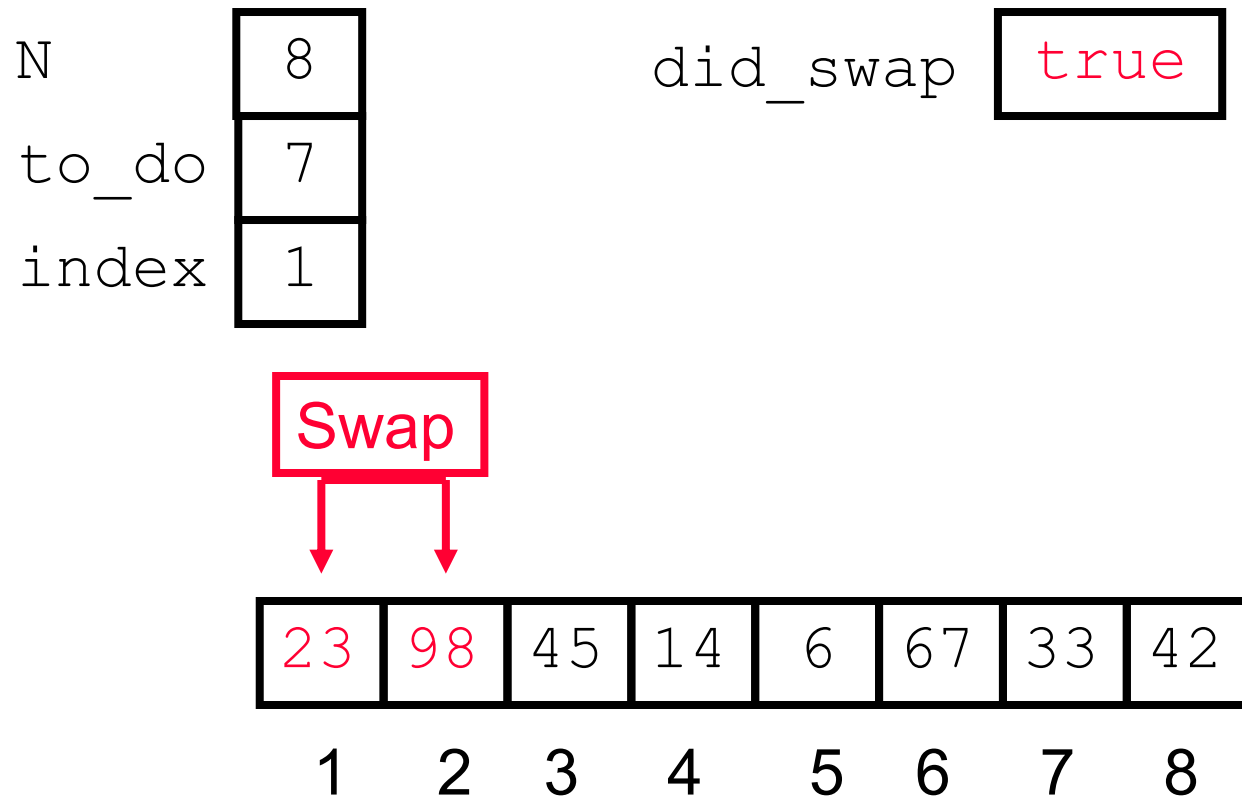
An Animated Example



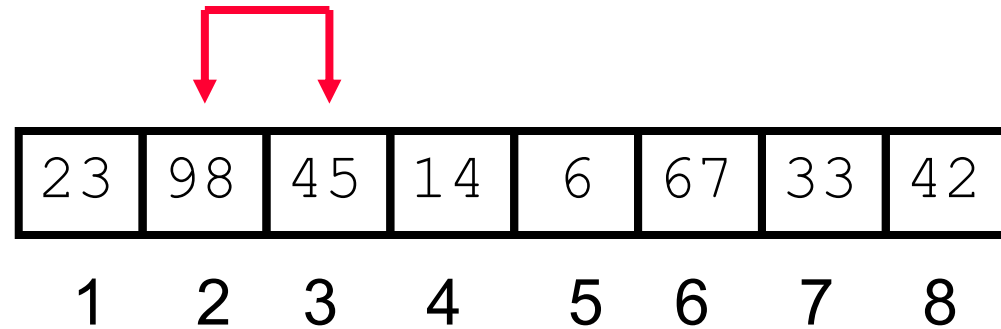
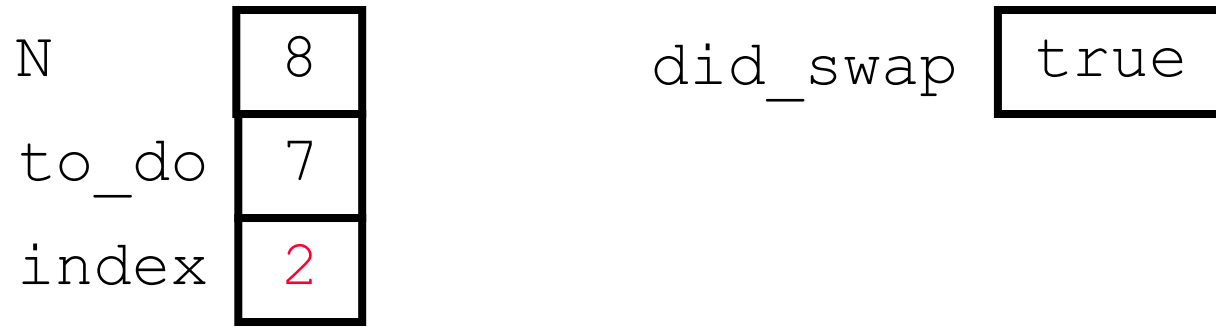
An Animated Example



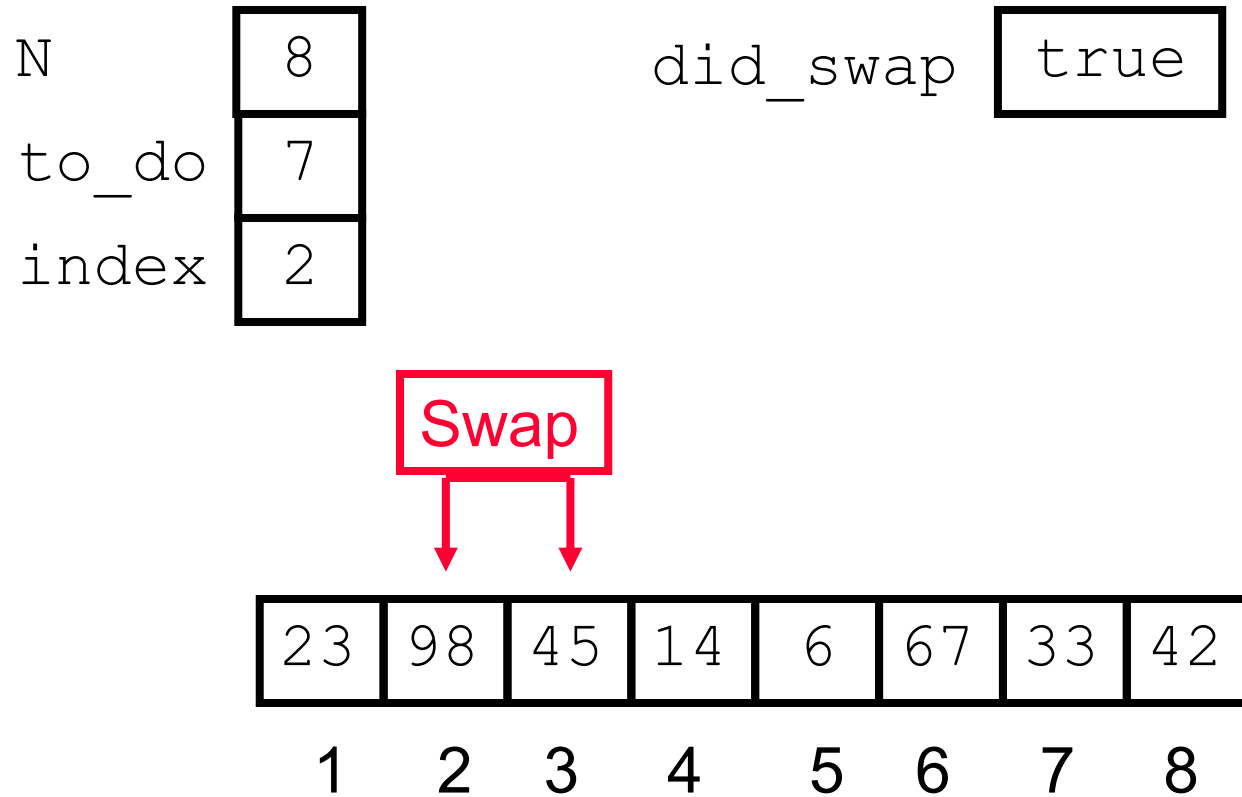
An Animated Example



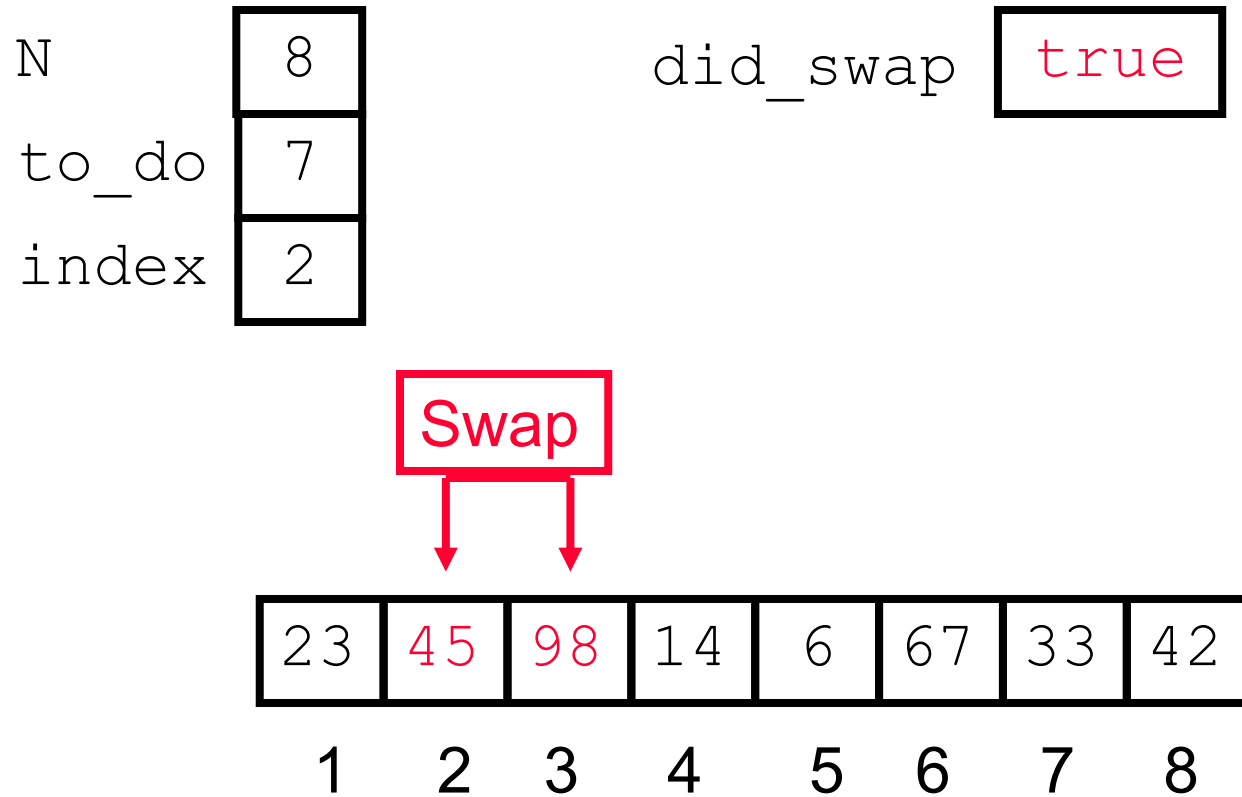
An Animated Example



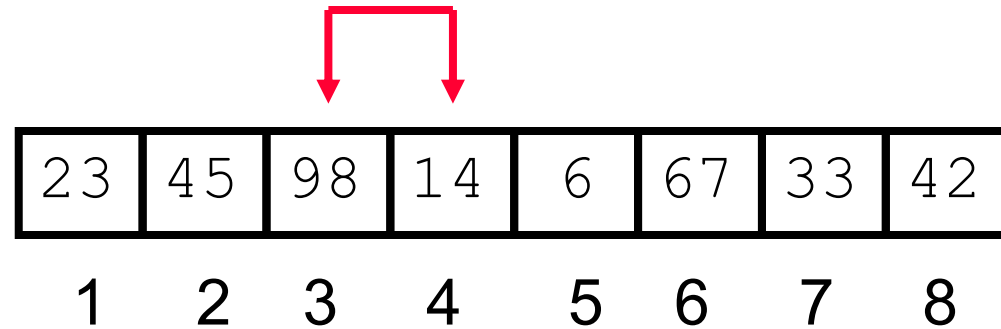
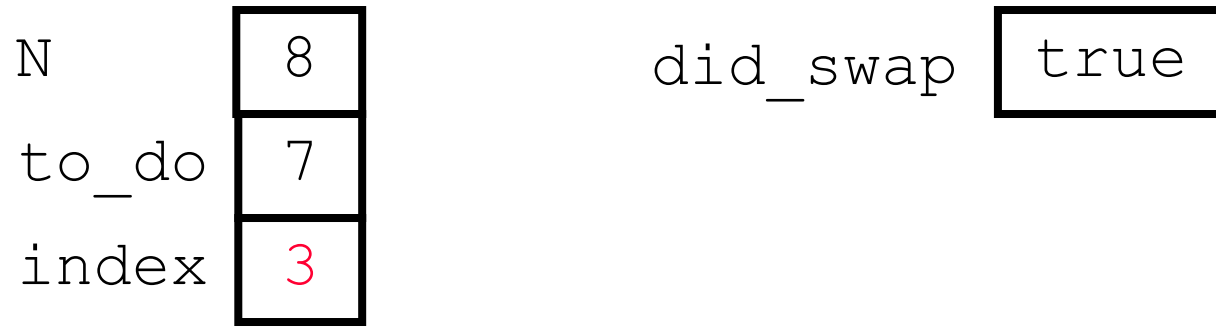
An Animated Example



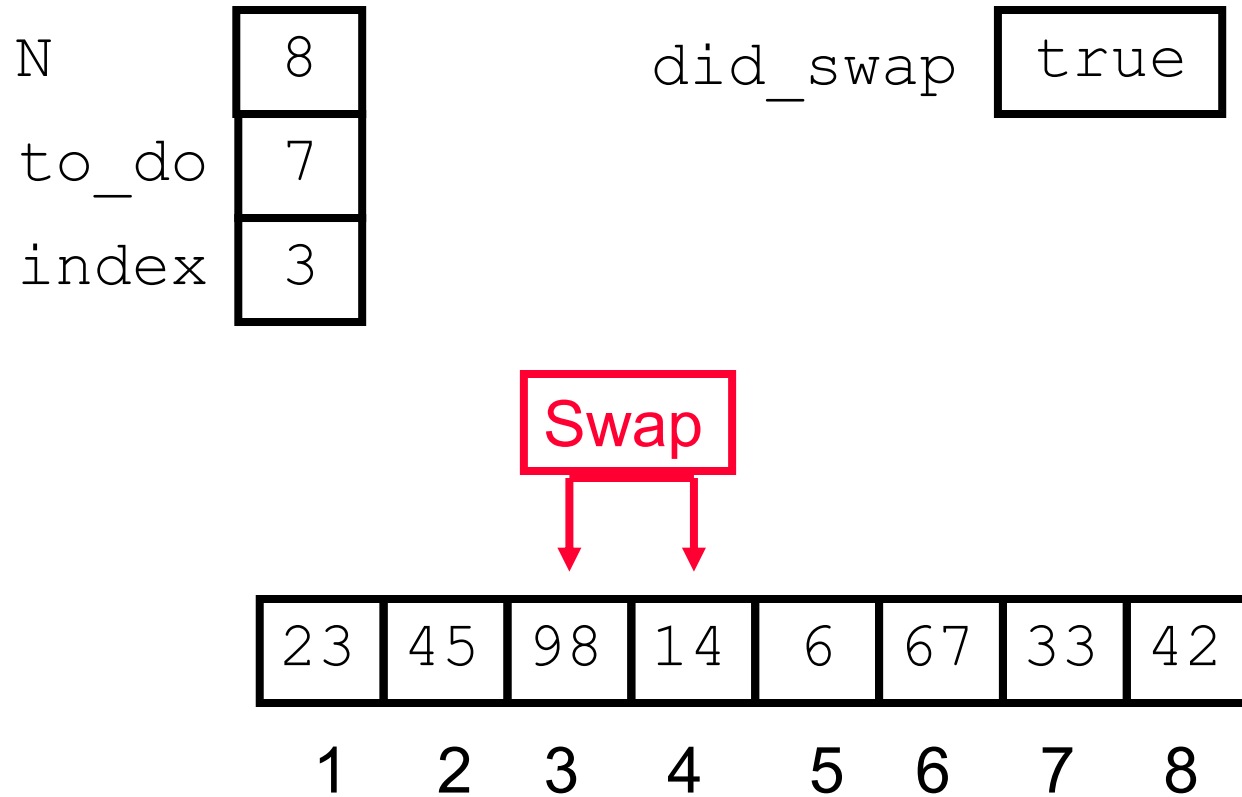
An Animated Example



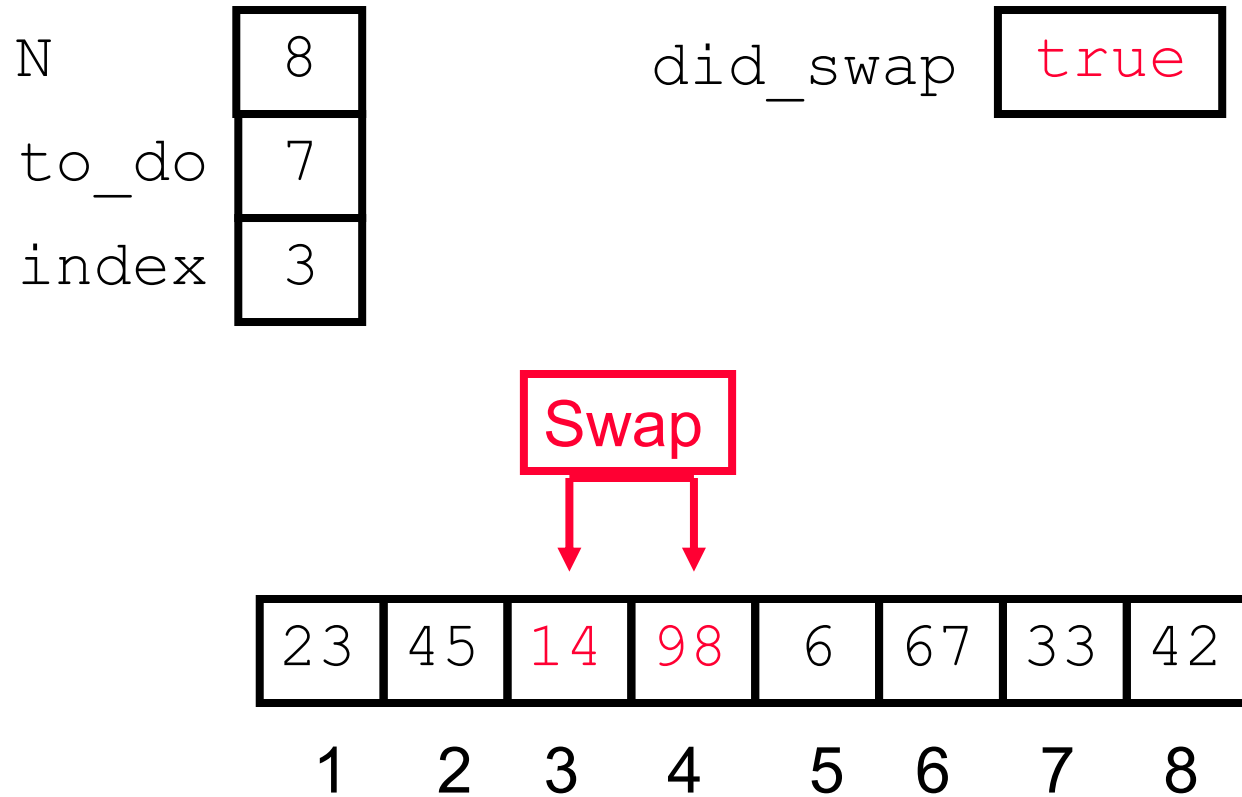
An Animated Example



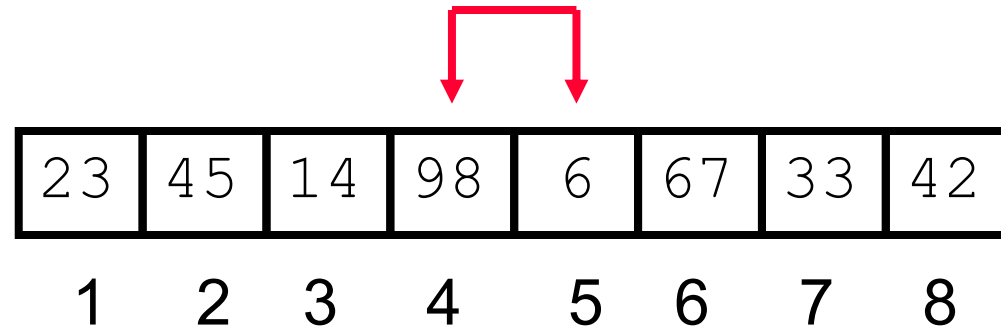
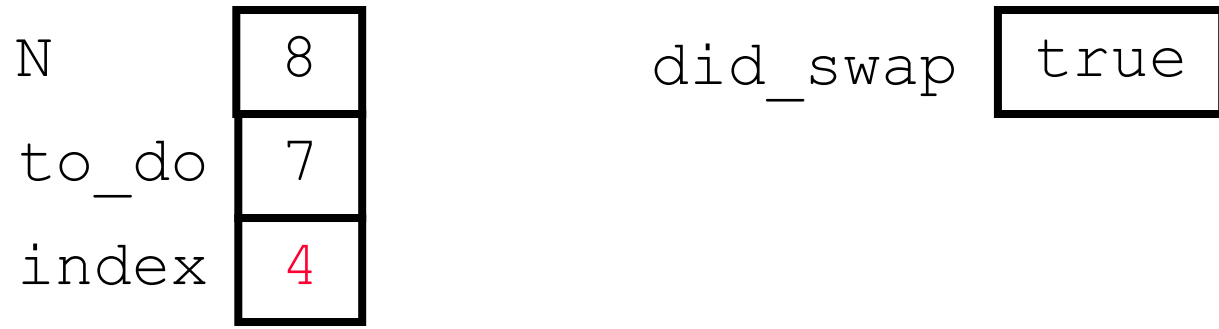
An Animated Example



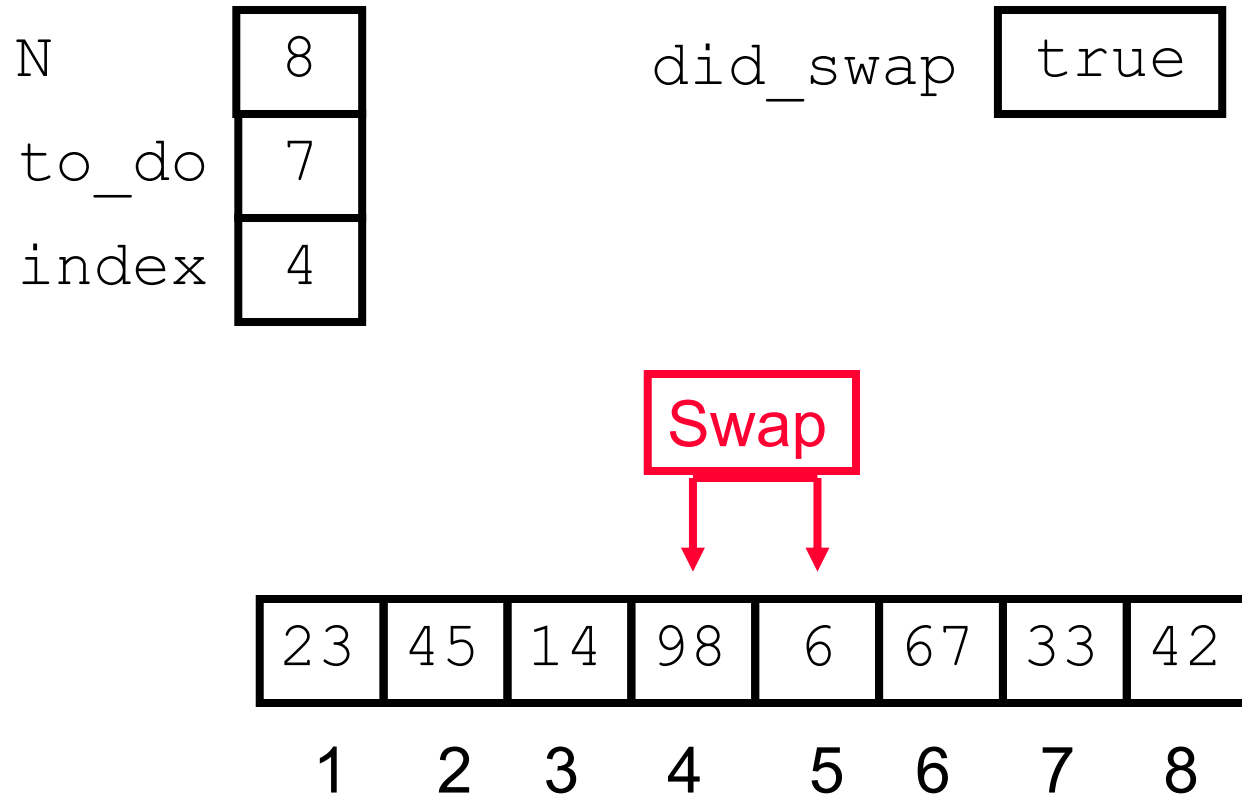
An Animated Example



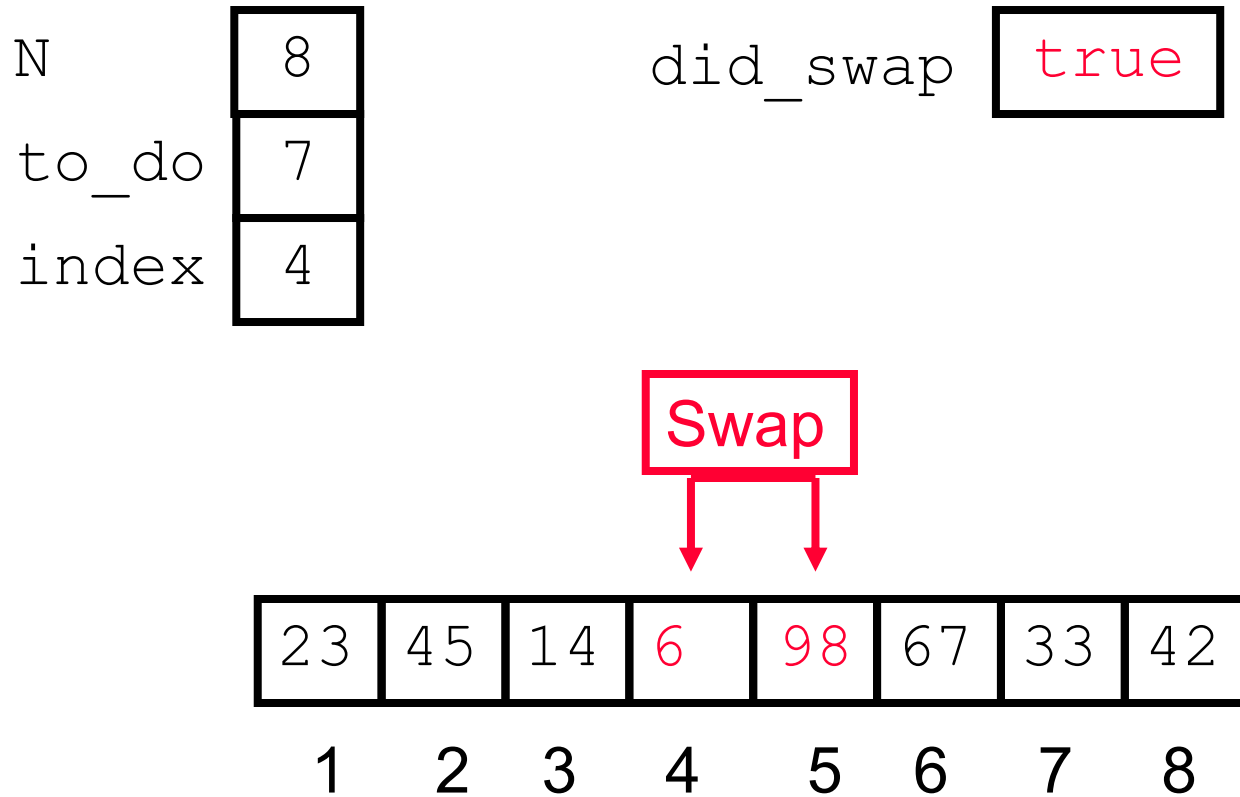
An Animated Example



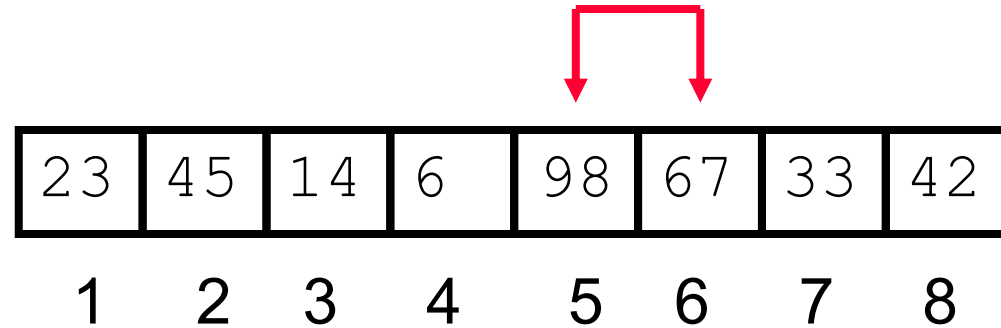
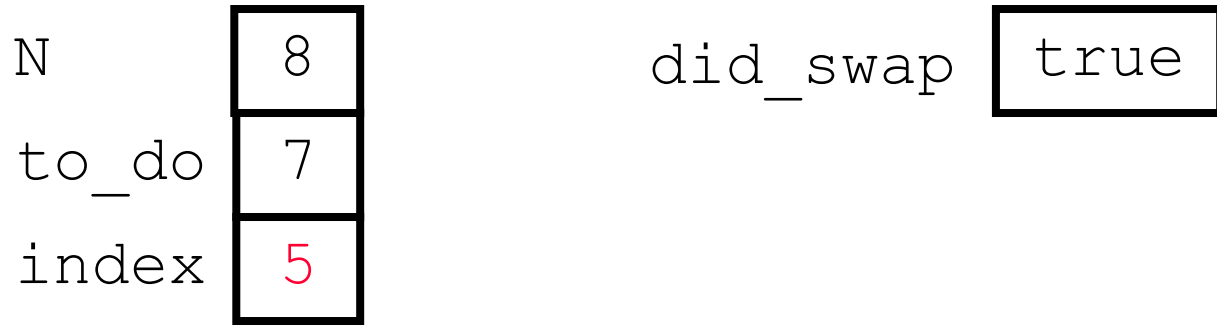
An Animated Example



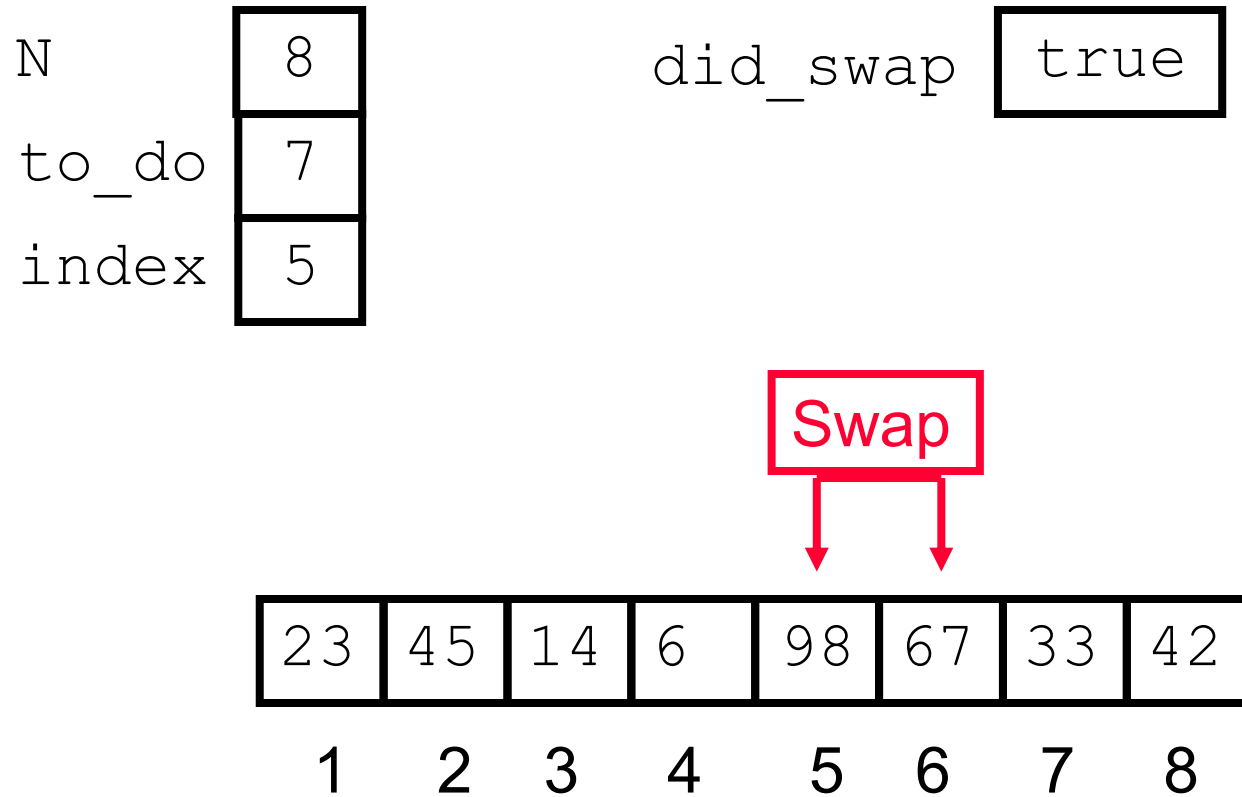
An Animated Example



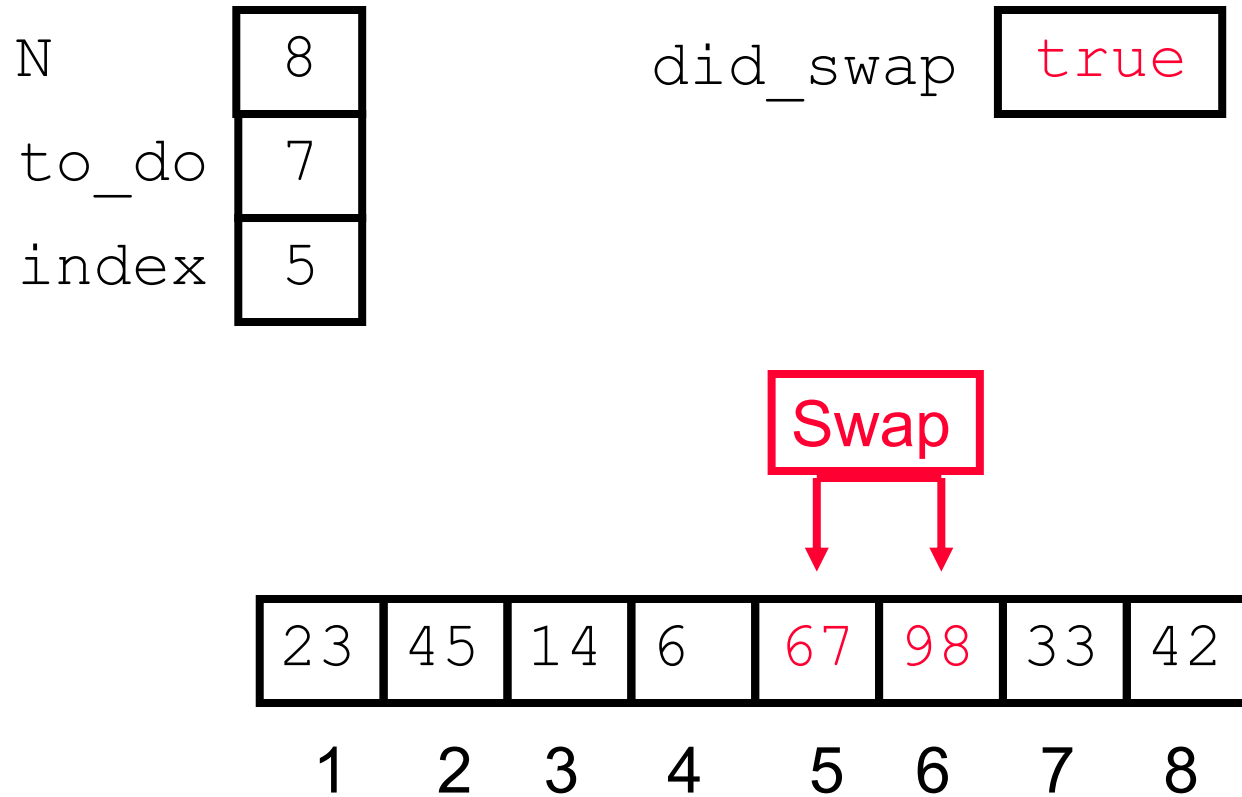
An Animated Example



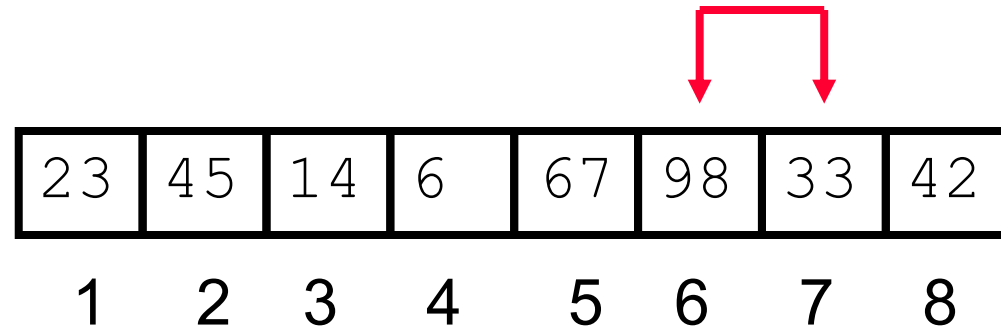
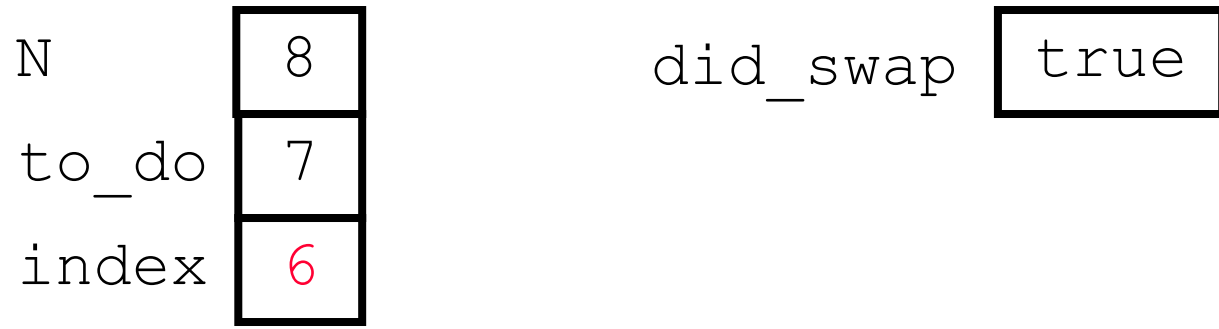
An Animated Example



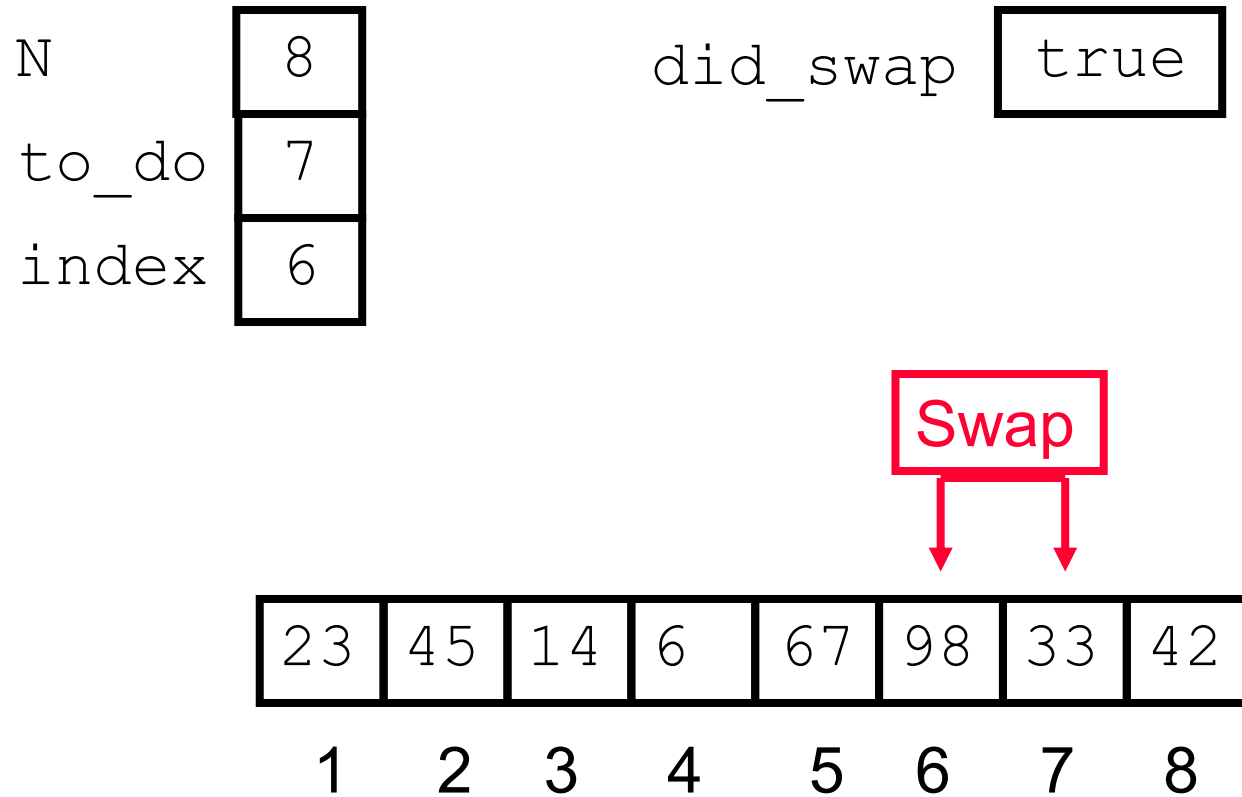
An Animated Example



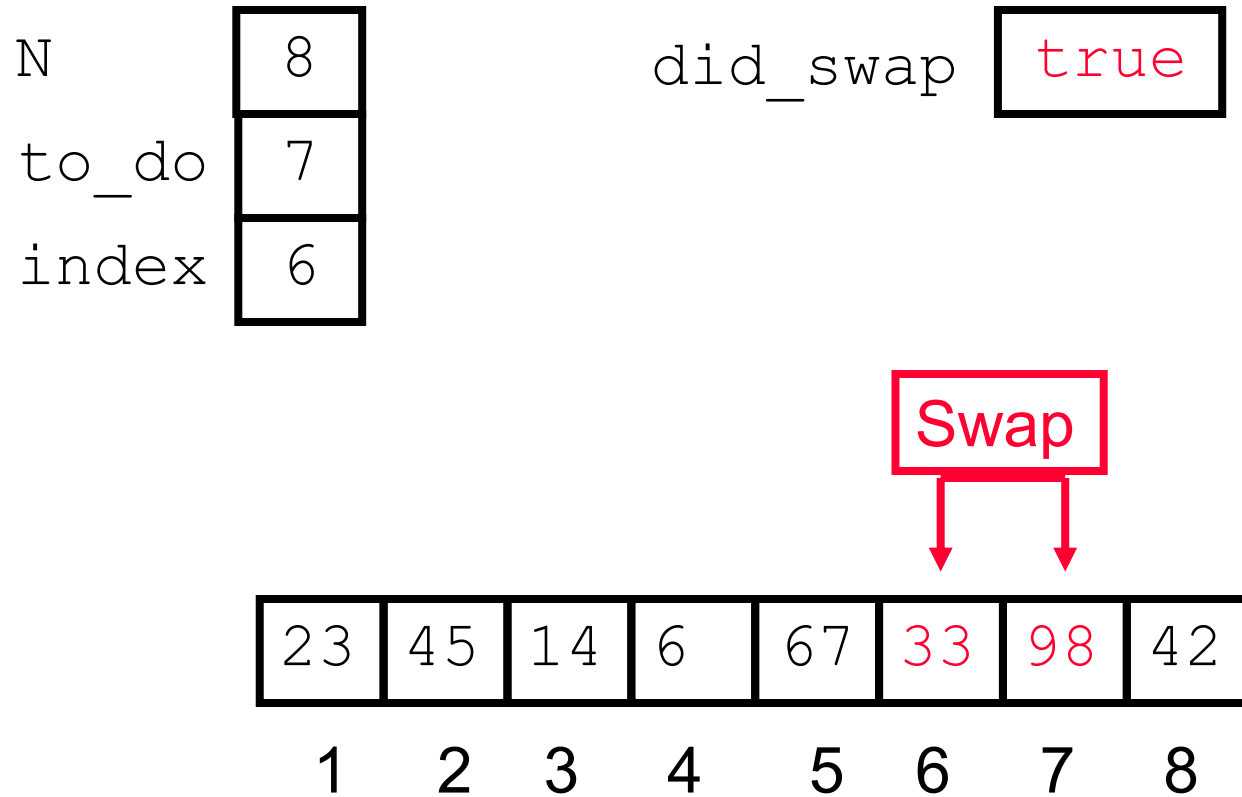
An Animated Example



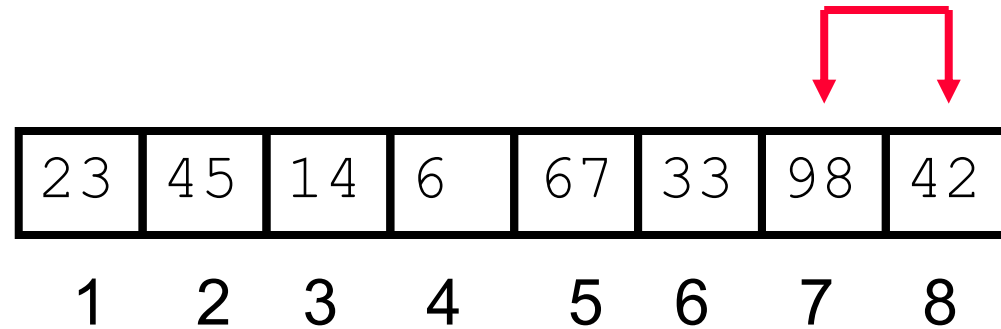
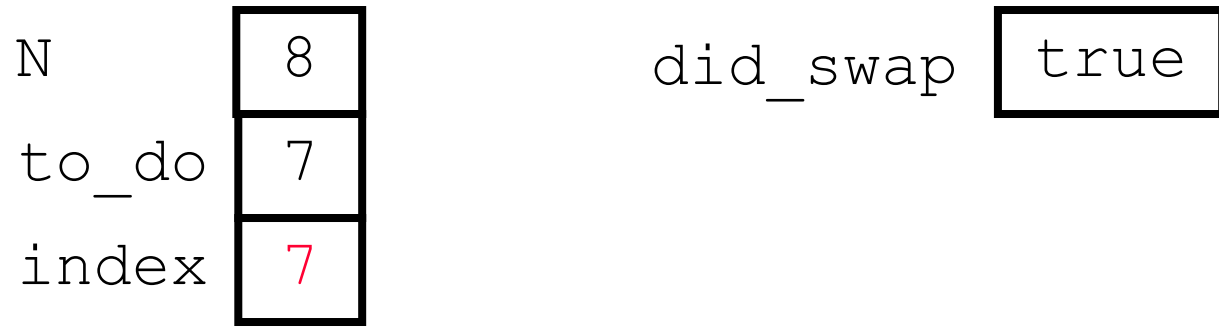
An Animated Example



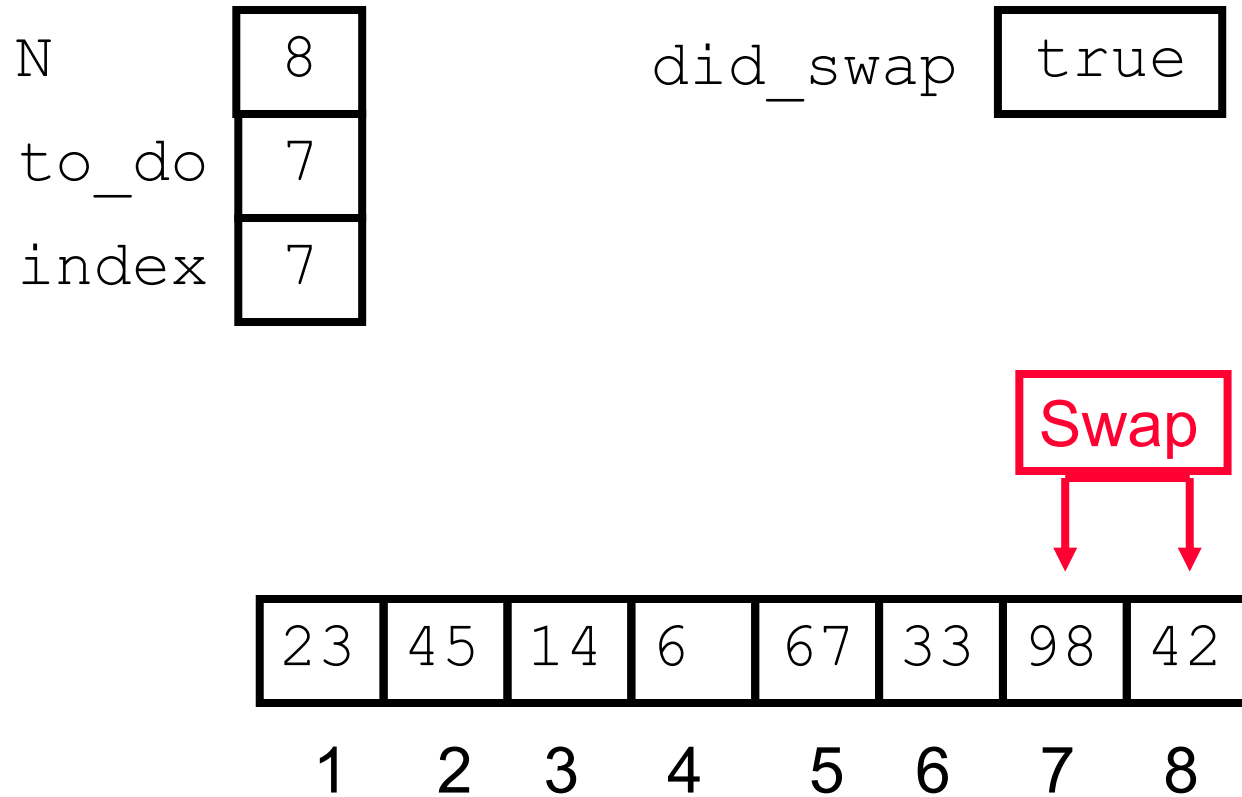
An Animated Example



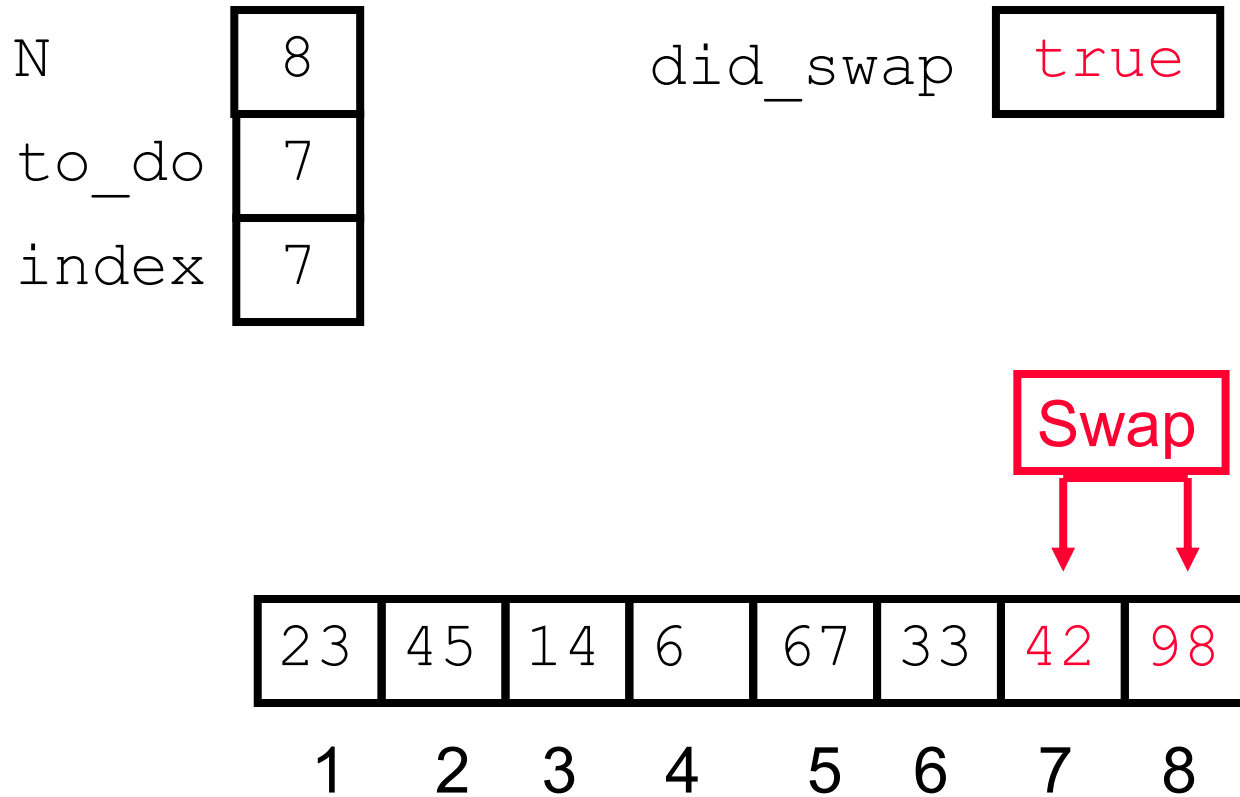
An Animated Example



An Animated Example



An Animated Example



After First Pass of Outer Loop

N

8

to_do

7

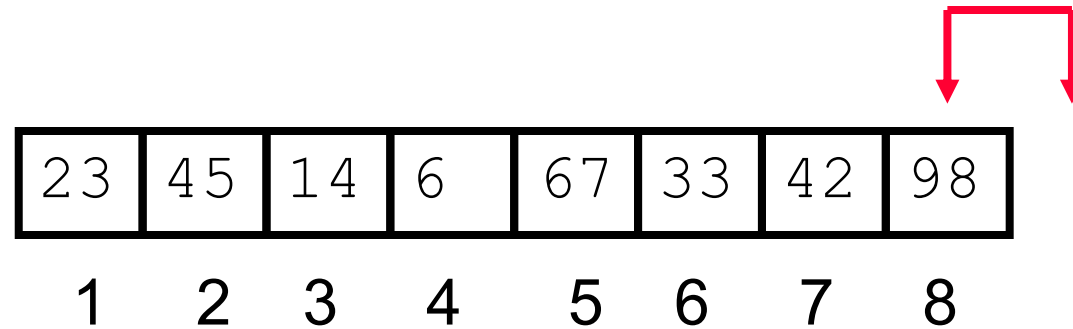
index

8

did_swap

true

Finished first "Bubble Up"



The Second “Bubble Up”

N

8

 did_swap

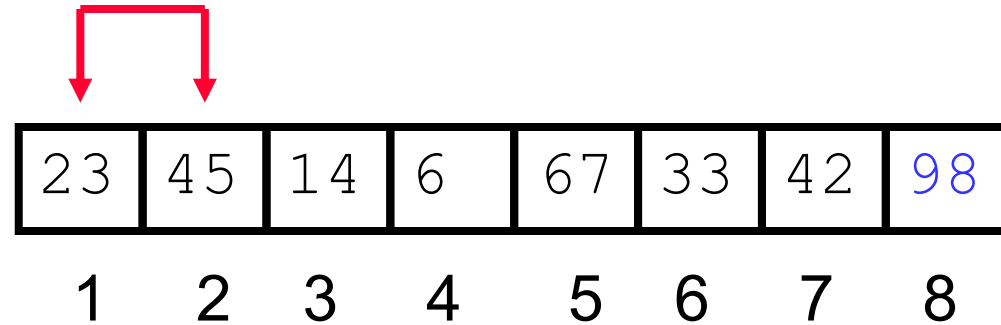
false

to_do

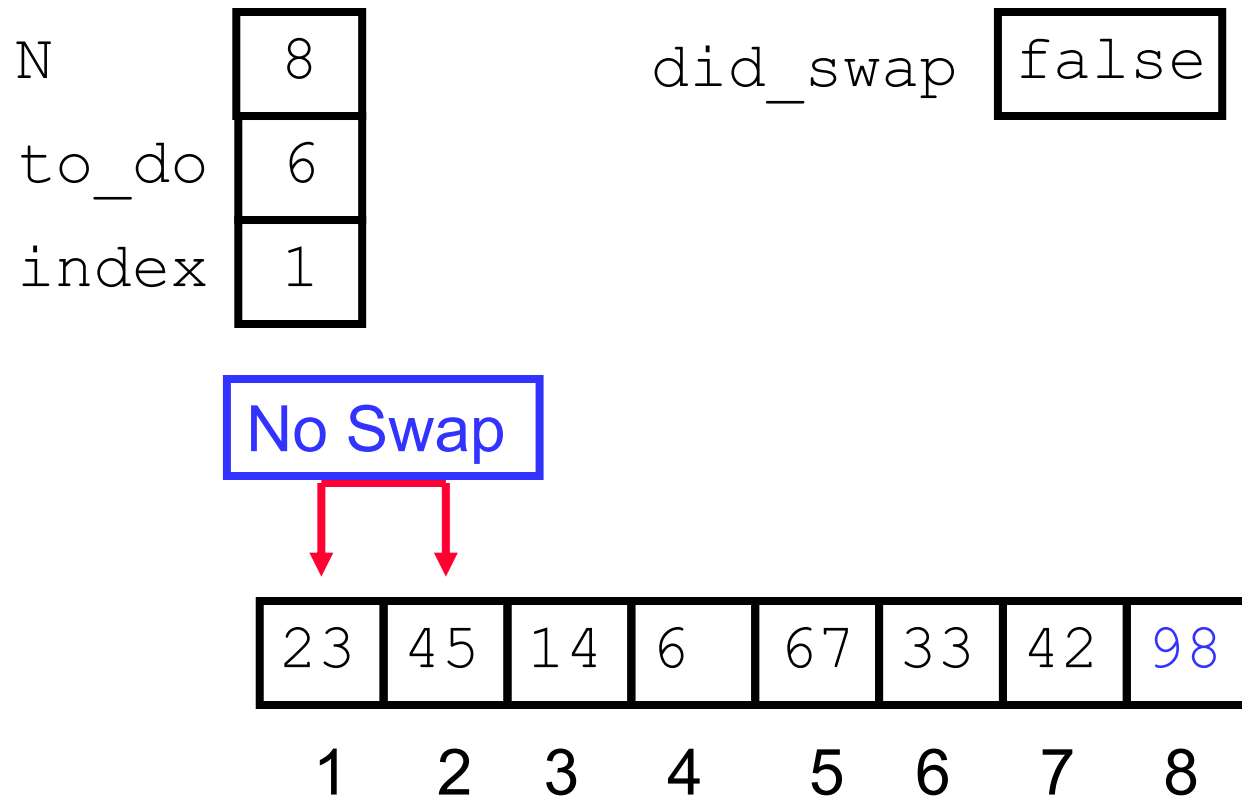
6

index

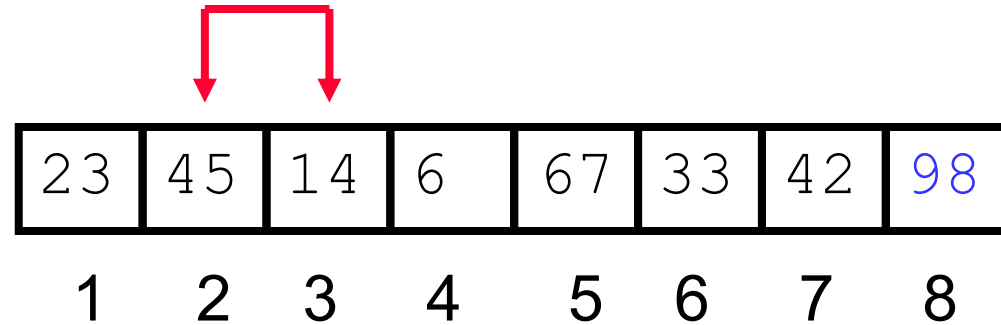
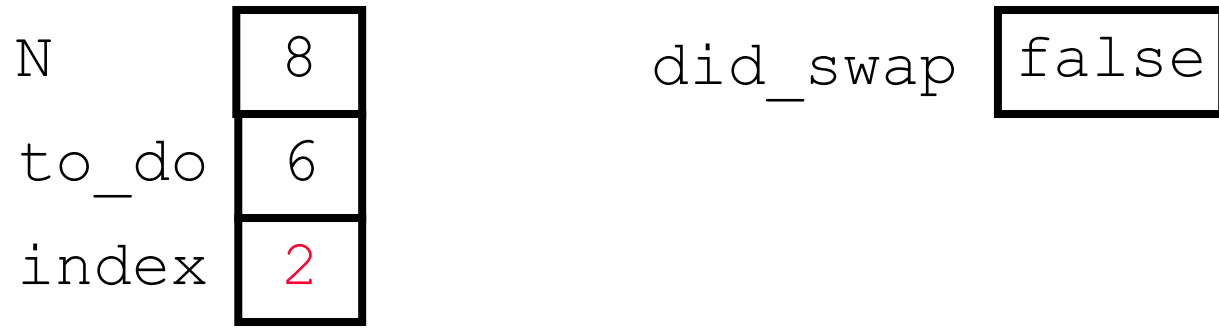
1



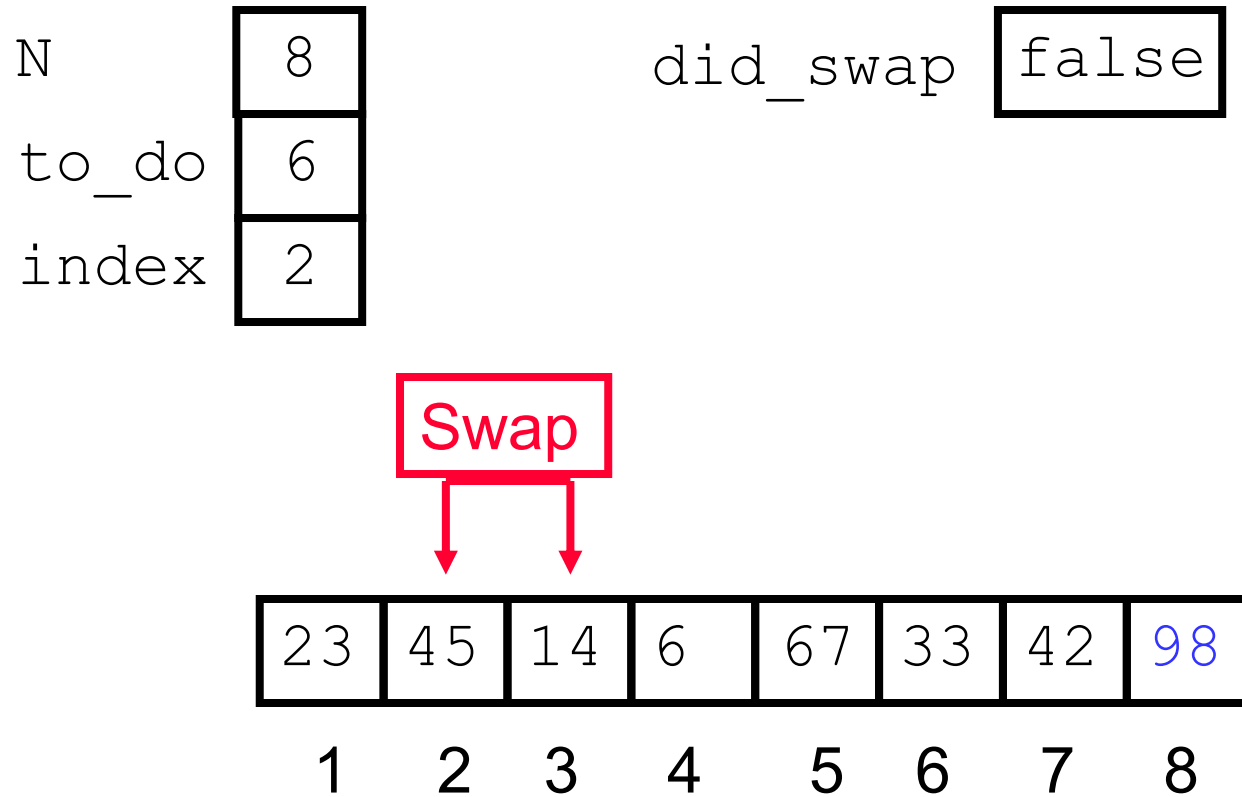
The Second “Bubble Up”



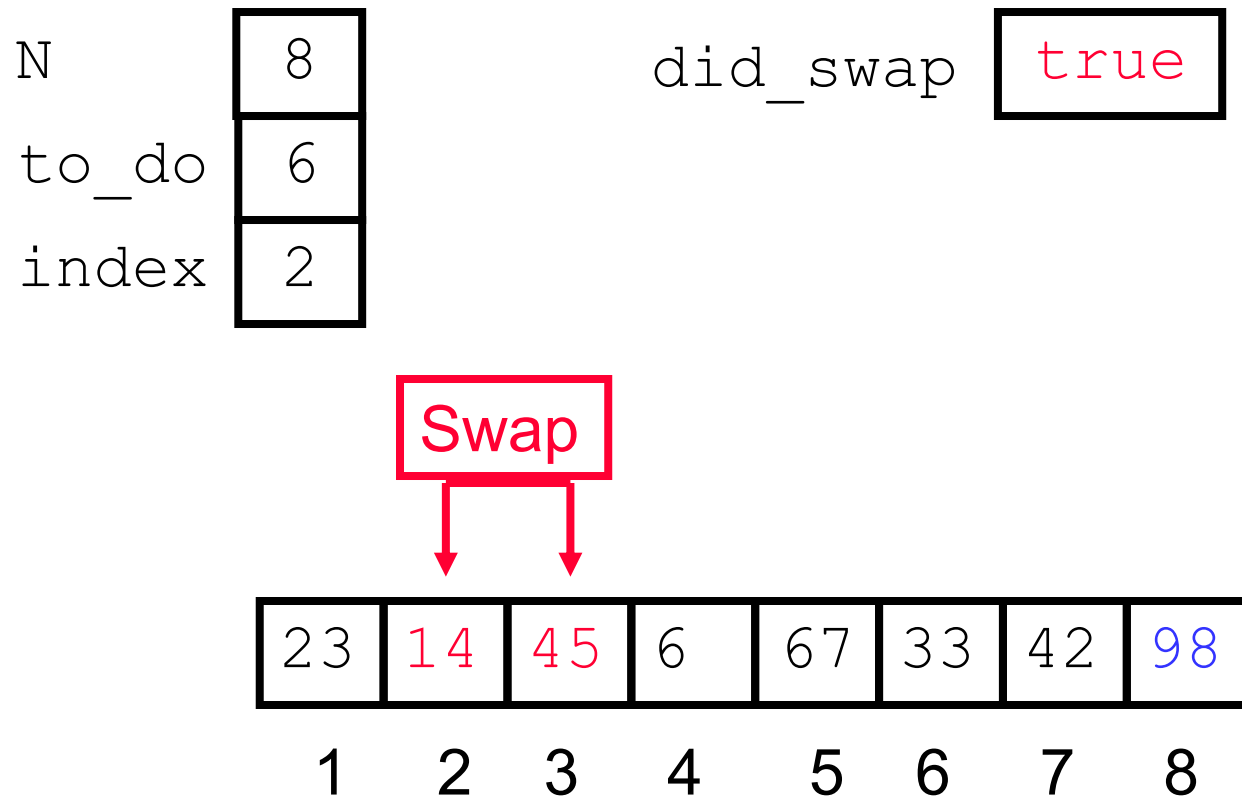
The Second “Bubble Up”



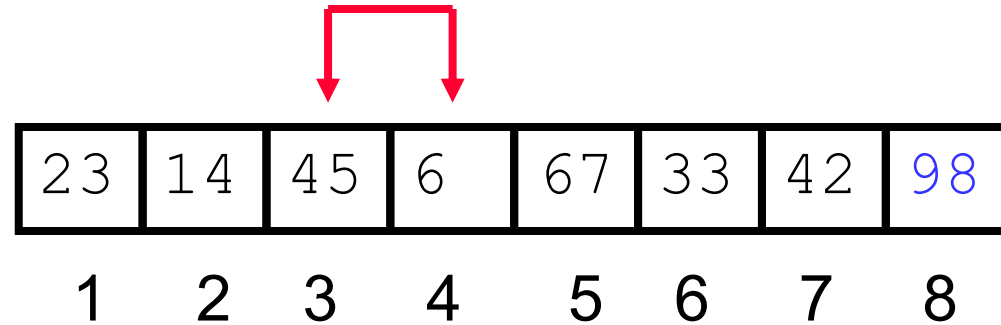
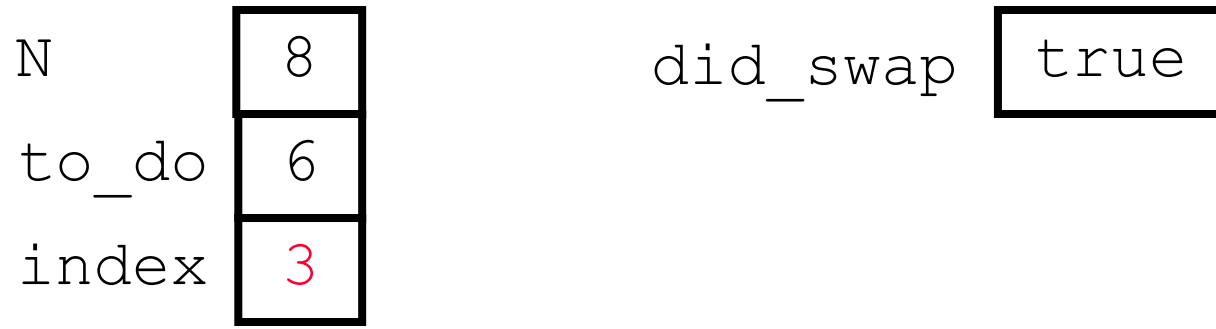
The Second “Bubble Up”



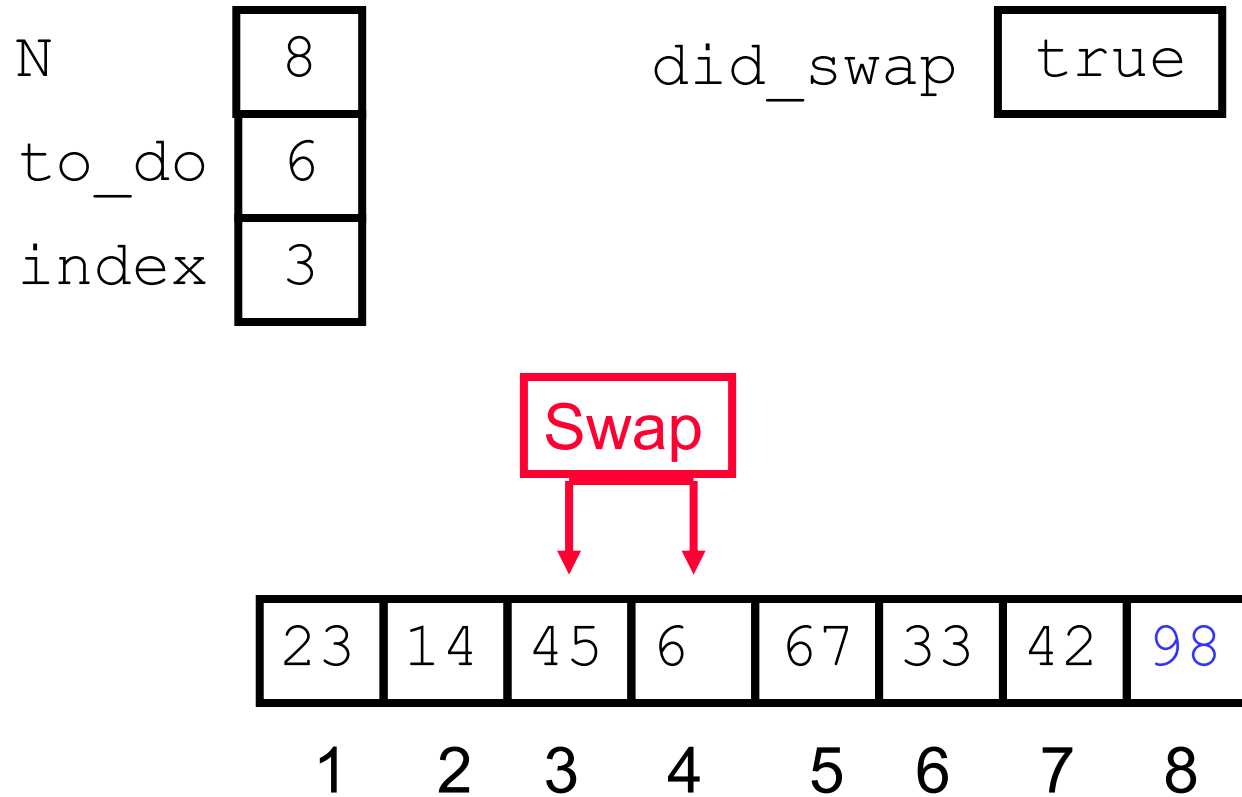
The Second “Bubble Up”



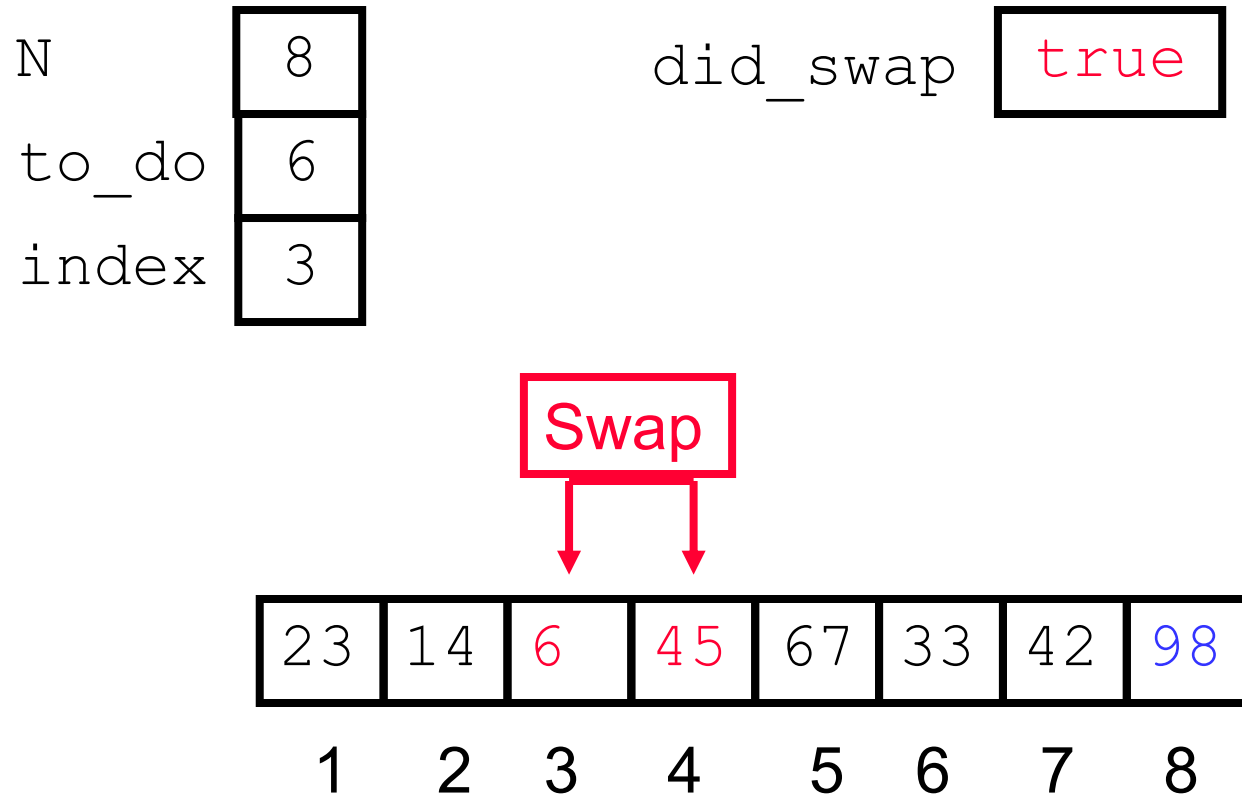
The Second “Bubble Up”



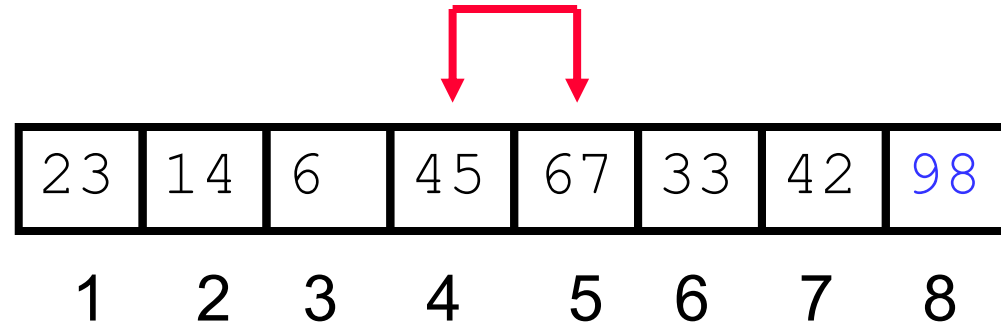
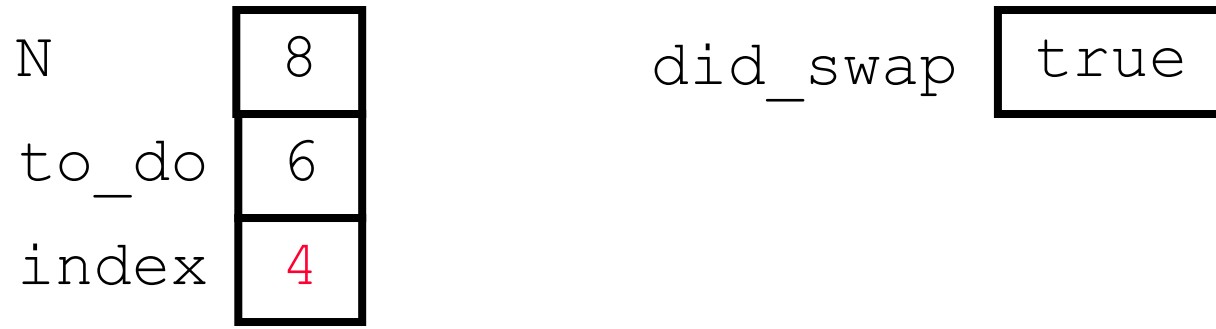
The Second “Bubble Up”



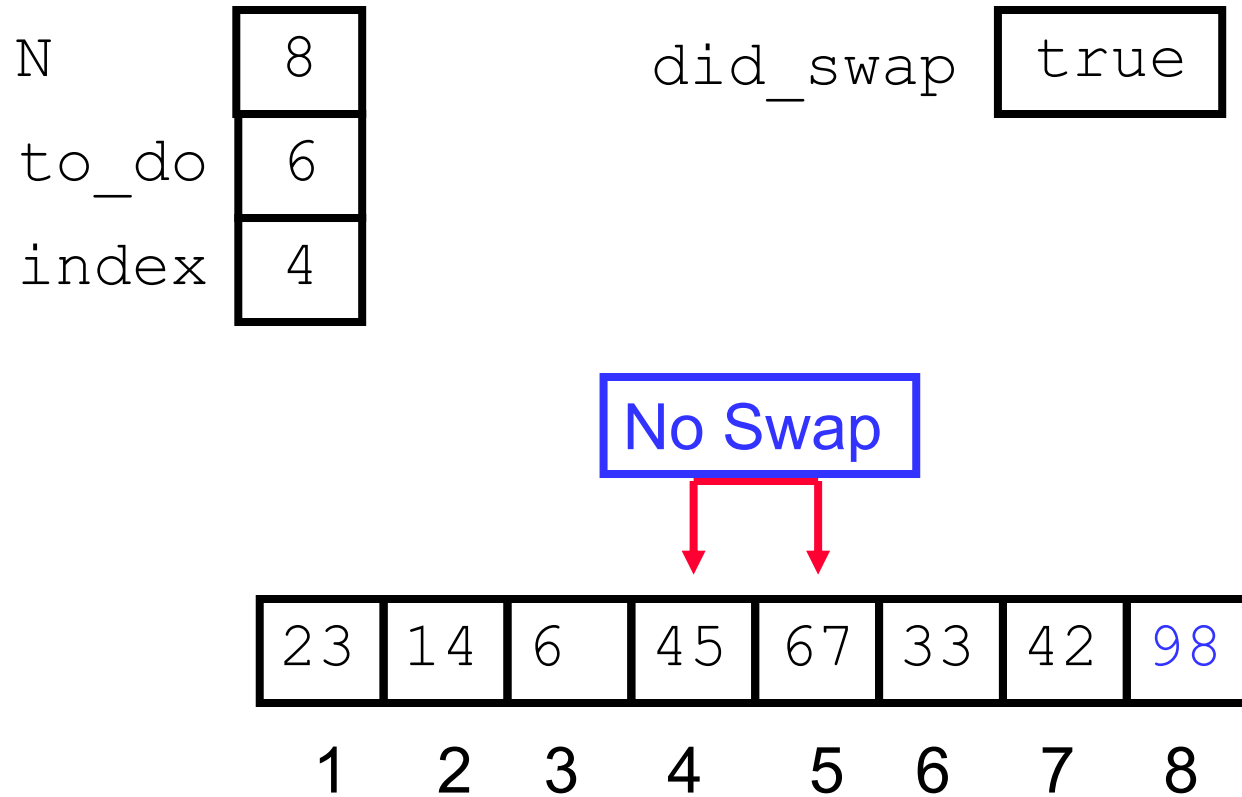
The Second “Bubble Up”



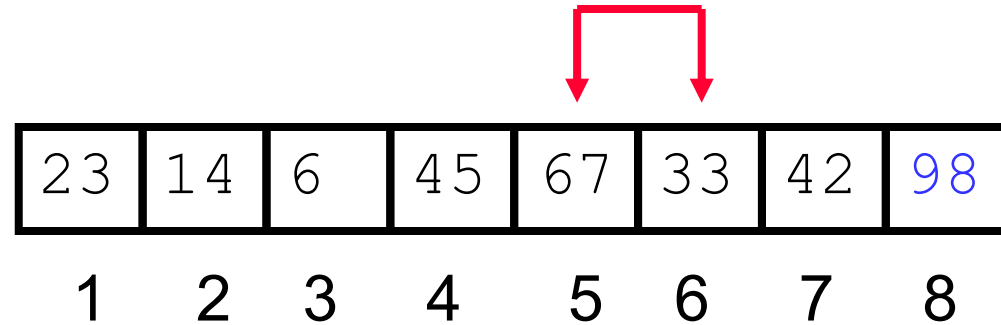
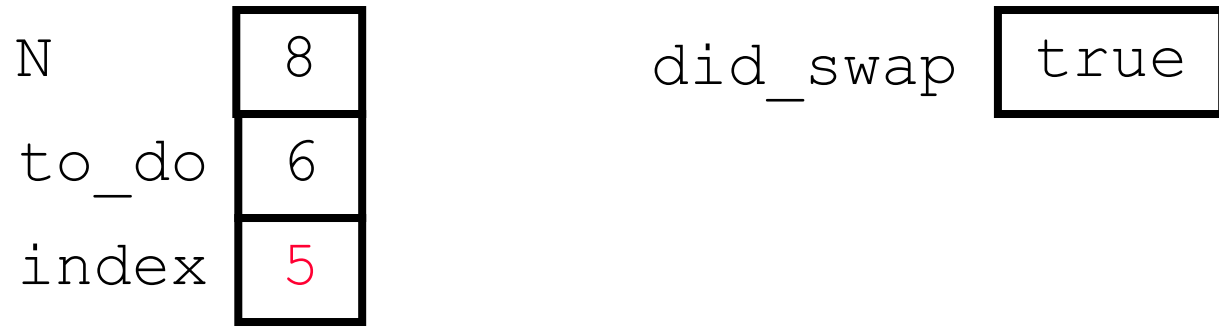
The Second “Bubble Up”



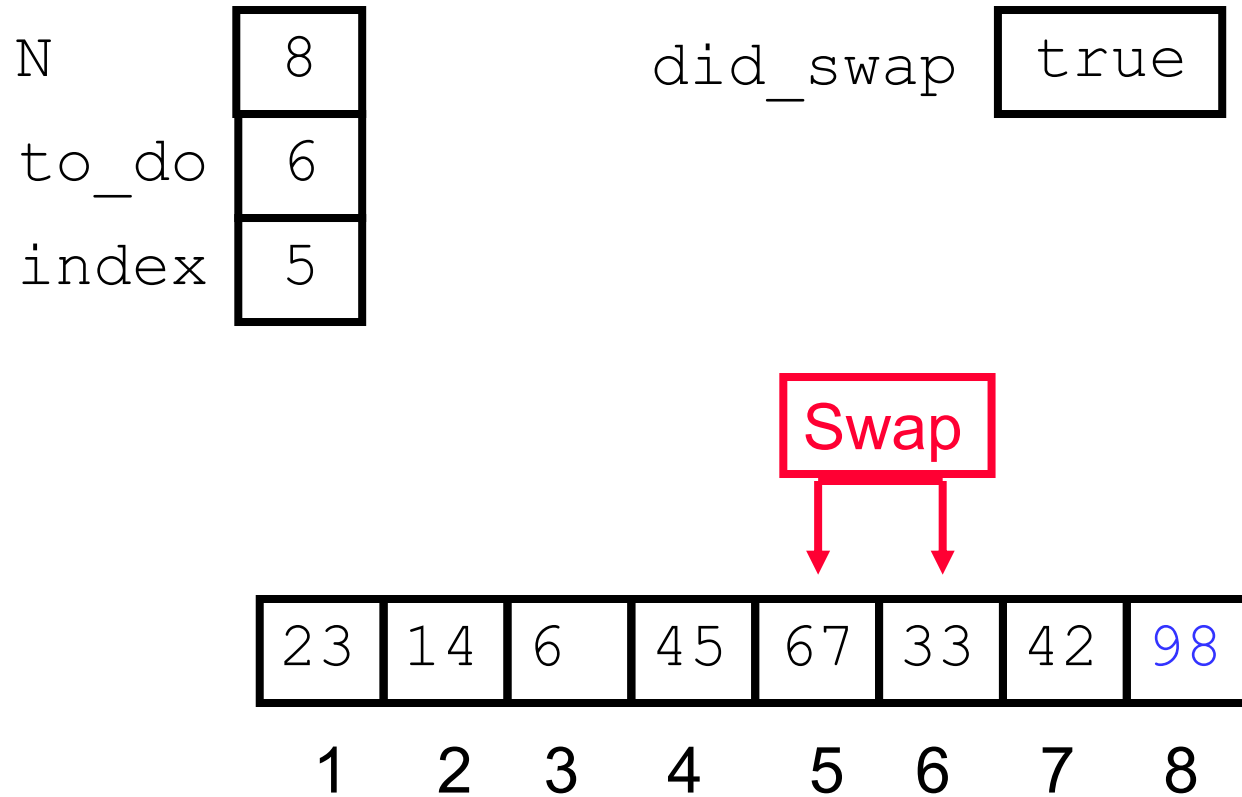
The Second “Bubble Up”



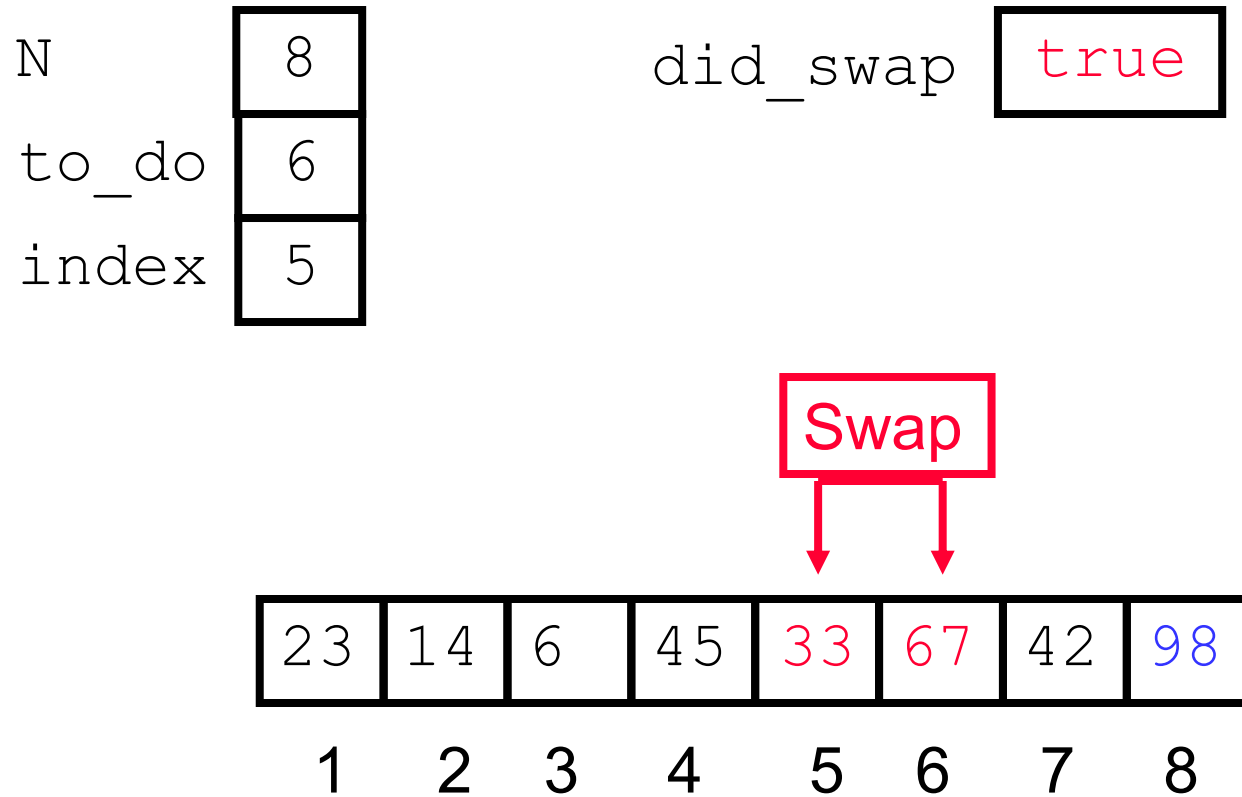
The Second “Bubble Up”



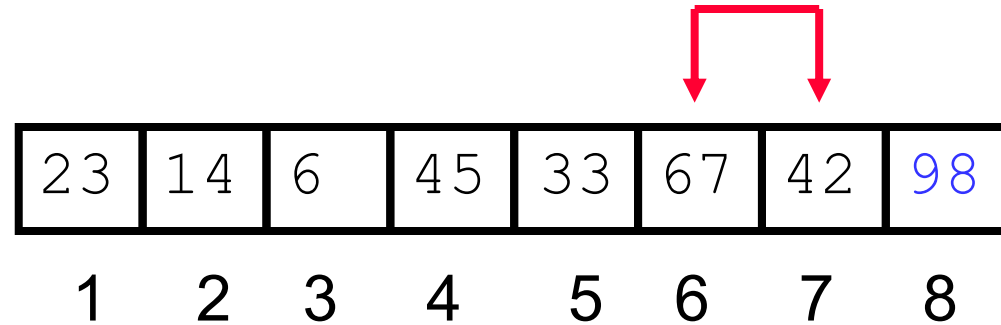
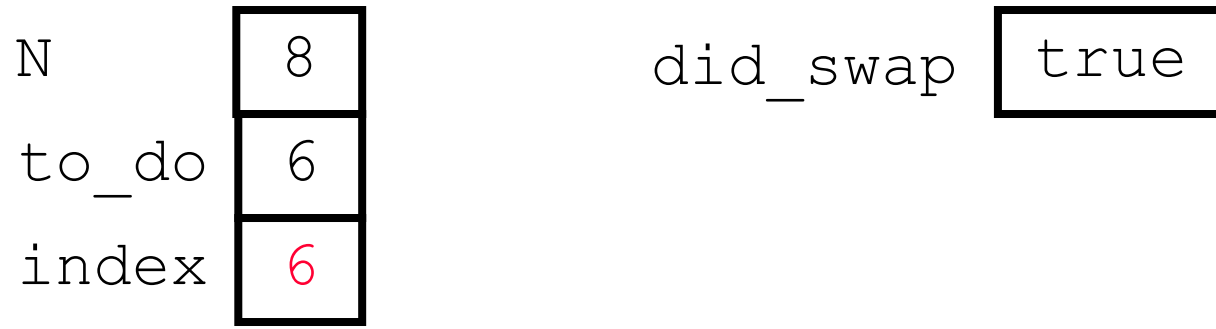
The Second “Bubble Up”



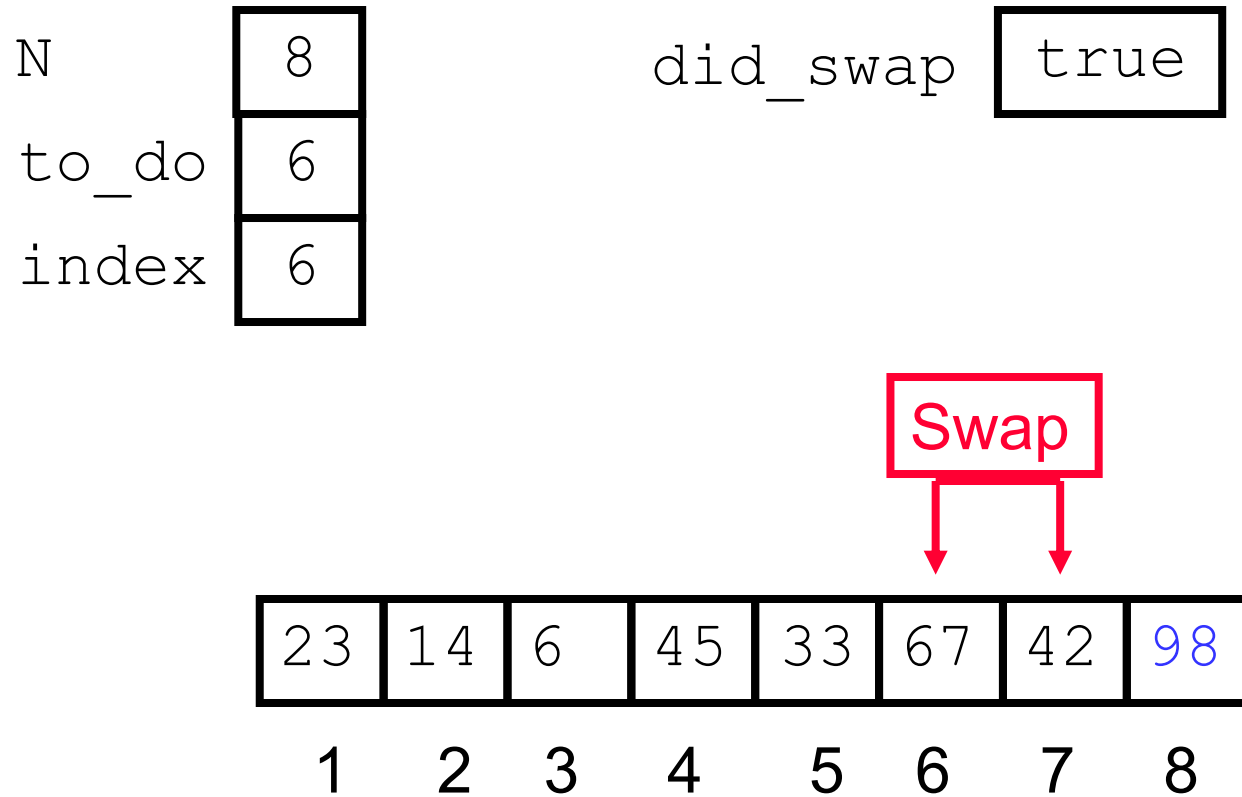
The Second “Bubble Up”



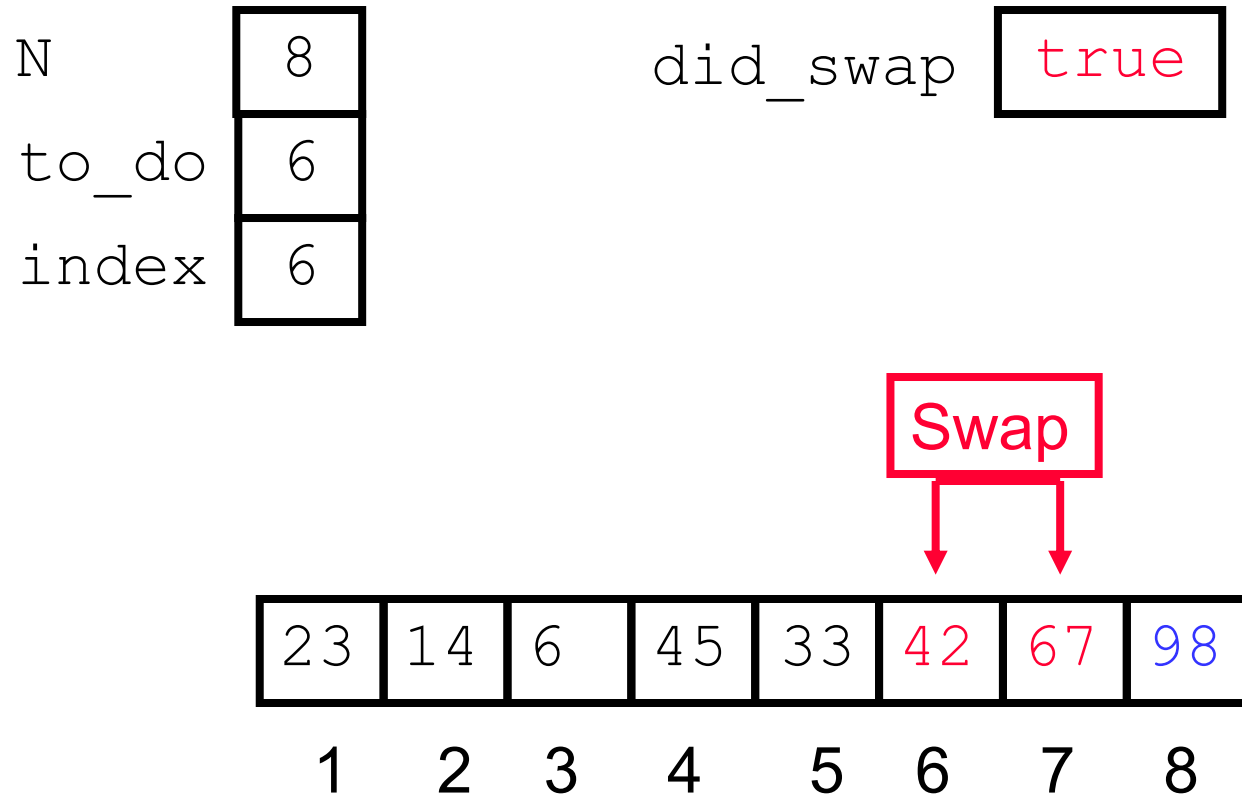
The Second “Bubble Up”



The Second “Bubble Up”



The Second “Bubble Up”



After Second Pass of Outer Loop

N

8

to_do

6

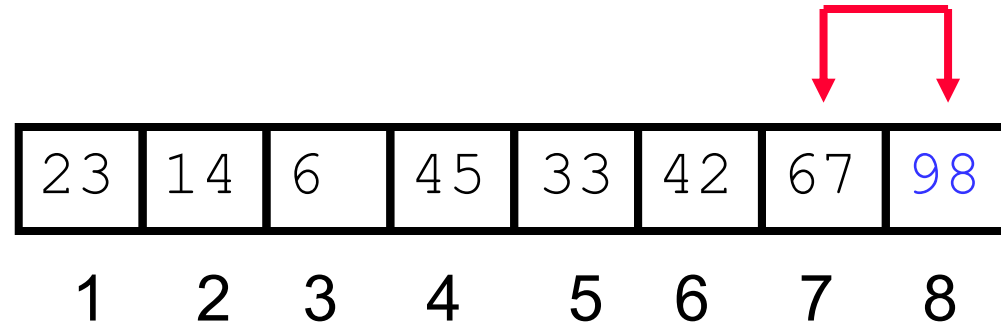
index

7

did_swap

true

Finished second "Bubble Up"



The Third “Bubble Up”

N

8

 did_swap

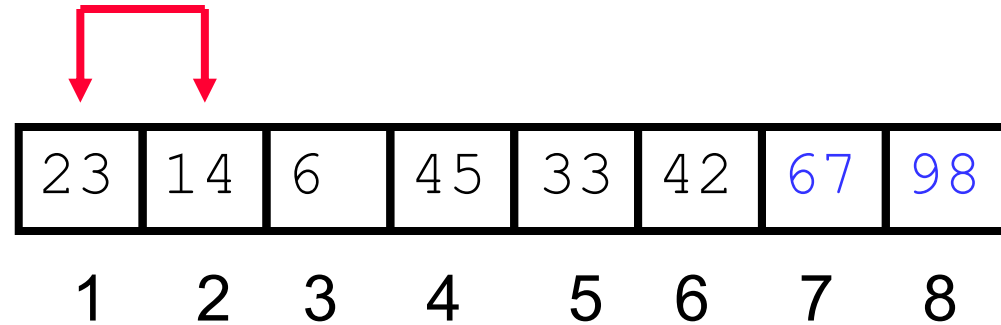
false

to_do

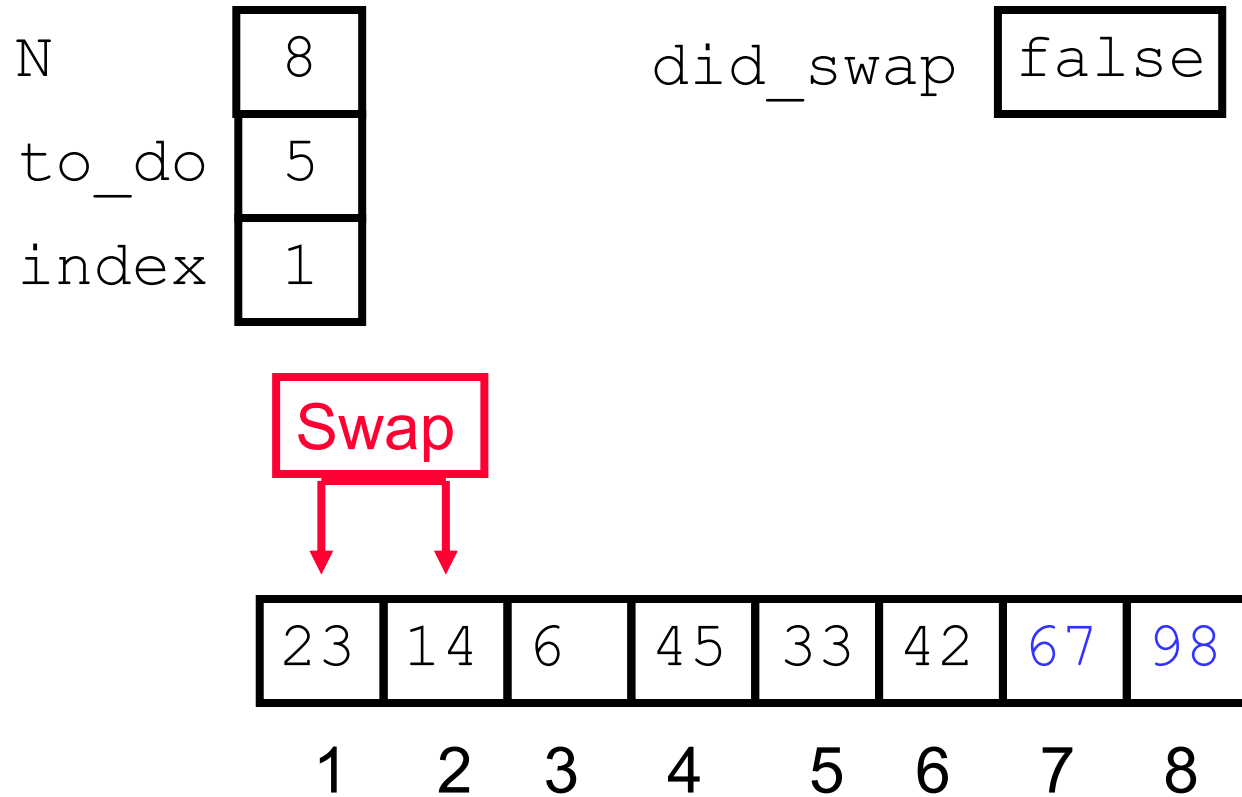
5

index

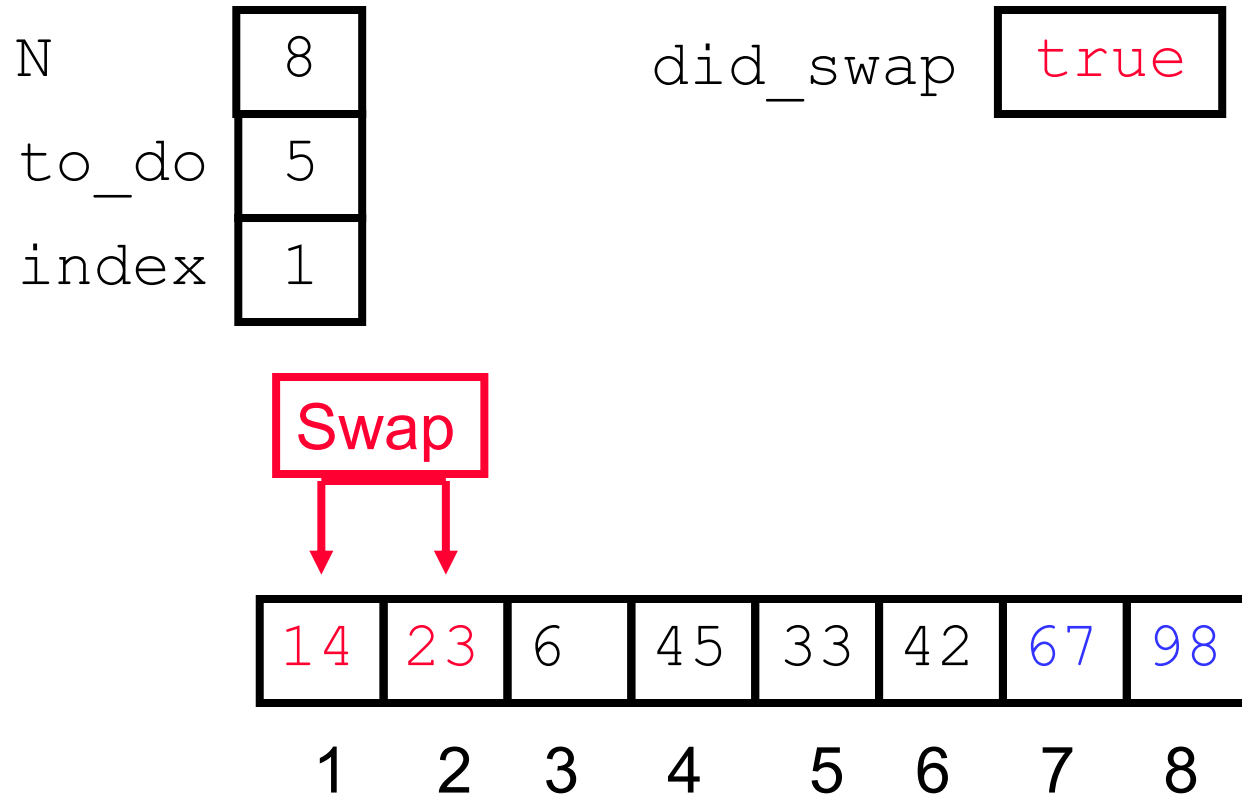
1



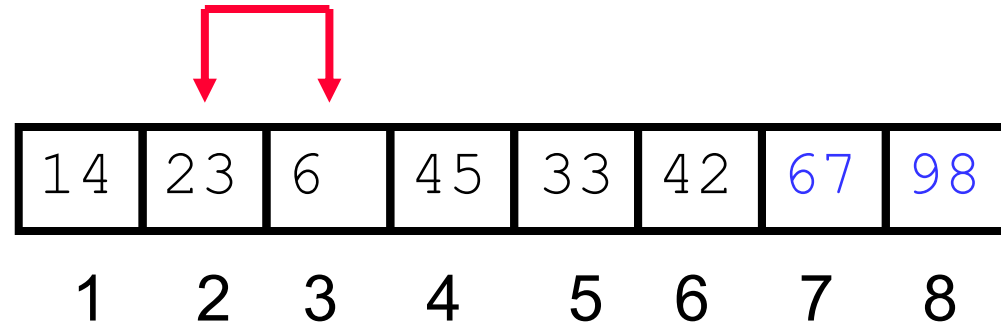
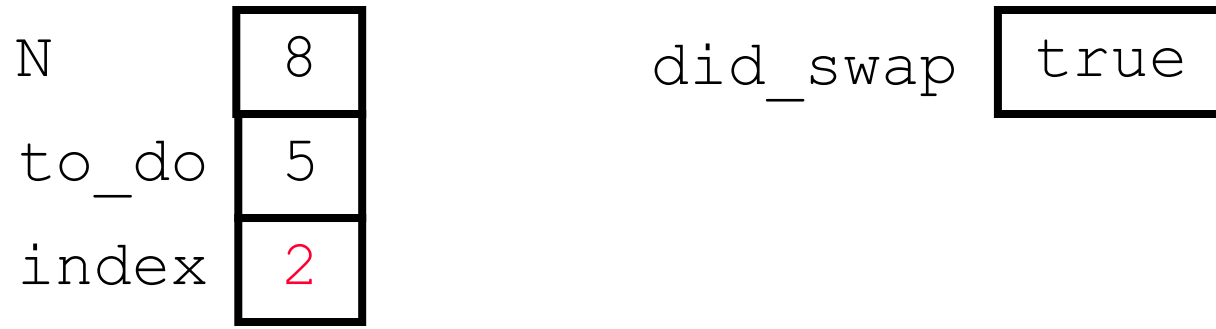
The Third “Bubble Up”



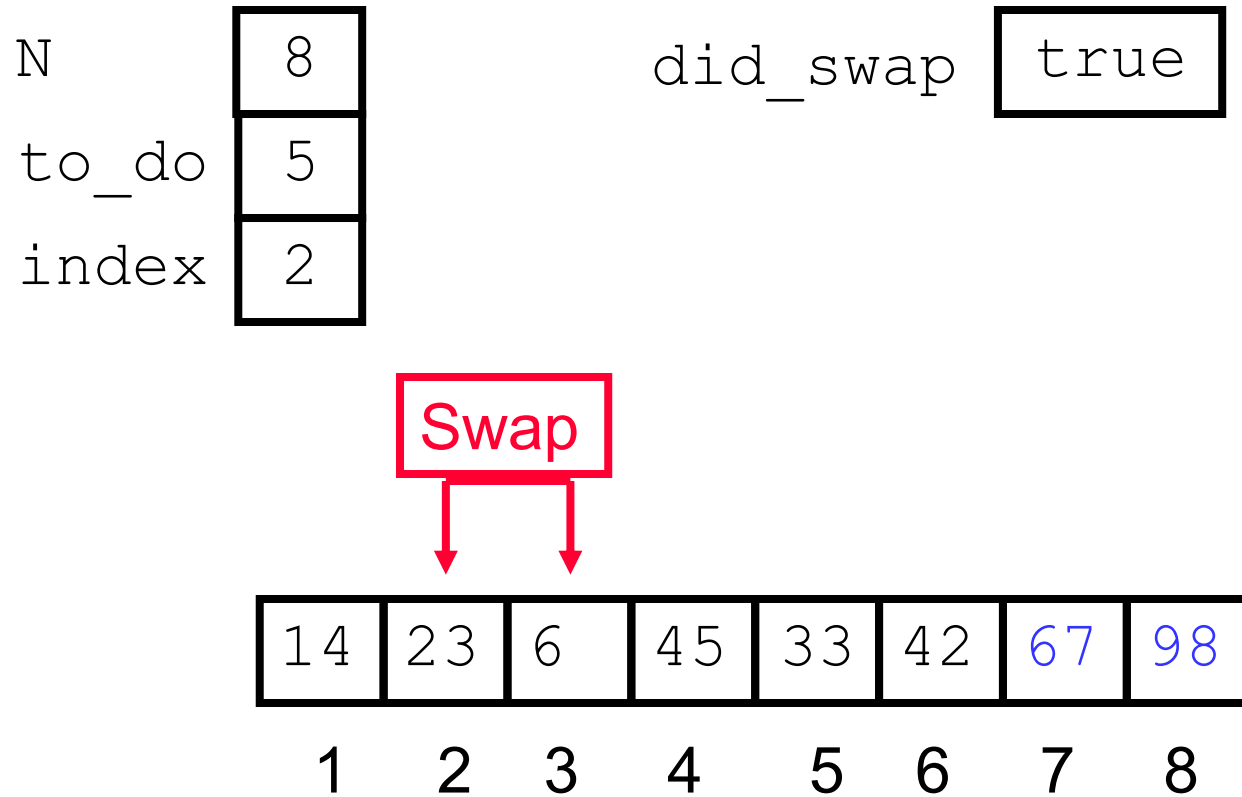
The Third “Bubble Up”



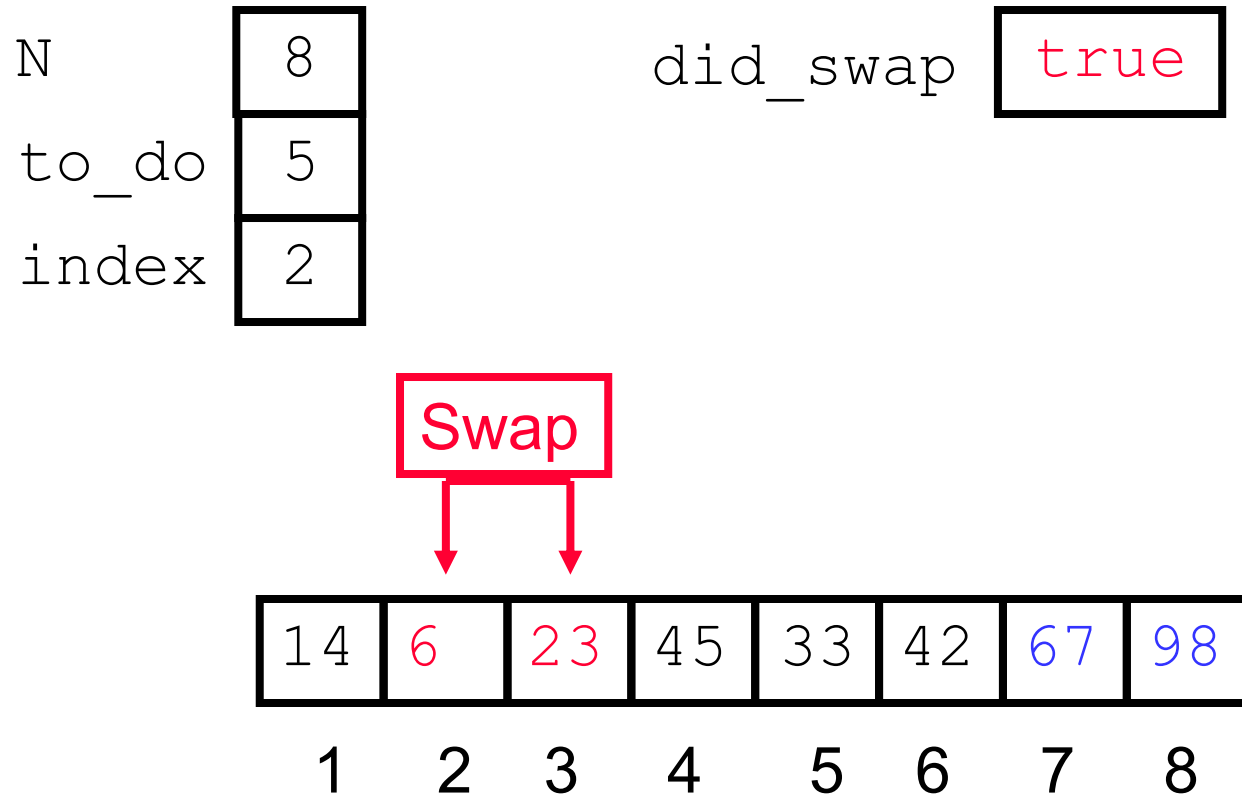
The Third “Bubble Up”



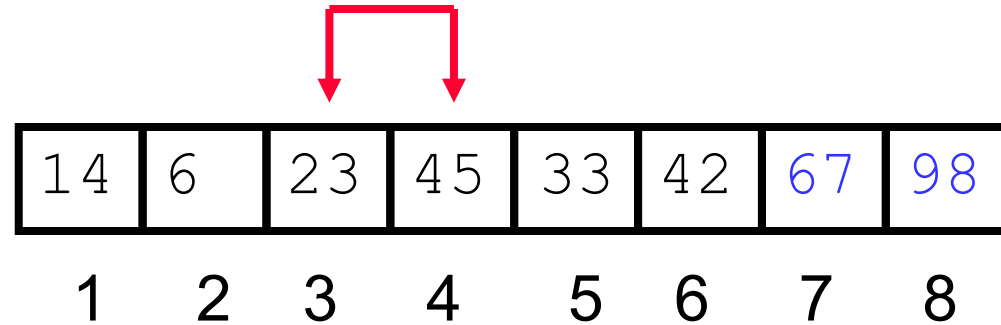
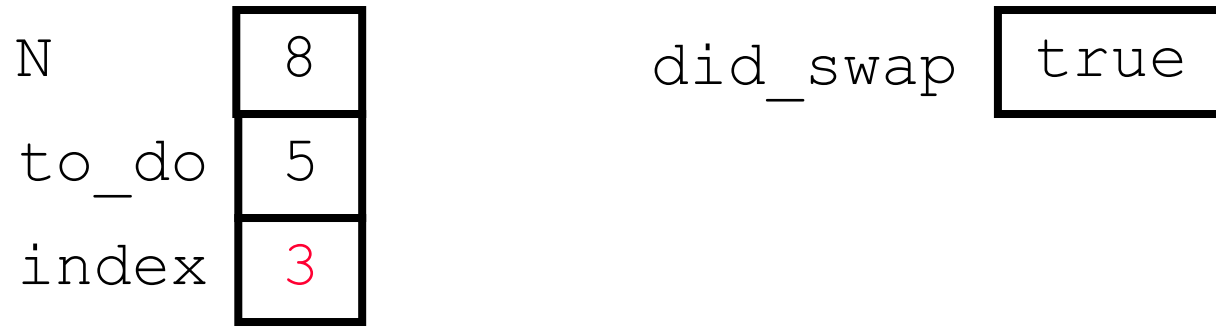
The Third “Bubble Up”



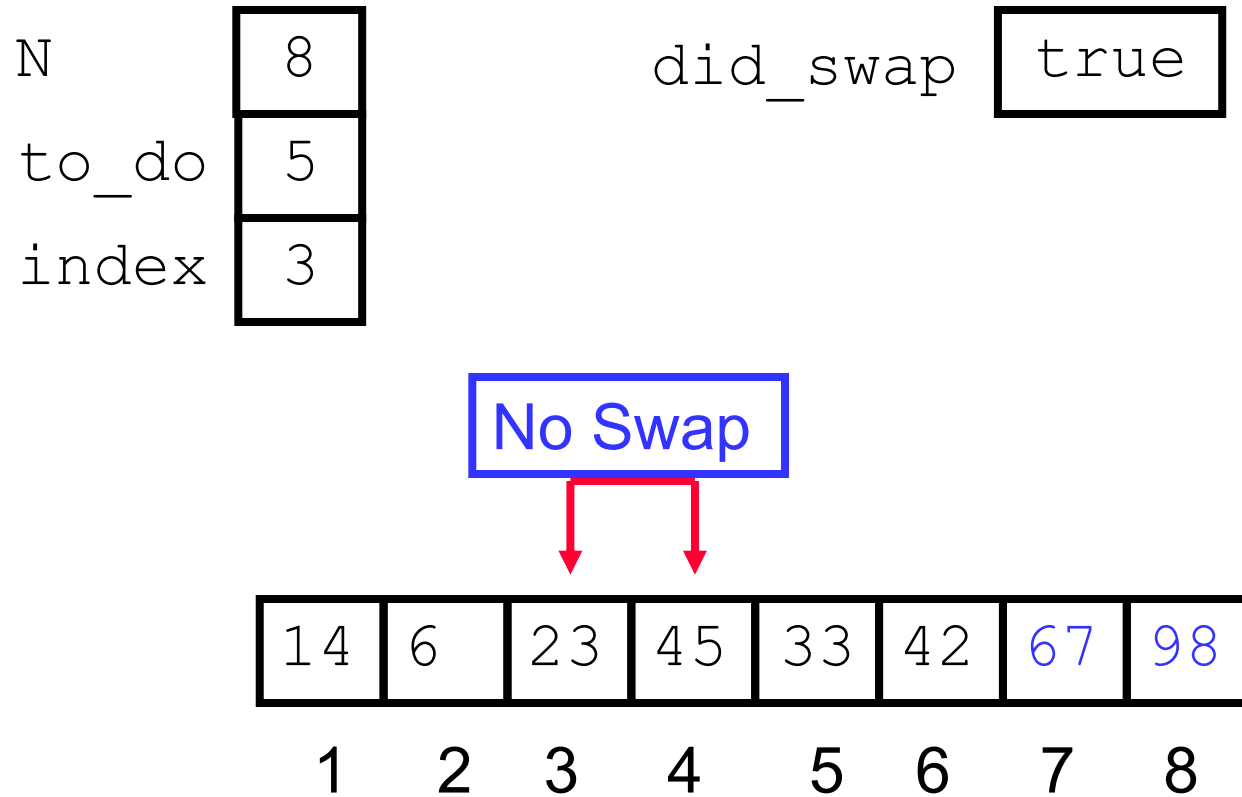
The Third “Bubble Up”



The Third “Bubble Up”



The Third “Bubble Up”



The Third “Bubble Up”

N

8

 did_swap

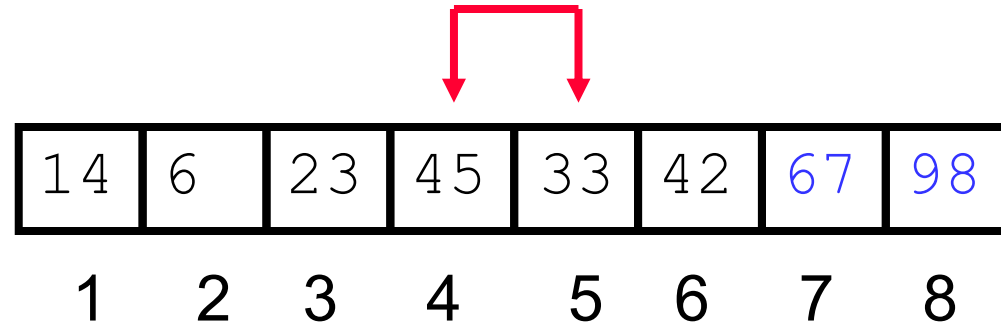
true

to_do

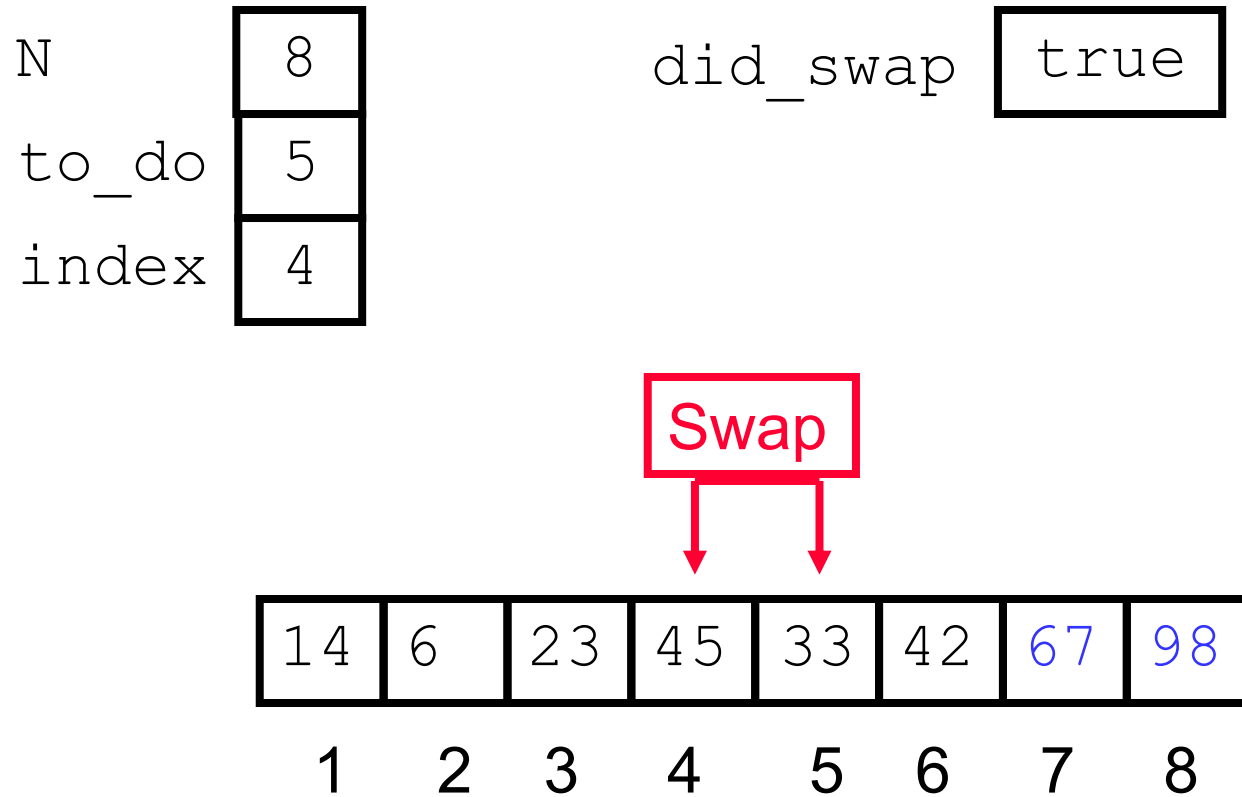
5

index

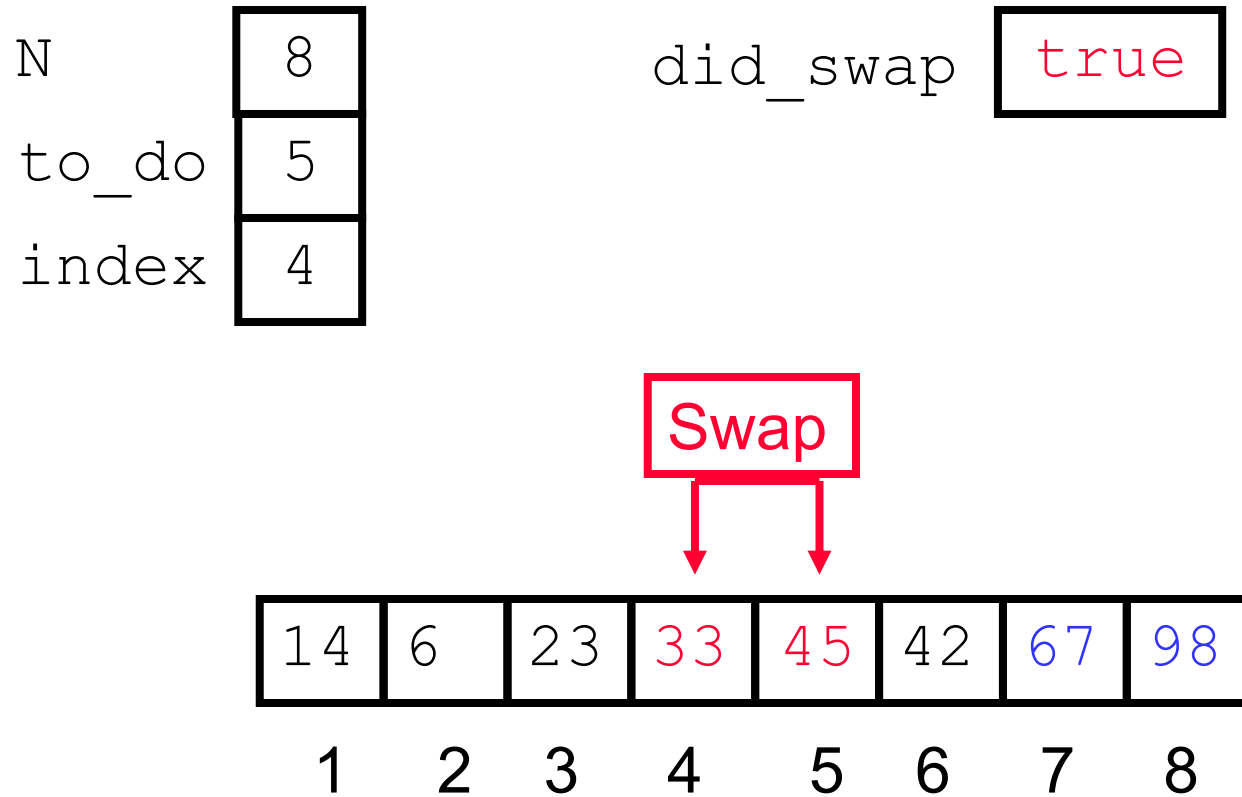
4



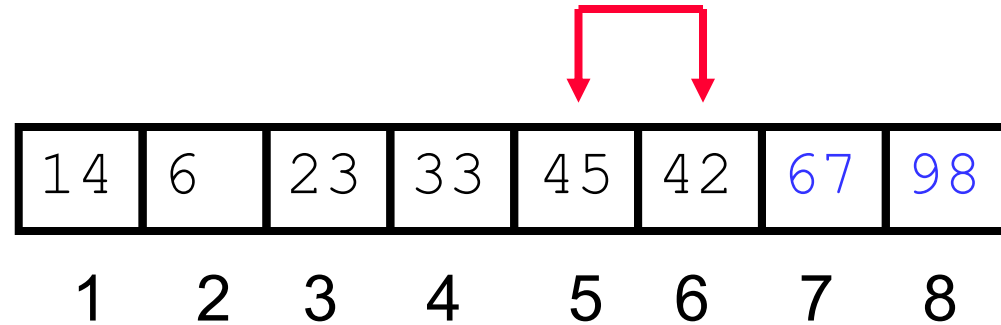
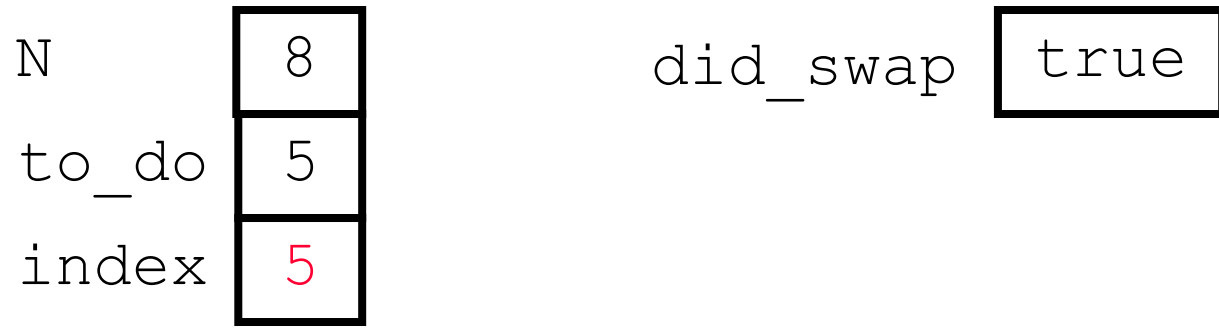
The Third “Bubble Up”



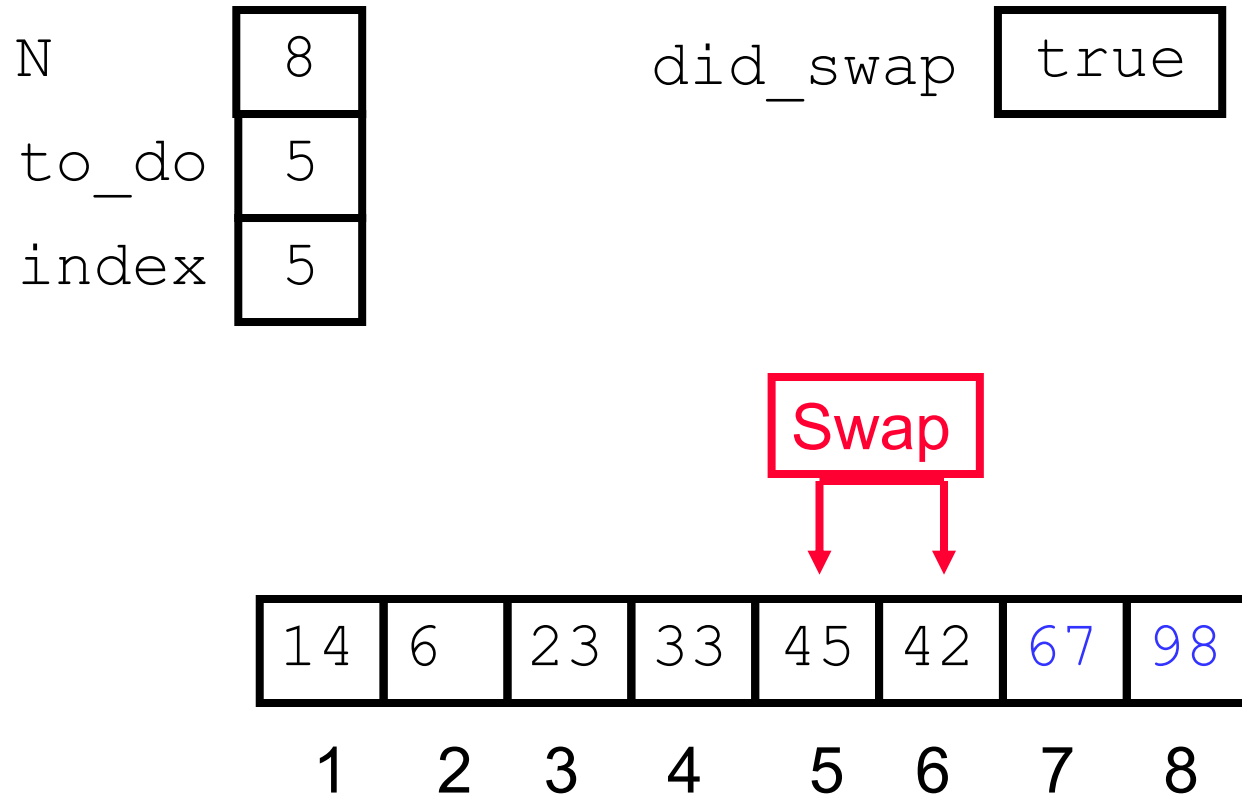
The Third “Bubble Up”



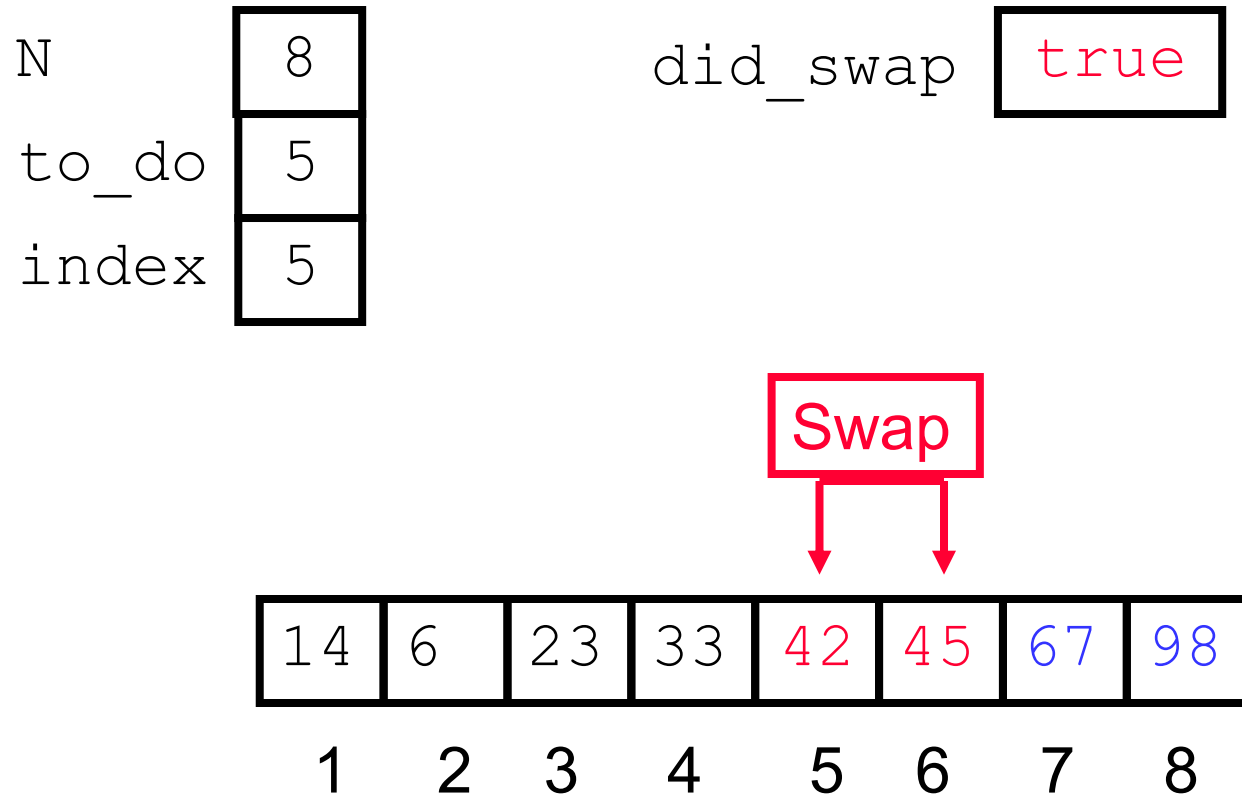
The Third “Bubble Up”



The Third “Bubble Up”



The Third “Bubble Up”



After Third Pass of Outer Loop

N

8

to_do

5

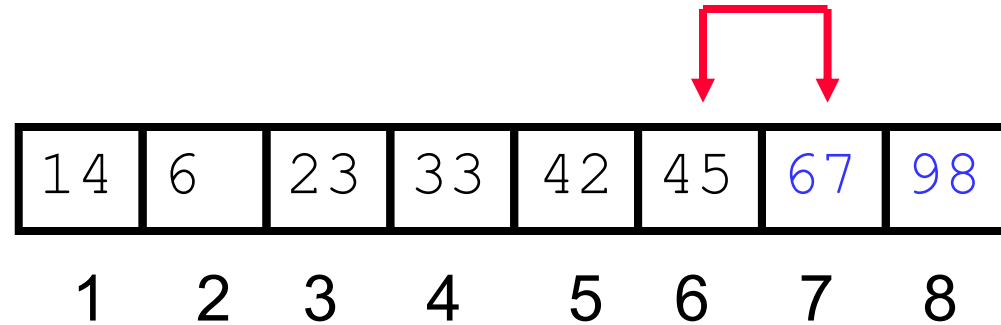
index

6

did_swap

true

Finished third "Bubble Up"



The Fourth “Bubble Up”

N

8

 did_swap

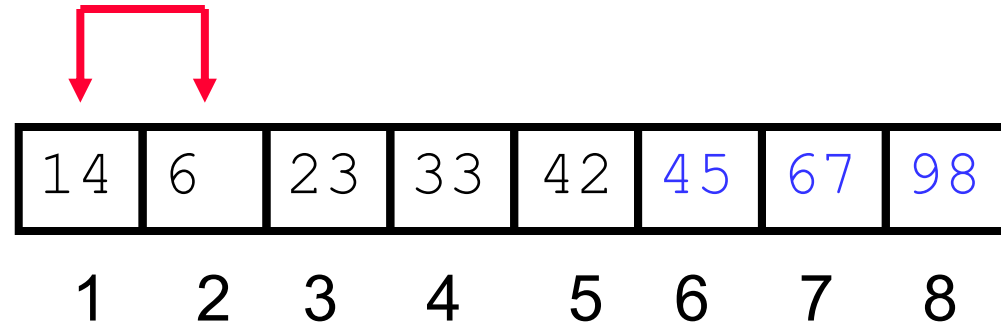
false

to_do

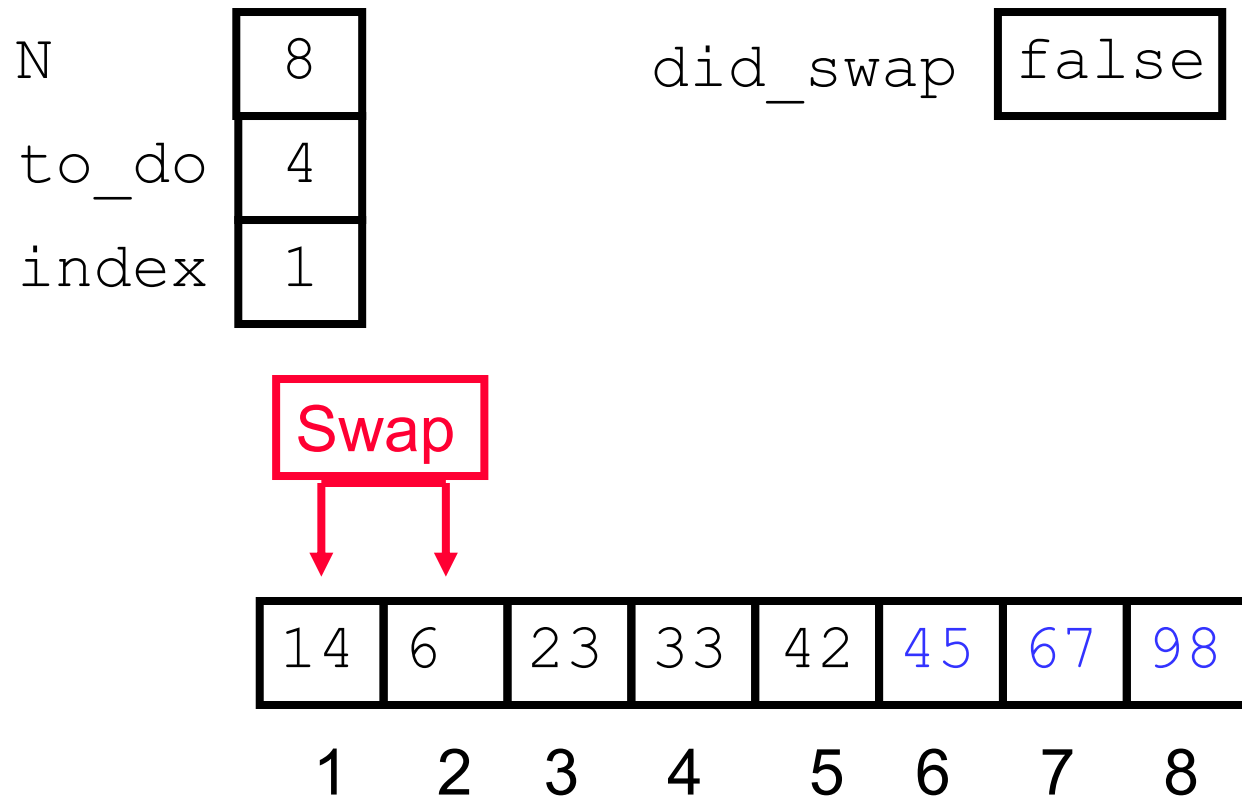
4

index

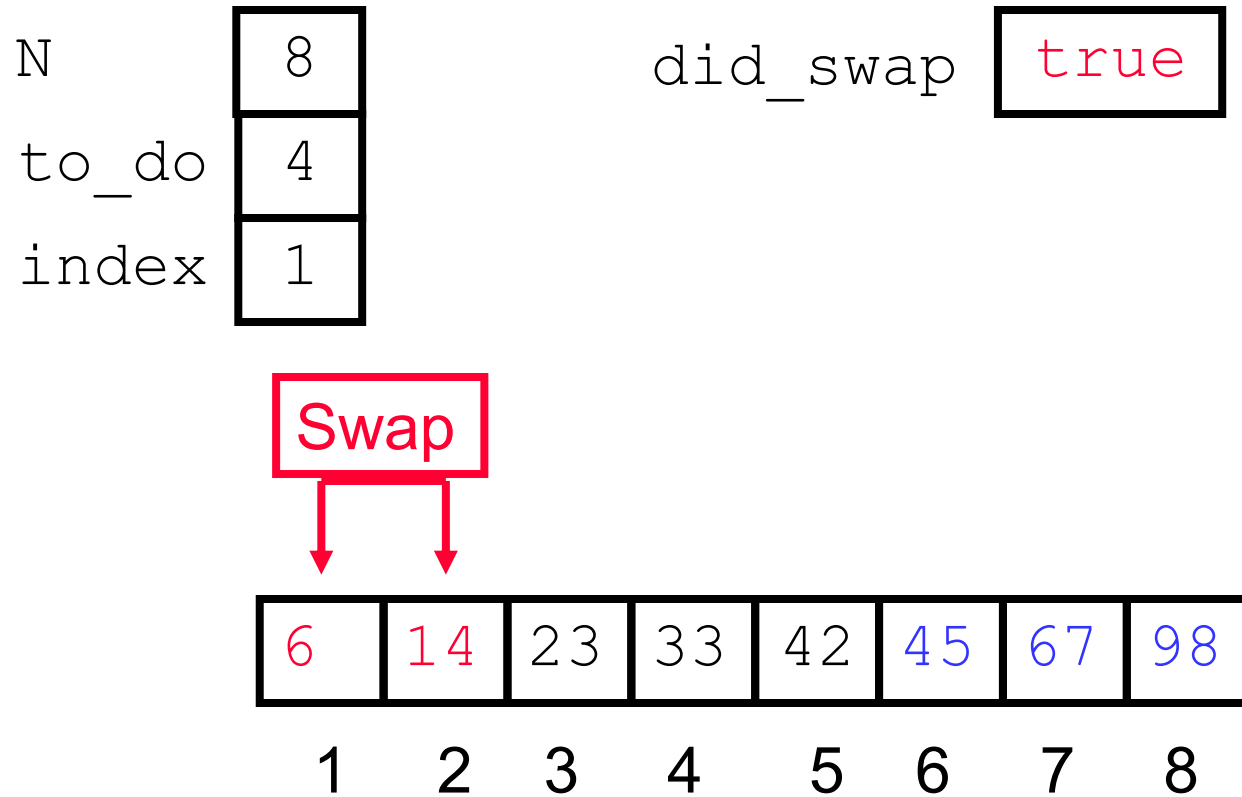
1



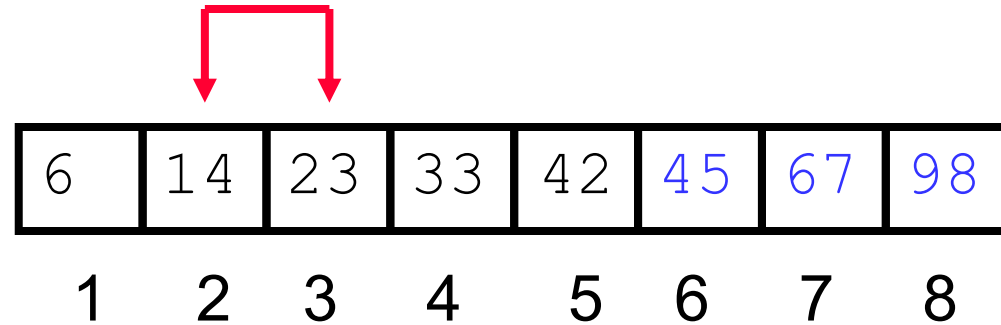
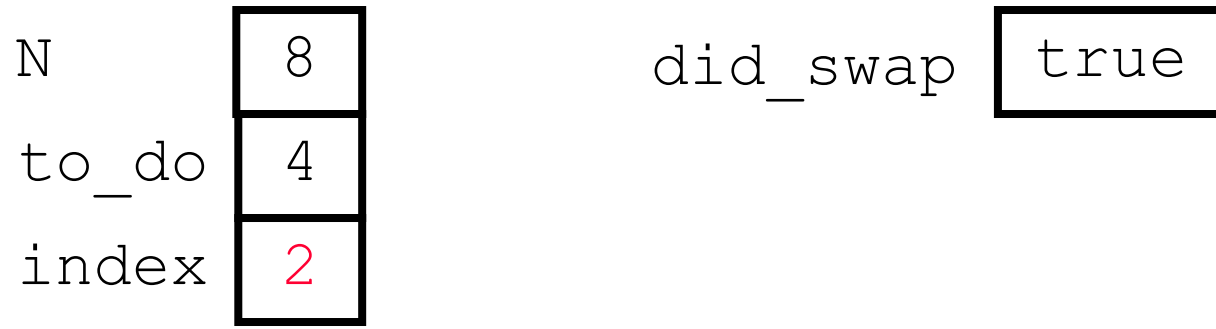
The Fourth “Bubble Up”



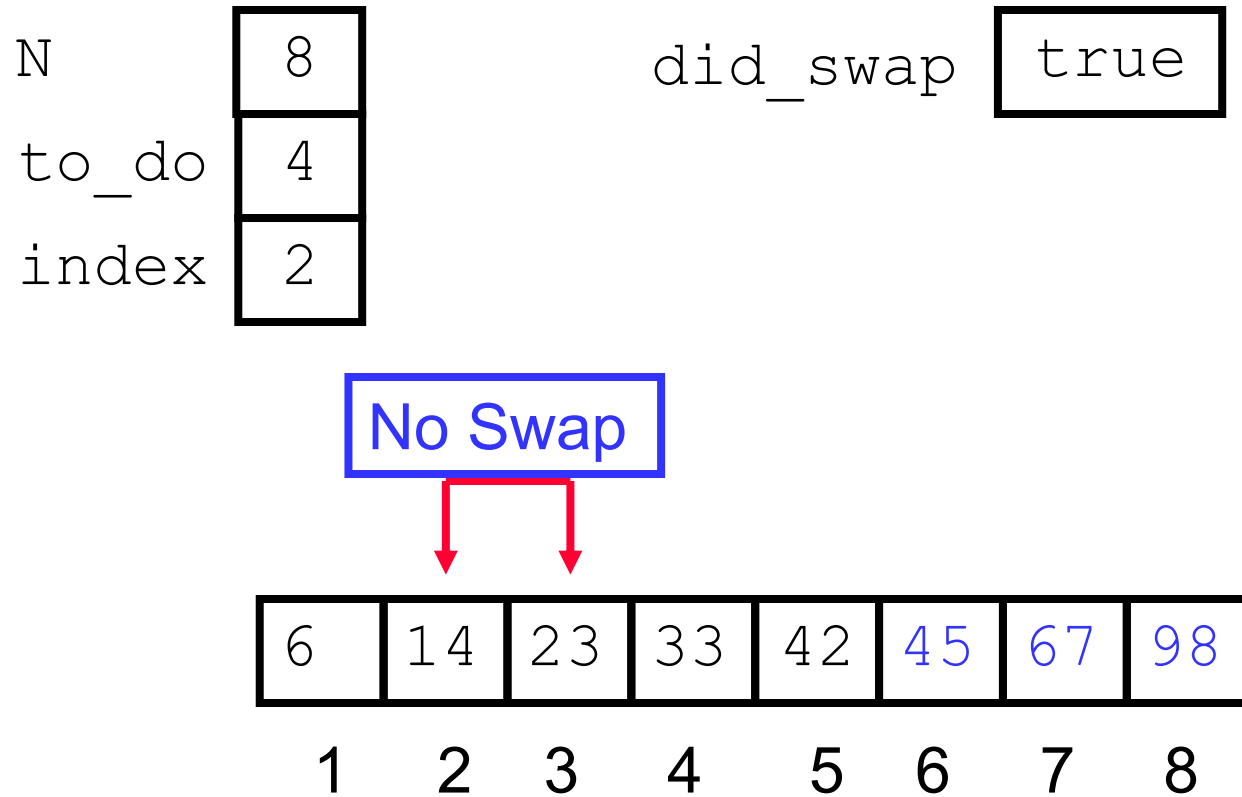
The Fourth “Bubble Up”



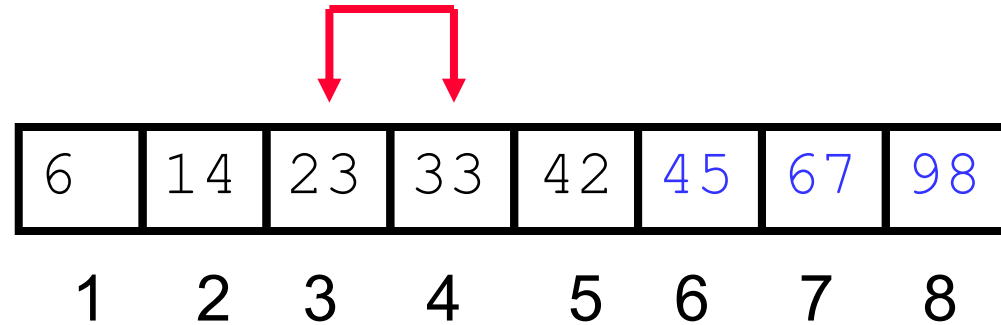
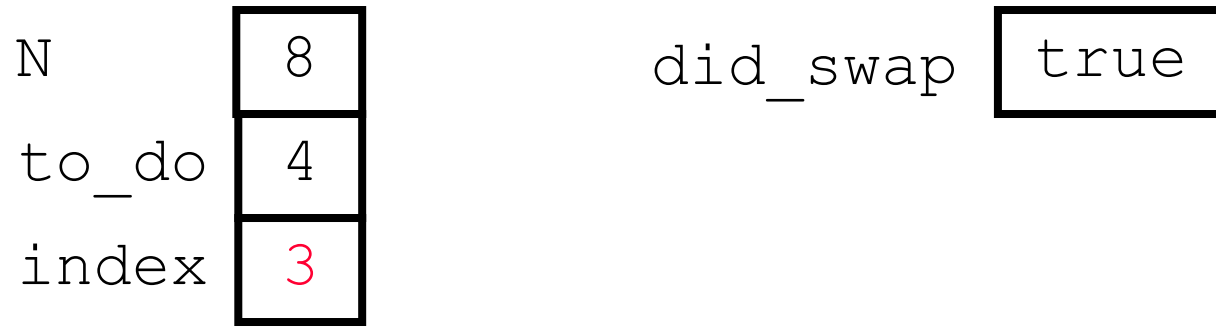
The Fourth “Bubble Up”



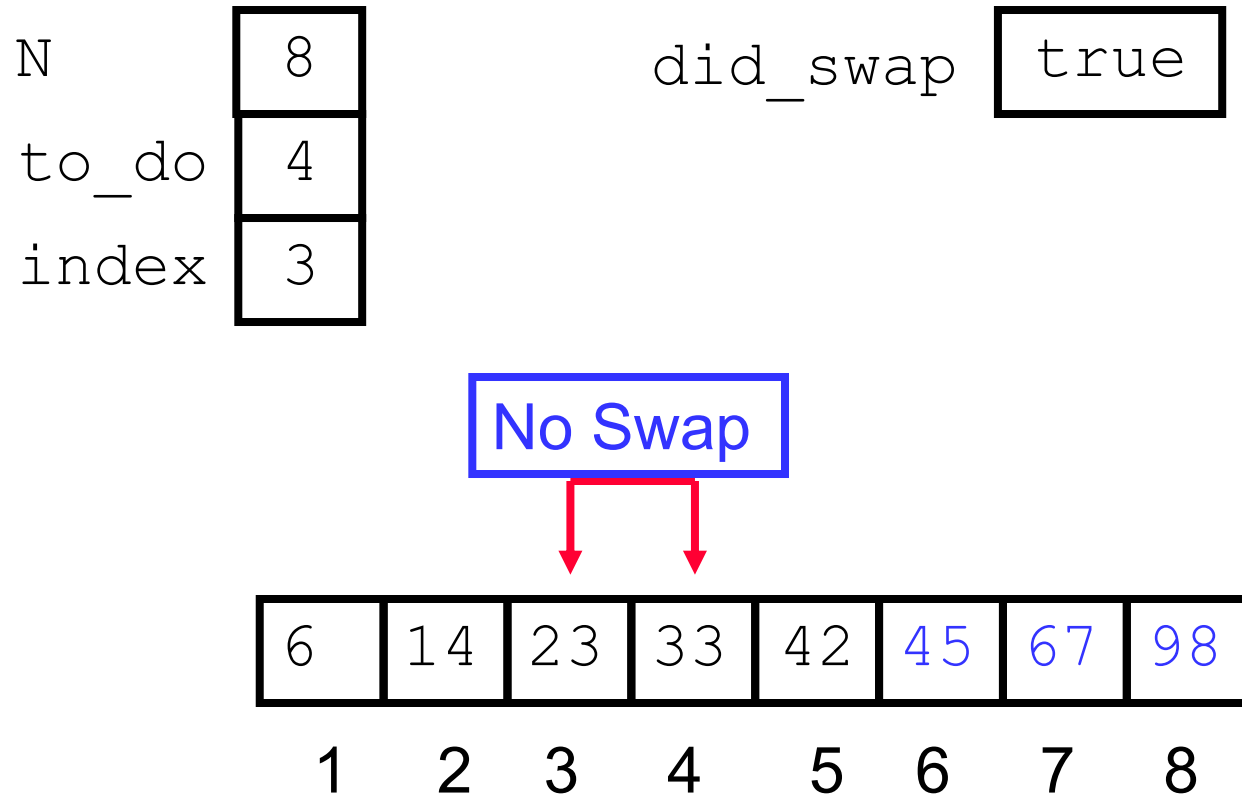
The Fourth “Bubble Up”



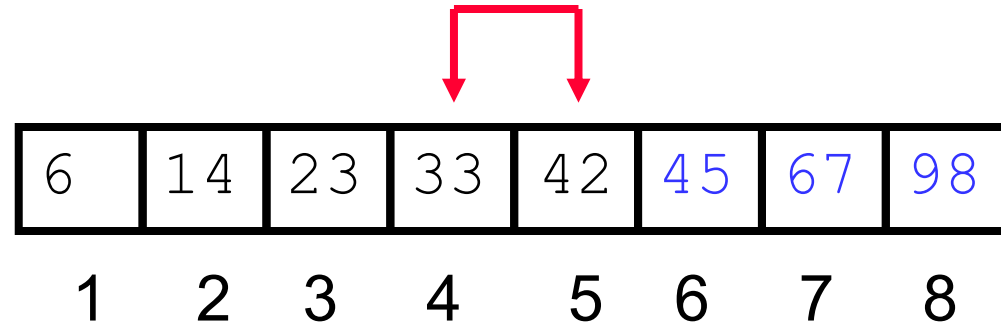
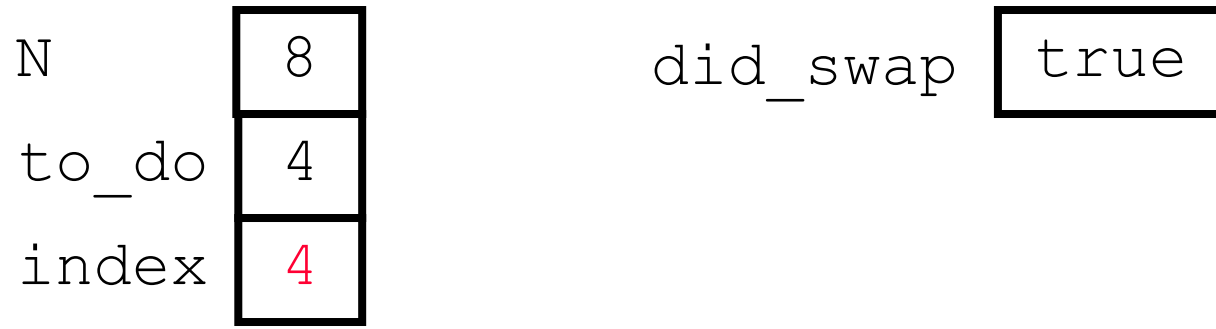
The Fourth “Bubble Up”



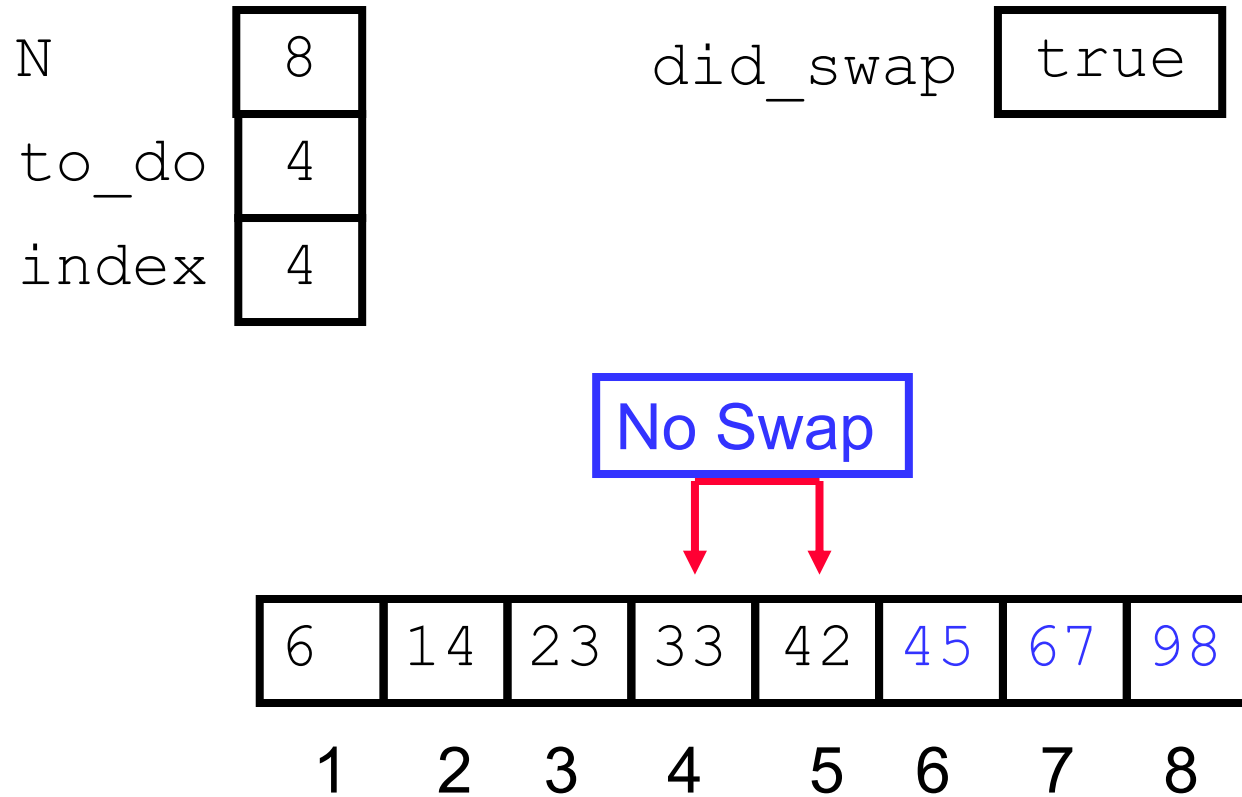
The Fourth “Bubble Up”



The Fourth “Bubble Up”



The Fourth “Bubble Up”



After Fourth Pass of Outer Loop

N

8

to_do

4

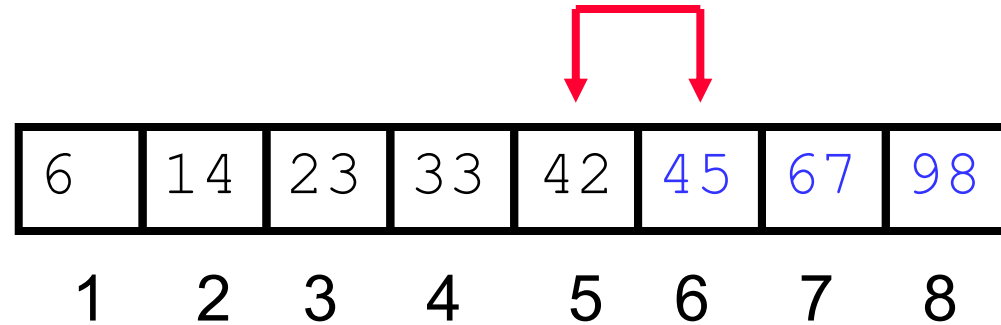
index

5

did_swap

true

Finished fourth "Bubble Up"



The Fifth “Bubble Up”

N

8

 did_swap

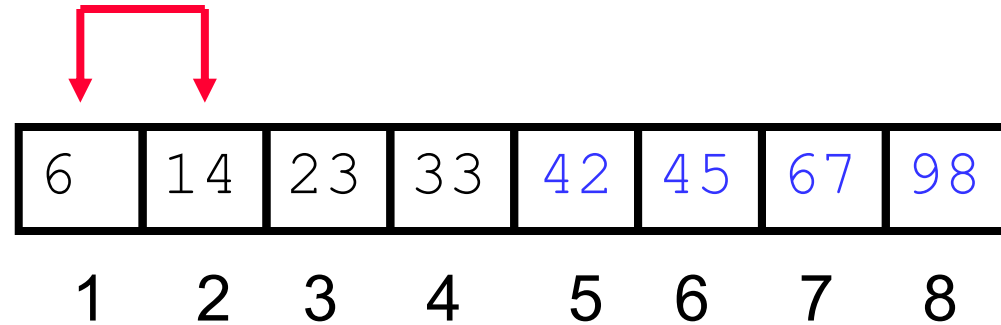
false

to_do

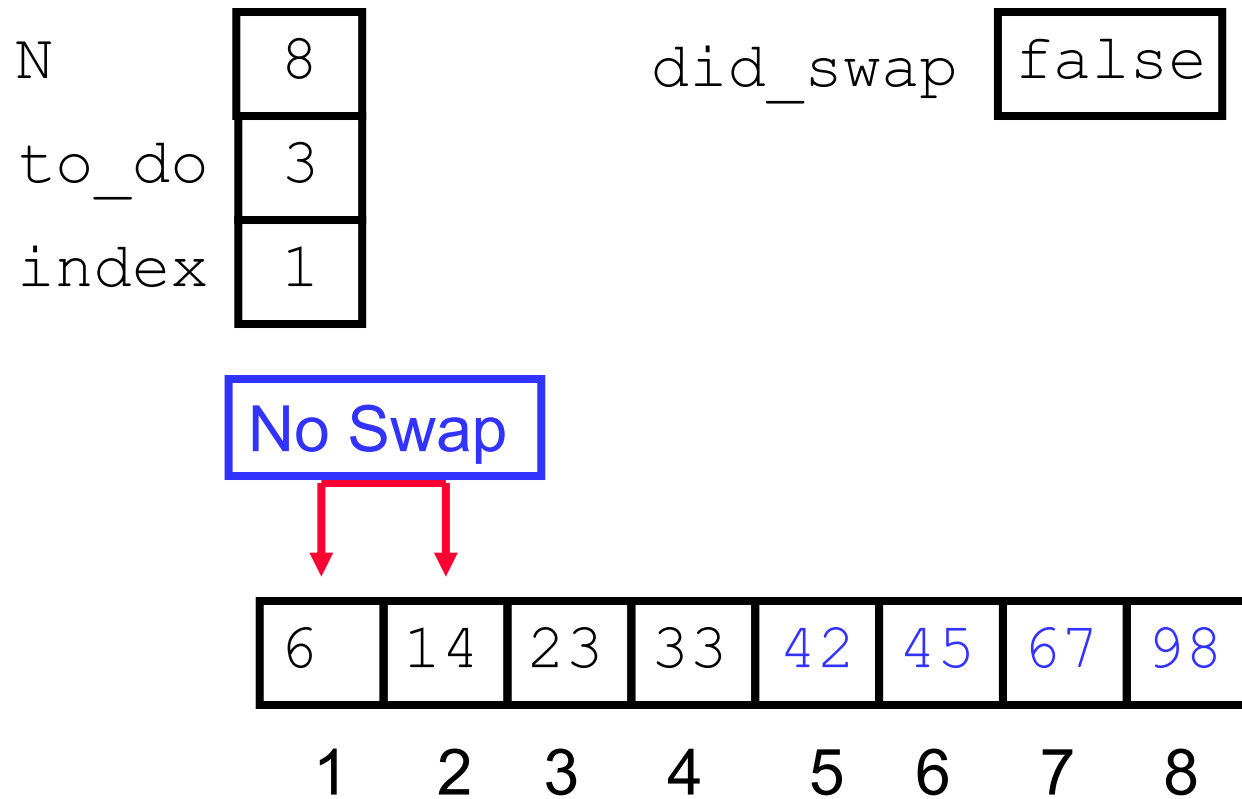
3

index

1



The Fifth “Bubble Up”



The Fifth “Bubble Up”

N

8

 did_swap

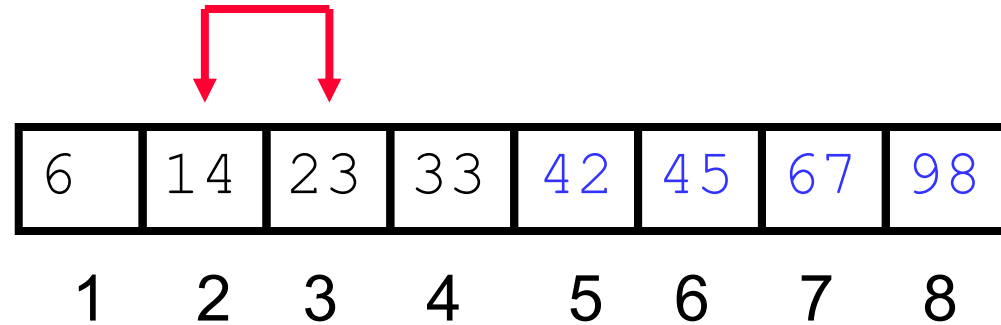
false

to_do

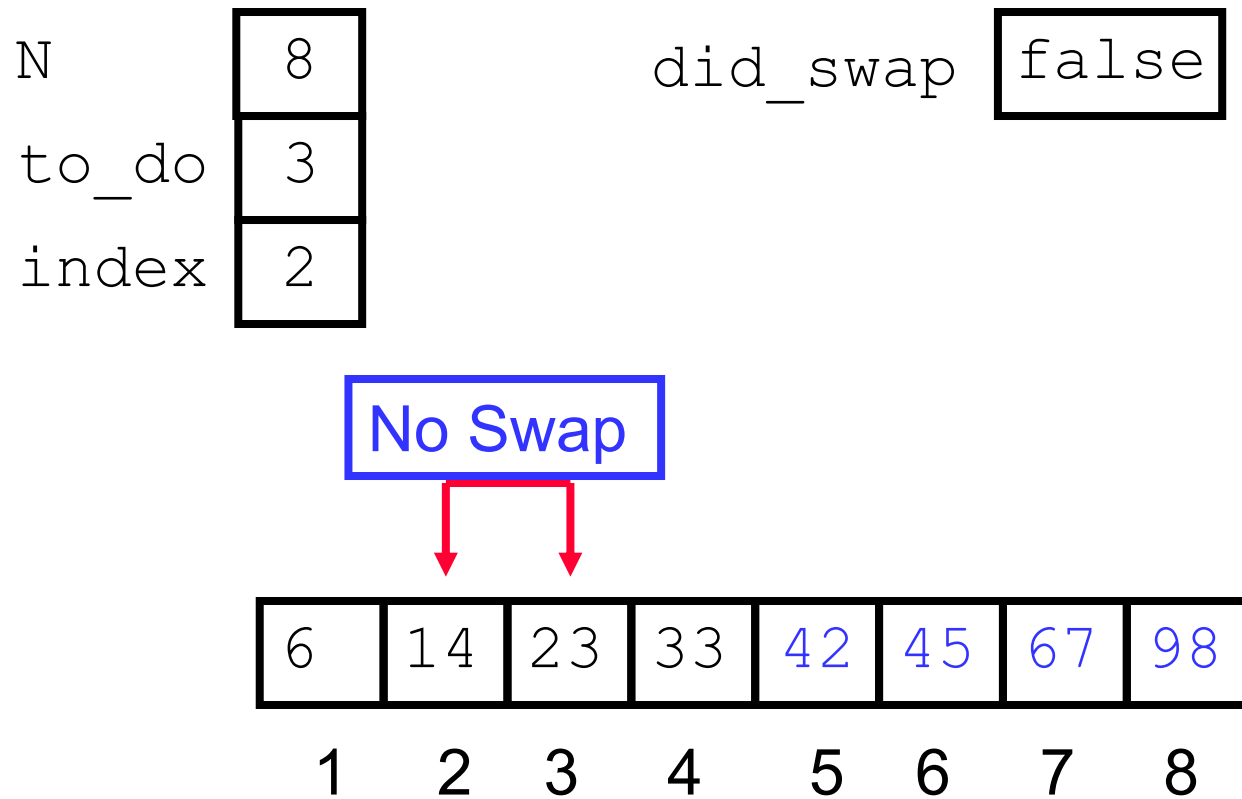
3

index

2



The Fifth “Bubble Up”



The Fifth “Bubble Up”

N

8

 did_swap

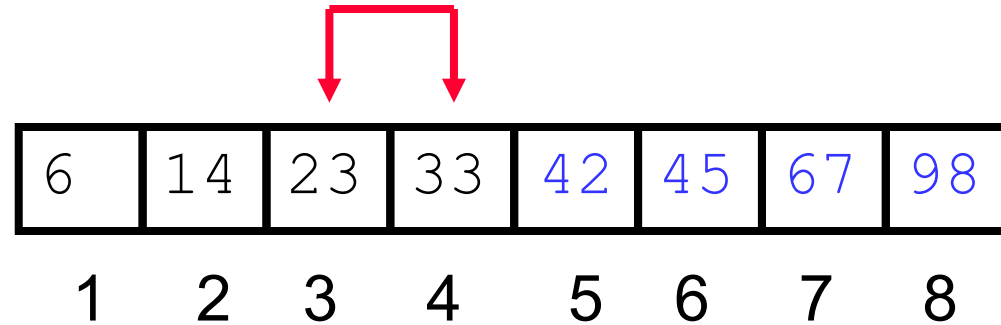
false

to_do

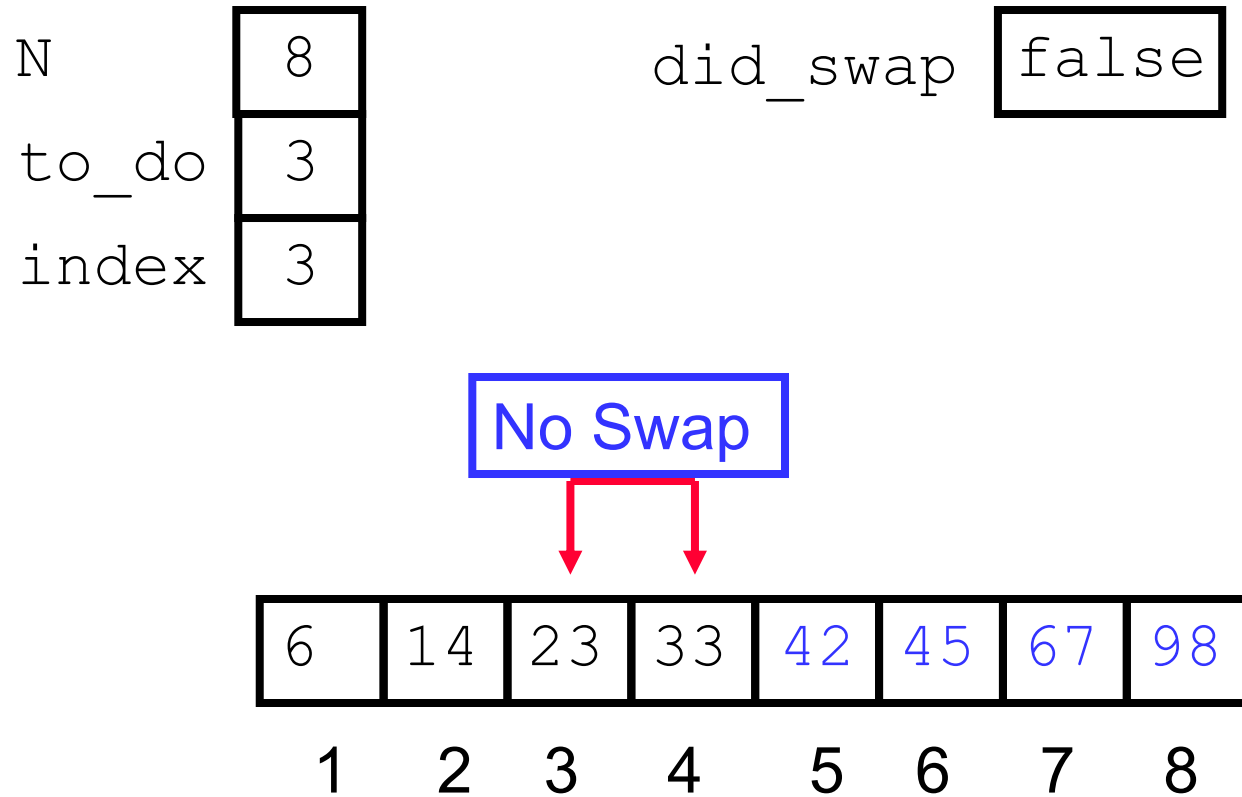
3

index

3



The Fifth “Bubble Up”



After Fifth Pass of Outer Loop

N

8

to_do

3

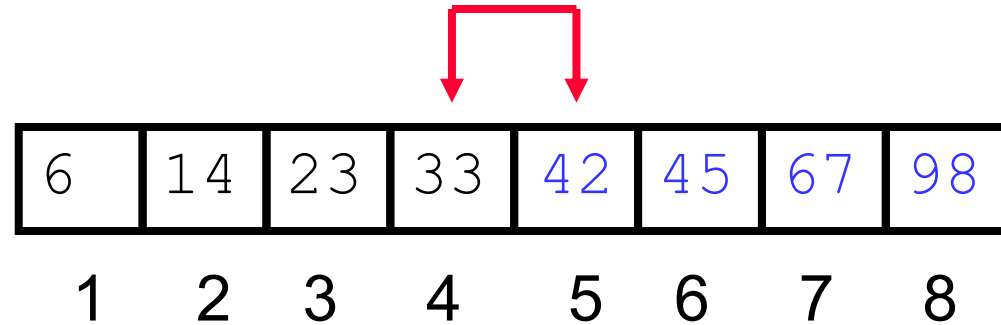
index

4

did_swap

false

Finished fifth "Bubble Up"



Finished “Early”

N	8
to_do	3
index	4

did_swap false

We didn't do any swapping,
so all of the other elements
must be correctly placed.

We can “skip” the last two
passes of the outer loop.

6	14	23	33	42	45	67	98
1	2	3	4	5	6	7	8

Summary

- “Bubble Up” algorithm will **move largest value to its correct location** (to the right)
- Repeat “Bubble Up” until all elements are correctly placed:
 - **Maximum of $N-1$ times**
 - Can finish early if **no swapping** occurs
- We reduce the number of elements we compare each time one is correctly placed

STRINGS

STRINGS

- Strings are sequence of characters.
- In C, there is no built-in data type for strings.
- String can be represented as a **one-dimensional array of characters**.
- String should always have a **NULL** character ('\0') at the end, to represent the **end of string**.
- String constants can be assigned to character array variables.
- String constants are always enclosed within double quotes and character constants are enclosed within single quotes.

Declaration and initialization of string variables:

The following declaration and initialization create a string consisting of the word "Hello".

```
char str[6] = {'H','e','l','l','o','\0'};
```

Other examples:

- `char c[4]={‘s’,‘u’,‘m’,‘\0’};`
- `char str[16]="qwerty";` Creates a string. The value at `str[5]` is the character ‘y’. The value at `str[6]` is the null character. The values from `str[7]` to `str[15]` are undefined.

`char name[5];`

`int main( )`

`{`

`name[0] = ‘G’;`

`name[1] = ‘O’;`

`name[2] = ‘O’;`

`name[3] = ‘D’;`

`name[4] = ‘\0’;`

`return 0;`

`}`

Example:

char c[20]; // Here c represents array of 20 characters and it is also called as string variable.

scanf("%s", c);

printf("%s",c);

String Manipulation Functions

C does not provide any operator, which manipulates the entire string at once. Strings are manipulated either using pointers or special routines (built-in) available from the standard string library **string.h**.

Following is the list of string functions available in **string.h**:

`strcpy(string1, string2)` Copy string2 into string1

`strcat(string1, string2)` Concatenate string2 onto the end of string1

`strcmp(string1, string2)` Lexically compares the two input strings

(ASCII comparison)

returns 0 if string1 is equal to string2

< 0 if string1 is less than string2

> 0 if string1 is greater than string2

- `strlen (string)` Gives the length of a string
- `strrev (string)` Reverse the string and result is stored in same string.
- `strncat(string1, string2, n)` Append n characters from string2 to string1
- `strncmp(string1, string2, n)` Compare first n characters of two strings.
- `strncpy(string1,string2, n)` Copy first n characters of string2 to string1
- `strupr (string)` Converts string to uppercase
- `strlwr (string)` Converts a string to lowercase

Examples

strcpy(string1, string2)

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char string1[50];
```

```
    char string2[] = "Hello, World!";
```

```
    strcpy(string1, string2);
```

```
    printf("After strcpy: string1 = %s\n", string1);
```

```
    return 0;
```

```
}
```

Examples

strcat(string1, string2)

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char string1[50] = "Hello";
```

```
    char string2[] = "World!";
```

```
    strcat(string1, string2);
```

```
    printf("After strcat: string1 = %s\n", string1);
```

```
    return 0;
```

```
}
```

Examples

strcmp(string1, string2)

```
#include <stdio.h>
#include <string.h>

int main() {
    char string1[] = "Hello";
    char string2[] = "World";

    int result = strcmp(string1, string2);
    if (result == 0) {
        printf("Strings are equal.\n");
    } else if (result < 0) {
        printf("string1 is less than string2.\n");
    } else {
        printf("string1 is greater than string2.\n");
    }

    return 0;
```

Examples

strlen(string1)

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char string[] = "Hello, World!";
    int length = strlen(string);
    printf("The length of the string is: %d\n", length);

    return 0;
}
```

Examples

strrev(string1)

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char string[] = "Hello, World!";  
    strrev(string);  
    printf("Reversed string: %s\n", string);  
  
    return 0;  
}
```

Examples

strncat(string1, string2,n)

```
#include <stdio.h>
#include <string.h>

int main() {
    char string1[50] = "Hello";
    char string2[] = ", World!";

    strncat(string1, string2, 5);
    printf("After strncat: string1 = %s\n", string1);

    return 0;
}
```

Examples

strncmp(string1, string2,n)

```
#include <stdio.h>
#include <string.h>

int main() {
    char string1[] = "Hello";
    char string2[] = "Helium";

    int result = strncmp(string1, string2, 3);
    if (result == 0) {
        printf("First 3 characters are equal.\n");
    } else if (result < 0) {
        printf("string1 is less than string2 in the first 3 characters.\n");
    } else {
        printf("string1 is greater than string2 in the first 3 characters.\n");
    }
    return 0;
}
```

Examples

strncpy(string1, string,n)

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char string1[50];
```

```
    char string2[] = "Hello, World!";
```

```
    strncpy(string1, string2, 5);
```

```
    string1[5] = '\0'; // Null-terminate the string
```

```
    printf("After strncpy: string1 = %s\n", string1);
```

```
    return 0;
```

```
}
```

String input and output functions

String input/read functions: **scanf()**, **getchar()** and **gets**

String output/write functions: **printf()**, **putchar()** and **puts**

Using scanf and printf function:

The scanf() can be used to read a string like any other data types.

Printf() is used to print or display a string on display screen.

Format:

scanf(<string format specifier>, string_variable/s);

printf(<string format specifier>, string_variable/s);

Here <string format specifier> is %s used within the "".

getchar() Function

- getchar() is used to read a single character from the standard input (keyboard).
- It waits for the user to input a character and then returns the ASCII value of the entered character.

```
#include <stdio.h>
int main() {
    char ch;
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: %c\n", ch);
    return 0;
}
```

putchar() Function

- putchar() is used to write a single character to the standard output (screen).
- It takes a character as an argument and prints it to the console.

```
#include <stdio.h>
```

```
int main() {  
    char ch = 'A';  
    printf("The character is: ");  
    putchar(ch);  
    printf("\n");  
    return 0;  
}
```

gets() Function

gets() reads a string from the standard input (keyboard) until a newline character is encountered.

It does not check the buffer limit, which can lead to buffer overflow, making it unsafe.

```
#include <stdio.h>
int main() {
    char str[100];
    printf("Enter a string: ");
    gets(str); // Reads a string from the user
    printf("You entered: %s\n", str);
    return 0;
}
```

puts() Function

puts() writes a string to the standard output (screen) and automatically appends a newline character at the end.

It is a simpler alternative to printf() for printing strings without formatting requirements.

```
#include <stdio.h>
int main() {
    char str[100] = "Hello, World!";
    printf("The string is: ");
    puts(str); // Prints the string followed by a newline
    return 0;
}
```

- `char name[5] = "INDIA";` - Strings are terminated by the null character, it is preferred to allocate one extra space to store null terminator.

Example for reading string using loop:

```
char name[20];  
  
int i=0;  
while((name[i] = getchar ()) != '\n' )  
    i++;  
scanf( "%s" , name);  
printf("%s" , name);
```

End of Unit 3

Thank You